

On the behavior of parallel island models[☆]

Lucas A. da Silveira^{a,*}, Thaynara A. de Lima^b, Jessé B. de Barros^c, José L. Soncco-Álvarez^d, Carlos H. Llanos^c, Mauricio Ayala-Rincón^{a,e,**}

^a Department of Computer Science, Universidade de Brasília, Brasília D.F., Brazil

^b Department of Mathematics, Universidade Federal de Goiás, Goiânia, Brazil

^c Department of Mechanical Engineering, Universidade de Brasília, Brasília D.F., Brazil

^d Department of Informatics, Universidad Nacional de San Antonio Abad del Cusco, Peru

^e Department of Mathematics, Universidade de Brasília, Brasília D.F., Brazil

ARTICLE INFO

Keywords:

Genetic algorithms
Parallel genetic algorithms
Parallel island models
Migration policies

ABSTRACT

Parallel island models (PIMs) are used to enhance the performance of evolutionary algorithms (EAs). In PIMs, each island executes an EA for evolving its local population, and periodically individuals migrate to neighborhoods synchronously or asynchronously. Neighborhoods are organized through a topology of communication, and migration policies guide individual exchange. This work explores migration policies over different communication topologies in synchronous and asynchronous PIMs to improve the speed-up and accuracy of genetic algorithms (GAs). The aim is to explain such models' adequacy from a general perspective, attempting to answer questions such as the best way to implement GAs in PIMs. To reach this goal, the quality of the solutions and the running time provided by proposed PIMs are evaluated over four $\mathcal{NP}$ -hard problems of different combinatorial nature: reversal and translocation evolutionary distance, task mapping and scheduling, and N -Queens. The experiments show that tuning the parameters of the breeding cycle and migration policies is vital to guarantee that all proposed PIMs reach good speed-ups and more accurate solutions than the sequential GA. In addition, experiments ratify that synchronous models provide the best solutions, while asynchronous models deliver the best speed-ups. Finally, the results show that no model provides either better speed-up or accuracy in general since, as for sequential models, the nature of the problem define which would be the best-adapted PIM.

1. Introduction

Parallel Island Models (PIMs) have received considerable attention in evolutionary computation because of their potential to improve speed-up and accuracy (e.g., [1–3]). The strategy is to map islands in processors, such that each island runs an evolutionary algorithm and occasionally shares genetic material with other islands. The communication between islands is established by a *migration topology* responsible for organizing neighborhoods, over which individuals are periodically exchanged according to a migration policy. Such a migration policy is responsible for controlling the movement of individuals and involves parameters as: *number of individuals* that are sent (emigrants) and received (immigrants) from a local to a target island; *migration interval*

between one migration and another; *emigration policy* that defines whether emigrating individuals are just *moved* or *cloned* and sent to a target island, among others. It is well-known that genetic algorithms (GAs) are relatively easy to be implemented in parallel (e.g., [4]) as well as that they perform successfully to deal with $\mathcal{NP}$ -complete problems (e.g., [5,6]). The aim of this paper is to obtain general observations about the performance of PIMs using GAs.

PIM architectures are characterized by their communication topologies and may implement either *synchronous* or *asynchronous* migration between their islands. PIMs' neighborhood is organized through *static* or *dynamic* topologies. In a static topology, the communication between islands is fixed at the beginning of execution and remains unchanged, while in a dynamic topology, it can change. In synchronous PIMs,

[☆] Research funded by the Brazilian agencies CNPq, grants Universal 409003/21-2 and RG 313290/21-0 (M. Ayala-Rincón); FAPDF, Brazil, grant DE 00193-00001175/2021-11 and FAPDF, Brazil, grant 202310267000223.

* Corresponding author.

** Corresponding author at: Department of Computer Science, Universidade de Brasília, Brasília D.F., Brazil.

E-mail addresses: lucasangelo@ibict.br (L.A. da Silveira), thaynaradelima@ufg.br (T.A. de Lima), jesseh.barreto@gmail.com (J.B.d. Barros), jose.soncco@unsaac.edu.pe (J.L. Soncco-Álvarez), llanos@unb.br (C.H. Llanos), ayala@unb.br (M. Ayala-Rincón).

URL: <http://ayala.mat.unb.br> (M. Ayala-Rincón).

<https://doi.org/10.1016/j.asoc.2023.110880>

Received 5 December 2021; Received in revised form 12 July 2023; Accepted 19 September 2023

Available online 27 September 2023

1568-4946/© 2023 Elsevier B.V. All rights reserved.

islands evolve performing each generation simultaneously. On the other hand, in asynchronous PIMs, islands evolve independently even during migration (the behavior found in nature).

In contrast with recent works, our approach presents some advantages in constructing neighborhoods and other aspects to explore dynamic topologies. Recently, Duarte et al. [7,8] designed synchronous dynamic PIMs with five islands in which the neighborhood is built according to their *attractiveness* dynamically adjusted along migrations. The attractiveness between islands is computed using the fitness of the individuals in a target island and the migration history from the local (source) to the target island giving rise to different size populations in each island. Although migration history is an interesting bio-inspired factor (based on the phenomenon of stigmergy), a technique that maintains processors running evolutionary algorithms over a few individuals may waste the parallelism's potential. Besides, processors executing computations over attractive islands could be overloaded, giving rise to performances similar to a sequential algorithm. Indeed, Duarte et al. [8] mentioned that their approach provided lower quality results than the sequential version in some cases concluding that the best results are achieved when there is no extinction of islands. In our approach, to maintain processors working in a balanced manner, we define the neighborhood based on the fitness of individuals and its standard deviation in each island. Also, we design synchronous and asynchronous, static and dynamic PIMs and perform experiments dealing with a total population of the same size equally distributed over PIMs architectures with a different number of islands (12 and 24). Another significant difference is that Duarte et al. [8] configure the migration parameters manually, whereas our approach performs an exhaustive calibration phase to establish the migration parameters. As a result, all our proposed PIM architectures surpassed their sequential versions regarding speed-up and accuracy of solutions.

Some related works proposed models focusing on their adaptability to specific problems. For instance, Federici et al. [9] proposed an exciting class of synchronous PIMs with a unique unidirectional radial dynamic topology aiming to improve the quality of the solutions for real-world problems involving satellite trajectories. In that work, the quality of solutions was improved using fixed parameter values but changing just the number of individuals and the number of generations. Since we seek to provide general observations about PIMs' performance, our experiments are performed for various calibrated static, dynamic, synchronous, and asynchronous PIMs over four $\mathcal{NP}$ -complete problems. Furthermore, experiments are executed using the same resources for all PIMs (number of generations and individuals) to guarantee a fair comparison.

We design synchronous and asynchronous PIMs to investigate which synchronization mechanism would be superior, in general. Indeed, there are some divergences about which alternative provided better results. For example, in early works, it was suggested that asynchronous PIMs were capable of providing just better speed-ups than synchronous architectures (e.g., [10]). Indeed, recently, Abdelhafez et al. [3] show the running time superiority of asynchronous architectures, which in some cases achieved super-linear speed-ups regarding their synchronous versions. However, several works conclude that asynchronous models can provide better accuracy than their synchronous versions (e.g., [11,12]). Moreover, unlike the authors mentioned above, we calibrate the parameters of each model separately to obtain the best specific configurations and thus have a fair and more robust analysis regarding the general behavior of each model, either synchronous or asynchronous.

In this work, well-known migration policies, which play a crucial role in the performance of PIMs, are fixed through calibrated parameters. Authors on Grosso [13], and Starkweather et al. [14] focused on the *migration interval* and concluded that high frequency leads to premature convergence and moderated migration intervals are propitious to find better solutions. Skolicki and De Jong [15] studied the influence of *migration interval* together with *number of immigrants and*

emigrants, concluding that the former is a dominant factor, and the latter plays a secondary role. Also, frequent migrations lead islands to dominate others and lose global diversity before exchanging genetic material. A similar conclusion is found in [16]. Cantú-Paz [1] observed that the response to increasing speed-up is linked to population convergence caused by selection pressure related to the *type of individuals* selected for migration. They noticed that convergence happens quickly with a migration policy, where good individuals are chosen on local islands to replace the worst individuals on the target island. Conversely, replacing random with random individuals has a moderate impact, and convergence happens slowly when migrants and immigrants are randomly selected. In this work, migration parameters as *number of migration individuals*, *type of migration individuals*, and *migration interval* are calibrated by the tuning method [17] in order to maximize the accuracy in synchronous and asynchronous models.

We design thirty-two different synchronous and asynchronous static and dynamic topologies and perform experiments over four $\mathcal{NP}$ -hard problems of different combinatorial nature: two of them related to evolutionary distance, the unsigned reversal and translocation distance problems (URD and UTD), task mapping scheduling (TMP) and N -Queens. All calibrated PIMs provide solutions that consistently surpass the quality of their sequential versions. In addition, breeding cycle and migration parameters tuning clarify how parameter setting is crucial to guarantee the best adaptation of the model for each problem. Finally, statistical tests confirm the results.

The feedback obtained from the experiments does not give rise to a generic model that offers better speed-up and better accuracy for all case studies. Regarding accuracy, synchronous PIMs provide the best solutions except for the N -Queens problem. Synchronous static 12-island models compute the most competitive solutions for URD, while for UTD, both static and dynamic 12-island models compute the best solutions. For TMP, the best solutions were delivered by synchronous dynamic 24-island PIMs, and for N -Queens, the best results were obtained through a synchronous static binary tree 24-island model. Regarding speed-up, in general, asynchronous PIMs present better speed-ups than synchronous models, but this is not a rule; indeed, the speed-up is strongly influenced by the used parameters. In most cases, 24-island models process inputs faster than their 12-island versions, but this does not hold for PIMs calibrated for the N -Queens problem as well as for asynchronous PIMs calibrated for TMP.

Organization Section 2 presents the four case studies, which can be omitted if the reader is familiarized with them, and Section 3 the terminology about the migration parameters in PIMs. Section 4 presents the implemented PIMs; afterwards, Section 5 presents experiments and discussion, and Section 6 discusses statistical tests. Finally, after discussing related work in Section 7, Section 8 concludes and proposes future work.

Note to the reviewers extended descriptions of the four case studies (Sections 2.1 to 2.4), parameter values and numerical results of the experiments (Section 5) are included as supplementary information in the Appendix. This supplementary information will be omitted in the final version of the manuscript. A preliminary version of this manuscript, the source code, and data used in the experiments are available at <http://genoma.cic.unb.br>.

2. Case studies

In this section, the four case studies used for experiments and PIMs are briefly described.

2.1. Reversal distance problem

A unichromosomal genome with n genes can be represented as a permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, where $\pi(i) = \pi_i$, $1 \leq i \leq n$ and $\pi_i \in \{1, \dots, n\}$. A reversal $\rho^{j,k}$, for $1 \leq j \leq k \leq n$ is an operation on π

that inverts the sub string between π_j and π_k transforming π into $\pi' = (\dots, \pi_{j-1}, \pi_k, \dots, \pi_j, \pi_{k+1}, \dots)$; for example: for $\pi = (1, 4, 3, 5, 8, 2, 7, 6)$, $\rho^{5,7}(\pi)$ gives $\pi' = (1, 4, 3, 5, 7, 2, 8, 6)$. There are two different types of permutations: signed and unsigned. In the unsigned case, genes are abstracted without any orientation, and in the signed case, each gene has a positive or negative sign reflecting its orientation. A reversal acts over a signed permutation by also inverting the genes' orientation in the reversed substring.

The *reversal distance* is the shortest length of a sequence of reversals that transform a permutation π into the identity permutation ι , where ι is the permutation $(1, 2, \dots, n)$, for the unsigned case, and $(+1, +2, \dots, +n)$, for the signed case. The *sorting by reversals problem* consists in determining the reversal distance of a permutation [18]. The signed version of this problem, for short SRD, was proved to be in $\mathcal{P}$, namely, an $O(n^2)$ algorithm was first given by Hannenhalli and Pevzner [19], and a linear algorithm was proposed by Bader et al. [20]. The unsigned version, for short URD, was proved to be $\mathcal{NP}$ -hard by Caprara [21]. The case study considered in this work is URD. SRD has been extensively studied in the field of combinatorics of permutations (e.g., [22,23]), and our algorithms apply Bader et al. linear solution for SRD as the fitness function to solve URD, as done in [24].

2.2. Translocation distance problem

A multi-chromosome unsigned genome consists of a set of chromosomes $\{X_1, \dots, X_t\}$, where each X_v , for $1 \leq v \leq t$, is a gene sequence such that each gene x_i is represented by a different natural number appearing only once in the genome. Consider two chromosomes $X = (x_1, x_2, \dots, x_k)$ and $Y = (y_1, y_2, \dots, y_m)$ of an unsigned genome. There are two kinds of translocations:

- A *prefix-prefix* translocation $\rho_{pp}(X, Y, i, j)$, $1 < i \leq k$, $1 < j \leq m$ that transforms the chromosomes X and Y into two new chromosomes $(x_1, \dots, x_{i-1}, y_j, \dots, y_m)$ and $(y_1, \dots, y_{j-1}, x_i, \dots, x_k)$.
- A *prefix-suffix* translocation $\rho_{ps}(X, Y, i, j)$, $1 < i \leq k$, $1 < j \leq m$ that transforms the chromosomes X and Y into two new chromosomes $(x_1, \dots, x_{i-1}, y_{j-1}, \dots, y_1)$ and $(y_m, \dots, y_j, x_i, \dots, x_k)$.

Suppose the genes in two genomes A and B with an equal number of chromosomes are the same. The *Unsigned Translocation Distance* (UTD) problem consists of finding the minimum number of translocations for transforming A into B . Without loss of generality, the genes of A and B can be renamed so that B can be rewritten as an identity. For the experiments, it is assumed that genome B is an identity genome, that is, a genome with all its genes sorted in increasing order; for instance, $A = \{(1, 3, 7)(5, 2, 6, 4)\}$ and $B = \{(1, 2, 3, 4)(5, 6, 7)\}$. UTD is addressed in this paper and was shown to be $\mathcal{NP}$ -hard by Zhu and Wang [25]. As for SRD, the Signed Translocation Distance problem (STD) considers signed genomes and belongs to $\mathcal{P}$. Indeed, algorithms of time complexity $O(n^3)$ and $O(n)$ were respectively designed by Hannenhalli [26] and by Bergeron et al. [27]. In genetic and evolutionary approaches, the fitness function to solve UTD problems is computed by an STD solution as the one proposed by Bergeron et al. [27] that is the solution applied in our approach (e.g., [28]).

2.3. Task mapping and scheduling

The *Task Mapping Problem* (TMP) consists of mapping tasks of a *Real-Time Application* (RTA) onto one of the multiple processor cores of a *Real-Time System* (RTS). The correctness of an RTS depends on its compliance with deadlines.

A type of multiple processors, *Multiprocessor System-on-Chip* (MP-SoC) [29] is considered in this case study. MPSoCs have a communication architecture that consists of a *Network-on-a-Chip*, a well-established intra-chip communication architecture paradigm [30]. Due to the real-time setting, the *Network-on-a-Chip* fits the category of a *Real-Time*

Network-on-a-Chip (RTNoC) as defined by Hesham et al. [31]. Such an RTNoC consists of a wormhole packet switching strategy and virtual channels, allowing the preemption of data flows during transmission by ones with a higher priority level.

For static schedulability analysis of feasible mapping solutions, this case study uses a specific model for both the RTNoC-based MPSoC platform and the RTA mapped onto it that is similar to the one by Indrusiak [32]. The considered TMP is $\mathcal{NP}$ -complete similar to the knapsack problem as given by Garey and Johnson [33]; thus, a brute force approach is unfeasible. Indeed, a case where n tasks are being mapped onto a system with m processors gives a search-space with m^n possible task mapping solutions.

The platform model, $\Psi = (\Pi, R, \Lambda)$, consists of $\Pi = \{\pi_1, \dots, \pi_n\}$, a set of homogeneous processing elements connected to its network interface elements, and a set of routing elements, $R = \{\rho_1, \rho_2, \dots, \rho_n\}$, to switch messages that connect elements in Π and themselves using $\Lambda = \{\lambda_{\pi_1, \rho_1}, \lambda_{\rho_1, \pi_1}, \lambda_{\pi_1, \rho_2}, \dots, \lambda_{\rho_n, \pi_{n-1}}\}$, a set of unidirectional links in a regular $i \times j$ processors mesh-grid topology, where $ij = n$. The model comprises a set of tasks Γ , such that $\tau \in \Gamma$ is given as a tuple $\langle C, T, D, P, \phi \rangle$ formed by the worst-case running time, inter-arrival period, relative deadline time, priority-level, and a transmitted message ϕ , respectively. At the end of its execution, the task τ may transmit a message ϕ towards another task. The schedulability analysis considers the utilization factor of processor cores and links. The utilization factor for a processor core, U_π , presents the ratio in which the processor is used by the tasks mapped onto it as in [34].

The same principle is used to evaluate the utilization rate of links in the RTNoC to check whether they do not exceed their maximum communication bandwidth.

This simple analysis is necessary to avoid non-schedulable solutions demanding resources that exceed the available resource capacity, implying that tasks and messages do not comply with deadlines. This holds for any given processor π if its utilization factor is such that $U_\pi > 1$ and for any given link λ such that $U_\lambda > 1$. However, the utilization analysis is not sufficient. Even in cases where all processors and links are not over-utilized, the task mapping may not be fully schedulable depending on the task set in the mapped RTA. Even though these tests are not sufficient, they are useful to quickly identify whether mappings suit the time requirements.

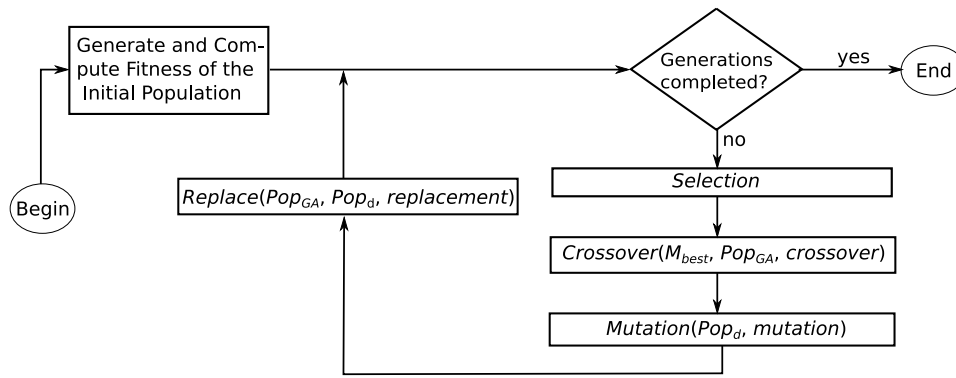
Given a set of tasks Γ and a platform Ψ , a task mapping solution can be encoded as an n -dimensional vector of positive integers $\mathbf{x}$, where each component, x_j , represents the index of the processor in the platform Ψ responsible for processing the task $\tau_j \in \Gamma$. The search-based optimization for a task mapping solution that reduces the number of overused resources is expressed by a function f_{util} that calculates the number of processors and links overburdened in the system given a task placement $\mathbf{x}$. The minimization goal using f_{util} as a fitness function is to find a task placement $\mathbf{x}^*$ that respects processors and links maximum capacities.

2.4. N-Queens

This problem aims to place n queen chess pieces in an $n \times n$ chessboard in a configuration in which no two queens can attack each other. The problem is related with the *N-Queens* completion problem that consists in completing a configuration from a given partial solution and is known to be both $\mathcal{NP}$ -complete and $\#P$ -complete (see [35]).

This problem is commonly used as a benchmark in different machine learning-based methods. For example, it has been used in constructive backtracking algorithms [36], and EA based meta-heuristics such as GA [37], and PSO [38].

The search for a solution corresponds to a minimization problem. Considering the search space is given by n different positions in an $n \times n$ -chessboard its size is given by $(n^2)/(n!(n^2 - n)!)$ and the measure of each possible solution is given by the number of attacking movements between queens at these positions. Here, each candidate placement

Fig. 1. Flowchart of the $\mathcal{GA}_S$.

solution is encoded as an n -dimensional vector $\mathbf{x}$, where the pair (j, x_j) , for j an index of $\mathbf{x}$, represent the placement of a queen onto the j th row and x_j th column of the chessboard. Notice that in this form the search space is reduced to a set S of n^n possible placements. The goal is to minimize a function $f_{attack} : S \rightarrow \mathbb{N}$ that maps each $\mathbf{x} \in S$ into the number of pairs of queen pieces under mutual attack given by the placement $\mathbf{x}$. The function f_{attack} , used as fitness function, is linearly computed.

3. Parallel Island Models (PIMs)

In PIMs, each island runs an EA with its population evolving independently and, periodically individuals are shared between islands by migration. The migration process can be *synchronous* occurring when all islands reach the number of generations controlled by the *migration interval*, or *asynchronous*, occurring when each island controls its migration process through the migration interval independently of each other.

An organizational architecture known as *migration topology* is responsible for establishing a communication model between islands, where at the given migration interval, individuals are selected in local islands and sent to target islands. For each island, the population is bi-partitioned ranking individuals from the one with the best fitness to the one with the worst fitness. This ranking is considered to select individuals in the migration process. The *migration parameters* (for short, Mig_p) are abstracted below as argued by Sudholt [2].

- *NumMigIndividuals*: number of migration individuals.
- *TypeEmIndividual*: type of individuals selected for emigration among
 1. **best**,
 2. **worst**, and
 3. **random**.
- *EmPolicy*: emigration policy to decide whether emigrating individuals in the local island are
 1. **cloned** or
 2. **removed** to be sent to the target island.
- *TypeImIndividual*: type of individuals in the target island that are replaced by immigrants among
 1. **worst**,
 2. **random**, and
 3. **similar**, where by similar individuals, it is understood replacement of individuals with the same fitness rank than the immigrants.
- *MigrationInterval*: percentage of breeding cycles (number of generations) in which migration occurs.

When these parameters are properly configured, it is expected that accuracy and speed-up in PIMs be improved.

4. Implemented parallel island models

Initially, a simple genetic algorithm ($\mathcal{GA}_S$) responsible for the evolution of the local population on each island is presented. Subsequently, the homogeneous static and dynamic PIMs implemented are detailed.

4.1. Evolutionary engine of the islands

The representation of the search space is dependent on the optimization problem. The individuals mirroring solutions are represented as an array of integers, and their representation is dependent on the case study.

For URD and UTD, a population of signed genomes is randomly generated assigning signs to the genes of an unsigned genome given as input. For instance, individuals $(+1, n-3, +2)(+4, -7, +6, +8)$ and $(+1, -2, -4, -3, +5)$ could be obtained from inputs $(1, 3, 2)(4, 7, 6, 8)$ and $(1, 2, 4, 3, 5)$, respectively. For TMP and N -Queens, each individual is a task mapping solution or a candidate placement solution. For instance, an individual for the 8-Queens problem considering could be $(4, 3, 8, 1, 7, 6, 3, 2)$, where the array indices and elements represent the row and column in which a queen is stored. For TMP, an individual for a problem with 13 tasks to be scheduled on a 4×3 grid, is given as a one-dimensional array as for instance, $(2, 1, 3, 5, 4, 6, 4, 7, 9, 10, 11, 12, 8)$, in which the array indices and elements respectively represent the task and the processor responsible for the task.

The fitness function is linked to the applied case study. For UTD and URD, the linear algorithms to solve STD and SRD, proposed by Bergeron et al. [27] and Bader et al. [20], respectively, are applied. For TMP, a utilization-based test proposed by Indrusiak [32] is applied. It has complexity $O(n_\lambda^2 + n_\pi^2)$, where n is the number of tasks in the used RTA model and n_λ and n_π are the number of links and processing cores in the platform, respectively. Finally, the algorithm used for evaluating the fitness function for N -Queens linearly counts the number of mutual attacks.

Fig. 1 shows the $\mathcal{GA}_S$ flowchart. The breeding cycle is fixed in n generations, and the initial population is randomly generated. Individuals represent possible solutions to the problem. Each individual's fitness is determined by calculating the objective function according to the problem. The breeding cycle works with three procedures: *Crossover*, *Selection*, *Mutation* and *Replace* and genetic breeding cycle parameters for the percentage of selection and replacement, and probability of application of mutation and crossover, which are stored in the BC_p structure.

In the *Selection* procedure, according to the *selection* parameter, a percentage of the best individuals is selected from the population Pop_{GA} and stored in a vector M_{best} . The *Crossover* procedure, *Crossover*(M_{best} , *crossover*), subjects individuals in M_{best} to crossing with probability given by the *crossover* parameter. All pairs of individuals selected for crossing are fragmented using a single cut-off point, producing four fragments that later combined produce two offspring stored in the vector Pop_d . It returns (in Pop_d) all offspring with their computed fitness.

The procedure *Mutation*(Pop_d , *mutation*) tries to apply mutation to each offspring in Pop_d according to the *mutation* probability parameter; when successfully, the offspring's fitness is calculated. This parameter

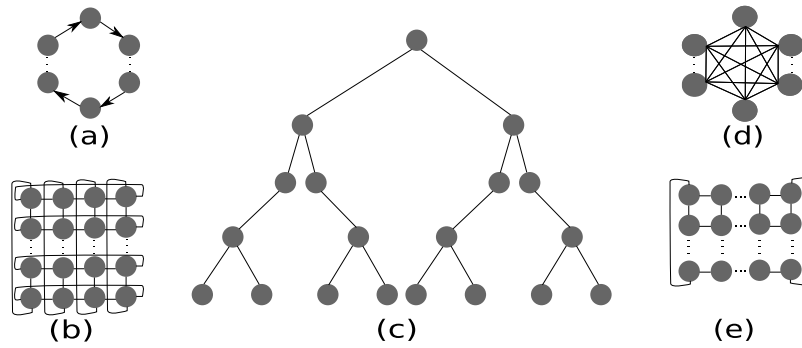


Fig. 2. (a) ring, (b) torus, (c) example of a binary tree with 12 nodes, (d) complete graph, and (e) $x \times y$ -net, where x and y represent rows and columns in the net.

should be small to avoid compromising the inherited genetic material. It is important to emphasize that mutating a gene varies from problem to problem. For UTD and URD, a mutation only inverts the signal of a gene; for N -Queens, it consists of allocating the queen to a random position on the board; and for TMP, it consists of randomly assigning a task to a processor. Only mutations with better fitness than the fitness of the mutated individuals replace the original individuals.

Finally, the *Replace* procedure, $\text{Replace}(\text{Pop}_{GA}, \text{Pop}_d, \text{replacement})$, includes descendants in Pop_d into the current population. The percentage of individuals to be eventually replaced from the current population, Pop_{GA} , is given by the *replacement* parameter. The fitness of each individual in Pop_d is compared with the fitness of a randomly selected individual from the *replacement* percent of individuals in Pop_{GA} , and the individual in Pop_{GA} is replaced by the descendant only if its fitness is worse; otherwise, the descendant is discarded.

Synchronous and asynchronous, homogeneous PIMs are implemented to solve each case study presented in Section 2. Each island runs an instance of the $\mathcal{GA}_S$ to evolve the native population.

As discussed by Cantú-Paz [4], Whitley et al. [39], and Duarte et al. [7], for a fair comparison, PIMs and sequential EAs must have total populations of the same, size. Thus, the initial population on each island is generated according to the methodology adopted in [40]: the population Pop_{GA} of the sequential $\mathcal{GA}_S$ is partitioned into sets of equal size and distributed among the islands. Each island receives $|\text{Pop}_{GA}|/N_I$ individuals, where N_I is the number of islands used by the implemented PIM.

The pseudo-code for homogeneous PIMs is presented in Algorithm 1. Each island, $0 < I \leq N_I$, in a PIM runs this algorithm. The parameter X is an instance of a problem; Pop_{GA} , the population of the sequential $\mathcal{GA}_S$; *maxIteration*, the number of generations to be executed; BC_p , the structure containing the breeding cycle parameters; and, Mig_p is the structure with the migration parameters. Mig_p contains the parameters (*NumMigIndividuals*, *TypeEmIndividual*, *TypeImIndividual*, *EmPolicy*, *MigrationInterval*) and *Topology*, and a Boolean variable *Synchronous*. Indeed, some parameters are pre-processed, as $\text{Mig}_p.\text{MigrationInterval}$ that is passed as $\text{MigrationInterval} * \text{maxIteration}$, i.e., the number of iterations between migration phases. Initially, the population of each island is created from Pop_{GA} computing fitness (lines 3 and 4); then, the evolutionary process starts (lines 6 to 20) repeating migrations after each $\text{Mig}_p.\text{MigrationInterval}$ generations (lines 12 to 17). Migration is controlled by the variable *Mig_Int* (lines 5, 12, and 16). In each generation, first, the evolutionary $\mathcal{GA}_S$ procedure (*selection*, *crossover*, *mutation* and *replacement*) is executed (lines 7 to 10). If migration should be executed (line 12), it is performed according to the PIM topology calling algorithms responsible for establishing neighborhoods and exchanging individuals within the neighborhood (lines 13 and 14). Algorithms *StaticMigTop* and *DynamicMigTop* implement migration policies over the different static and dynamic topologies (Sections 4.2 and 4.3). Finally, if the PIM is synchronous, the breeding cycle of the islands is synchronized (line 18).

4.2. Static topologies

In static topologies, migration happens between neighboring islands statically established by the topologies: ring, torus, binary tree, complete graph, and $x \times y$ -net. Except for the directed ring topology, all other topologies have bidirectional communication (see Fig. 2). Torus differs from the net topology in its neighborhood. All nodes (representing islands) on a torus are adjacent to four other nodes, while on the net topology, internal and boundary nodes are adjacent to three and four nodes, respectively. Regarding communication balancing, interactions between islands are more significant in the torus topology, where each island communicates with four islands. This causes a greater spread of genes through the neighborhood when compared with the tree, ring, and net topologies. In the tree topology, internal islands communicate with at least two and at most three other islands, whereas leaf islands communicate with a single island. In the net topology, all frontier islands are connected with three islands, and the internal islands are connected with four islands. Ring topology provides a smooth scenario for the dissipation of genetic material.

These topologies provide an environment conducive to observe how miscegenation will impact the running time and accuracy of results in PIMs. Thus, for each topology shown in Fig. 2, synchronous and asynchronous 12- and 24-island PIMs were proposed (according to experimental constraints). A total of twenty PIMs with static topologies were proposed. The notation below discriminates these PIMs. Subscripts prefixes C, R, Tr, To, and N represent complete, ring, tree, torus, and net topologies, respectively; 12 and 24, the number of islands; and, subscript suffixes S and A differentiate synchronous and asynchronous PIMs.

- Complete graph PIMs: P_{C12S} , P_{C12A} , P_{C24S} and P_{C24A} ;
- Ring PIMs: P_{R12S} , P_{R12A} , P_{R24S} and P_{R24A} ;
- Tree PIMs: P_{Tr12S} , P_{Tr12A} , P_{Tr24S} and P_{Tr24A} ;
- Torus PIMs: P_{To12S} , P_{To12A} , P_{To24S} and P_{To24A} ;
- 4×3 - and 6×4 -net PIMs: P_{N12S} and P_{N12A} , and P_{N24S} and P_{N24A} .

Whenever an island I of a static PIM reaches a migration generation, t , Algorithm 1 (line 13) calls $\text{StaticMigTop}(\text{Pop}_I^t, \text{Mig}_p)$, presented in the flowchart in Fig. 3. The arguments are the current population Pop_I^t of the local island and the migration parameters Mig_p . Initially, a number of emigrants defined in $\text{Mig}_p.\text{NumMigIndividuals}$ of type $\text{Mig}_p.\text{TypeEmIndividual}$ are selected from the Pop_I^t and stored in **Emigrants**. Subsequently, the neighbor islands that will receive emigrants from I are stored in **neighbors**. These neighbors depend on the PIM topology. Then, the individuals in **Emigrants** are sent to islands in **neighbors**. Also, island I receives $\text{Mig}_p.\text{NumMigIndividuals}$ immigrants from its neighbor islands and stores them in **Immigrants**. Individuals in **Immigrants** are incorporated to the population of island I according to the migration policy. If $\text{Mig}_p.\text{EmPolicy}$ is set as *Remove*, $\text{Mig}_p.\text{NumMigIndividuals}$ individuals in **Immigrants** replace the individuals in Pop_I^t that were selected for as emigrants and the

Algorithm 1 Pseudo code for PIMs**INPUT:** $X, Pop_{GA}, N_I, maxIteration$ and parameters: BC_P, Mig_p **OUTPUT:** Solution to X

```

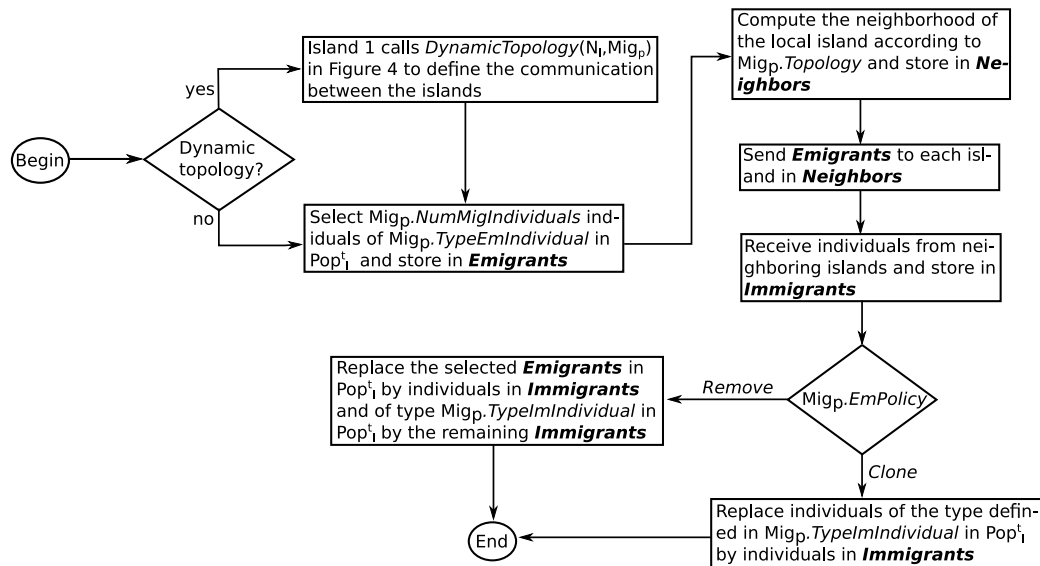
1: procedure IslandModels( $X, Pop_{GA}, N_I, maxIteration, BC_P, Mig_p$ )
2:   for each island  $I$  do in parallel
3:     Allocate a different population  $Pop_I^0$  from  $Pop_{GA}$ , of size  $|Pop_{GA}|/N_I$ 
4:     Compute fitness of  $Pop_I^0$ 
5:      $Mig\_Int \leftarrow Mig_p.MigrationInterval$ 
6:     for  $t = 1$  to  $maxIteration$  do
7:        $M_{best} \leftarrow$  Select the  $BC_P.selection * |Pop_I^{t-1}|$  best individuals in  $Pop_I^{t-1}$ 
8:        $Pop_d^t \leftarrow Crossover(M_{best}, BC_P.crossover)$ 
9:        $Pop_d^t \leftarrow Mutation(Pop_d^t, BC_P.mutation)$ 
10:       $Pop_I^t \leftarrow Replace(Pop_I^t, Pop_d^t, BC_P.replacement)$ 
11:       $Mig\_Int \leftarrow Mig\_Int - 1$ 
12:      if  $Mig\_Int == 0$  then
13:        if static topology then StaticMigTop( $Pop_I^t, Mig_p$ )
14:        else DynamicMigTop( $Pop_I^t, N_I, Mig_p$ )
15:        end if
16:         $Mig\_Int \leftarrow Mig_p.MigrationInterval$ 
17:      end if
18:      if  $Mig_p.Synchronous$  then Synchronize all islands
19:      end if
20:    end for
21:  end for
22:  return fitness of the best individual
23: end procedure

```

▷ Number of generations to run migration.

▷ Flowchart in Fig. 3

▷ Flowchart in Fig. 3

**Fig. 3.** Flowchart of algorithms *StaticMigTop*(Pop_I^t, Mig_p), and *DynamicMigTop*(Pop_I^t, N_I, Mig_p) - Implementing migration policies over static and dynamic topologies.

remaining individuals in **Immigrants** will replace local individuals of type $Mig_p.TypeImIndividual$ only when their fitness is better. Besides, genotype similarity is checked (as in [41]) to prevent inclusion of equal individuals. On the other hand, if $Mig_p.EmPolicy$ is set as *Clone*, local individuals in Pop_I^t of type $Mig_p.TypeImIndividual$ will be replaced by individuals in **Immigrants** applying the same criteria to fitness and genotype similarity.

4.3. Dynamic topologies

In dynamic PIMs, the neighborhood of each island is redefined before each migration. The dynamism applied in our implemented models

is obtained from the quality and diversity of each island population. Islands are qualified as **good**, **medium**, and **bad**. The addition of variance and average defines the metric to qualify islands. The variance measures the island diversity. High variance is associated with a slight resemblance between native individuals on the island. The average is associated with the island population quality. Since the optimization problems addressed are minimization problems, islands with low average fitness can be considered suitable (**good**) islands. Then, the metric is computed for each island, and islands are ranked in decreasing order before migration starts (see Fig. 5). From this qualification, three dynamic topologies are abstracted. (1) random island communication (for short, **Rd**); (2) same classification island communication: **good** with **good**, **medium** with **medium** and **bad** with **bad** (abbreviated as

Table 1
Numbers encoding synchronous and asynchronous PIMs.

1= P_{N12S}	3= P_{To12S}	5= P_{Tr12S}	7= P_{R12S}	9= P_{C12S}	11= P_{Rd12S}	13= $P_{\simeq12S}$	15= $P_{gbmm12S}$
2= P_{N24S}	4= P_{To24A}	6= P_{Tr24S}	8= P_{R24S}	10= P_{C24S}	12= P_{Rd24S}	14= $P_{\simeq24S}$	16= $P_{gbmm24S}$
17= P_{N12A}	19= P_{To12A}	21= P_{Tr12A}	23= P_{R12A}	25= P_{C12A}	27= P_{Rd12A}	29= $P_{\simeq12A}$	31= $P_{gbmm12A}$
18= P_{N24A}	20= P_{To24A}	22= P_{Tr24A}	24= P_{R24A}	26= P_{C24A}	28= P_{Rd24A}	30= $P_{\simeq24A}$	32= $P_{gbmm24A}$

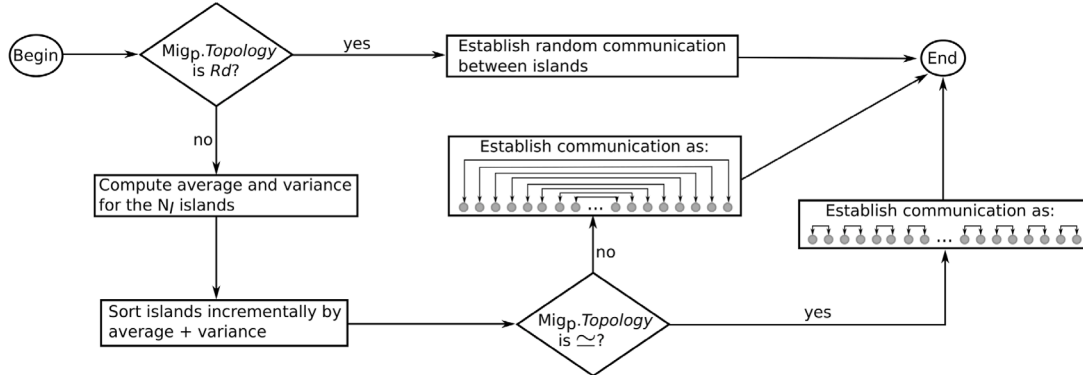


Fig. 4. Flowchart of algorithm $DynamicTopology(N_I, Mig_p)$ performed only by island one – Building island communication in dynamic topologies.

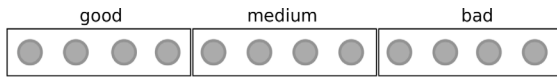


Fig. 5. Classification of sorted islands.

$\simeq$); (3) communication between **good** and **bad**, and between **medium** and **medium** islands (for short, **gbmm**).

The flowchart in Fig. 4 describes the algorithm $DynamicTopology$ that establishes the dynamic communication between islands. This algorithm receives as arguments the number of islands N_I and the migration parameters Mig_p . After computing the metric of the islands, they are non increasingly sorted. The first, middle and final third of the sorted islands are classified as *good*, *medium* and *bad* islands, respectively (Fig. 5). In dynamic PIMs, the migration process is executed after each $Mig_p.MigrationInterval$ generations (line 14 of Algorithm 1); before each migration, island 1 is responsible for building the neighborhood of all islands (according to flowchart in Fig. 3). After the dynamic communication is established, the migration flow follows the same steps as for the static topologies.

Similarly to static PIMs, the notation below discriminates the dynamic PIMs, where the subscript prefixes *Rd*, $\simeq$, and *gbmm*, represent random, topologies connecting islands of the same classification and *gbmm* topologies.

- Random PIMs: P_{Rd12S} , P_{Rd12A} , P_{Rd24S} and P_{Rd24A} ;
- PIMs communicating islands with the same classification: $P_{\simeq12S}$, $P_{\simeq12A}$, $P_{\simeq24S}$ and $P_{\simeq24A}$;
- PIMs communicating **good-bad** and **medium-medium** islands: $P_{gbmm12S}$, $P_{gbmm12A}$, $P_{gbmm24S}$ and $P_{gbmm24A}$.

5. Experiments and discussion

The PIMs were implemented using the MPI library of the C language in the Linux operating system. The experiments were executed on a computer using two Xeon E5-2620 processors with hyper-threading. Each processor has six cores with a CPU clock rate of 2.4 GHz.

The experiments compare the running time and accuracy of PIMs and their sequential versions over populations with the same total amount of individuals. The sequential $\mathcal{GA}_S$ uses populations of size $24n \log n$, where n is the length of the input. At the same time, PIMs have two configurations: (1) models with 24 islands, in which each island has $n \log n$ individuals, and (2) models with 12 islands, each with $2n \log n$ individuals. Thus, all experiments were run with total populations of the same size.

The number of breeding cycles (generations) in sequential $\mathcal{GA}_S$ and PIMs is fixed as the input size. The size for problems URD and UTD is the length of the genome, the size of the N -Queens problem is the number of queens, and the size of TMP is the number of tasks to be addressed. The experiments for TMP were performed for instances using a fixed mesh-grid platform with 6×4 processors; however, it is noteworthy that the PIMs work with any other mesh-grid platform by merely passing the platform as an argument in the models.

In some of the next tables, for brevity, the names of the models introduced in Section 4 are replaced by numbers as given in Table 1.

The speed-ups are computed as the ratio of the running time of the algorithm $\mathcal{GA}_S$ by the running time of each PIM. Each execution is measured using the Linux command *time*. The ratio is computed as the average of ten runs for each different input considering for each problem a number of inputs of significant size as specified in Sections 5.2.1, 5.3.1, 5.4.1 and 5.5.1.

5.1. Parameter setup

GA performance are highly dependent on the values of their parameters. Carefully choosing the values of the parameters is essential for a suitable *parameter tuning*. The parameter tuning approach consists of first finding good parameter values before running the algorithm. After that, the algorithm uses these values that will remain fixed during the breeding cycle. We adopt the so-called “taxonomy T1” discussed in [17]. In essence, the tuning algorithm works by the “generate and test principle”, that is, through the generation of parameter vectors and testing them to establish their usefulness. Here, we use the tuner known in the literature as “iterative tuner” that starts with a small number of

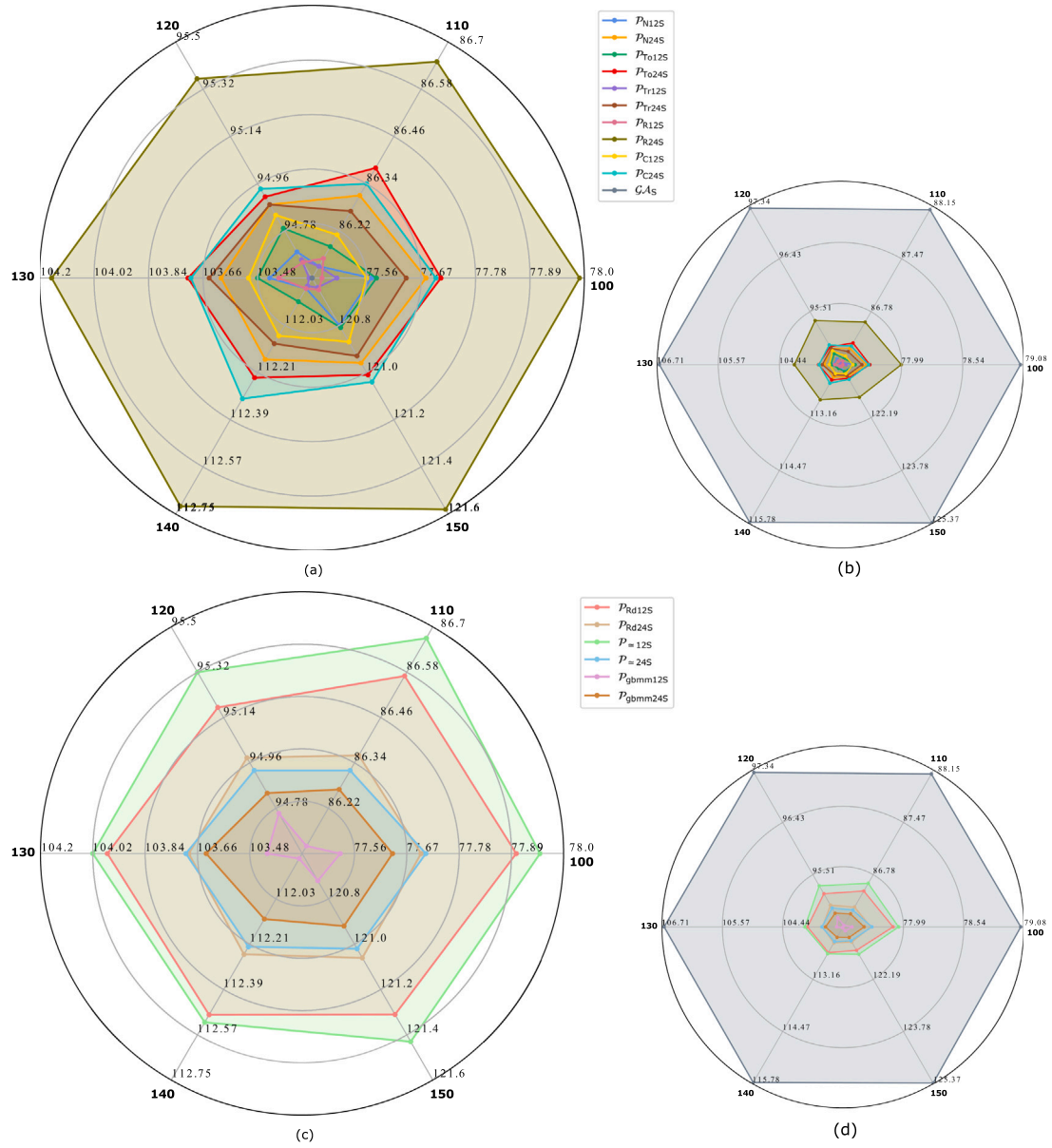


Fig. 6. The average scores of URD for synchronous: (a) static PIMs; (b) static PIMs versus sequential $\mathcal{GA}_S$; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential $\mathcal{GA}_S$.

Table 2
Estimated values for the parameters.

	Parameter	Estimated values
Breeding Cycle - BC_p	<i>crossover</i>	0.02, 0.04, ..., 0.98, 1.0
	<i>mutation</i>	0.01, 0.011, ..., 0.019, 0.02
	<i>selection</i>	2%, 4%, ..., 98%, 100%
	<i>replacement</i>	2%, 4%, ..., 98%, 100%
Migration - Mig_p	<i>NumMigIndividuals</i>	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13
	<i>TypeEmIndividual</i>	1=Best, 2=Worst, 3=Random
	<i>EmPolicy</i>	1=Clone, 2=Remove
	<i>TypeImIndividual</i>	1=Worst, 2=Random, 3=Similar
	<i>MigrationInterval</i>	2%, 4%, ..., 98%, 100%

parameter vectors and creates new vectors iteratively during execution. In the tuner algorithm described in the sequence, we start with just a randomly generated parameter vector.

To improve accuracy, the $\mathcal{GA}_S$ and PIMs were submitted to a parameter tuning taking into account each case study. Table 2 presents the ranges of possible parameter values. For the parameters involving percentage, the tested values range between 2% and 100%, checking in this manner a great variety of possibilities. The crossover rate was set from 0.02 to 1.0, while the mutation probability ranges between 0.01 and 0.02 since in evolutionary processes, it is expected mutation appears rarely. For migration parameters *TypeEmIndividual*, *EmPolicy*, *TypeImIndividual* we just label the individuals as explained in Section 3. Regarding *NumMigIndividuals*, the tested values are naturals from 1 to 13 due to experience collected from other works, where the number of individuals was never greater than 13, as seen in [28,40,42,43].

The size of inputs used in the experiments setup for each case study are described below.

- URD: 100 randomly generated permutations with 150 genes.
- UTD: 100 randomly generated genomes with 150 genes and five chromosomes.
- TMP: 10 task mapping problems with 50 task sets, as defined in Section 2.3, and using a platform 6 × 4 processors. The task sets were generated using uniform distributions to create the τ_i tuple' elements including messages and their destination tasks.
- N-Queens: six randomly generated inputs, each of n queens for $n \in \{100, 110, \dots, 150\}$.

5.2. Results and discussion for URD

5.2.1. Experiments for URD

Packages of one hundred unsigned permutations were randomly generated containing n genes of lengths $n \in \{100, 110, \dots, 150\}$. For each permutation, the PIMs and the sequential $\mathcal{GA}_S$ were executed ten times starting from the same population for all permutations of the same length. Subsequently, the averages of the ten results for each permutation and algorithm were calculated. These averages represent the number of reversals for each unsigned permutation.

Radar charts in Figs. 6 and 7 illustrate the accuracy results for synchronous and asynchronous PIMs, respectively. The small radar charts illustrate how all PIMs improved the accuracy of the $\mathcal{GA}_S$.

5.2.2. Discussion on accuracy for URD

Concerning the synchronous PIMs with static topologies (radar chart (a) in Fig. 6), the best average solutions were provided by P_{Tr12S} and P_{R12S} , while the worst solutions were obtained by ring topology using 24-island, P_{R24S} . Besides, all synchronous static 12-island PIMs provided better results than synchronous static 24-island PIMs. For synchronous dynamic PIMs (radar chart (c) in Fig. 6), the worst solutions were provided by $P_{\approx 12S}$, and the best by $P_{gbmm12S}$ that is the unique synchronous dynamic model that provides competitive

solutions regarding synchronous static 12-island models. The best asynchronous static PIMs (radar chart (a) in Fig. 7) were P_{Tr12A} and P_{R12A} , while P_{N24A} provided the worst solutions. For asynchronous dynamic PIMs (radar chart (c) in Fig. 7), $P_{\approx 24A}$ outputs the worst while $P_{gbmm12A}$ the best solutions. Also, as for synchronous static PIMs, asynchronous dynamic 12-island PIMs provided better results than all dynamic 24-island PIMs. Such better adaptation of 12-island PIMs for the case of synchronous dynamic and asynchronous static PIMs does not happen. Comparing synchronous versus asynchronous PIMs, the experiments showed a balance as described: P_{N12S} , P_{N24S} , P_{Tr12S} , P_{Tr24S} , P_{R12S} , P_{C12S} , P_{C24S} and $P_{gbmm24S}$ surpassed their asynchronous versions; P_{Rd24S} and P_{Rd24A} , $P_{gbmm12S}$ and $P_{gbmm12A}$ output similar results; and, P_{To12A} , P_{To24A} , P_{R24A} , P_{Rd12A} , $P_{\approx 24A}$ and $P_{gbmm12A}$ provided better solutions than their synchronous versions. In all cases, 12-island PIMs were the most competitive: P_{Tr12S} , P_{R12S} , $P_{gbmm12S}$, P_{Tr12A} , P_{R12A} , $P_{gbmm12A}$. In general, the models that best and worst-adapted to the URD problem were the synchronous static PIMs P_{Tr12S} and P_{R24S} , respectively.

5.2.3. Discussion on speed-up for URD

Table 3 presents the speed-ups for all PIMs highlighting the best in bold. The speed-up is computed as the ratio average of the set of the one hundred inputs of length 150 used in the experiments, being each input executed ten times.

Asynchronous and synchronous 24-island PIMs presented better speed-ups. Analyzing synchronous models, P_{R24S} presented the best speed-up (7.17), however the accuracy of its results is low, while P_{Tr12S} , which provided the best solutions, achieved the worst speed-up (4.86). When analyzing the speed-ups of PIMs P_{R12S} (5.35), and $P_{gbmm12S}$ (5.63), models with accuracy close to P_{Tr12S} , there is a slight improvement in the running time, however, they did not provide the best speed-ups. The synchronous PIMs are responsible for providing the best solutions. Concerning speed-up, only 31.25% of the synchronous models present better speed-ups than their asynchronous versions, as shown in Table 3. The small running time advantage of asynchronous over synchronous PIMs can be explained by the values of parameters fixed combined with the overhead imposed in the synchronization of the evolutionary process in the synchronous versions.

5.3. Results and discussion for UTD

5.3.1. Experiments for UTD

Experiments were performed with the selected parameters for calculating translocation distances for all PIMs and the sequential $\mathcal{GA}_S$. For this purpose, synthetic genomes were taken from [42] with n genes, for $n \in \{100, 110, \dots, 150\}$, and with N chromosomes, for $N \in \{3, 4, 5\}$. Then, for all these lengths and number of chromosomes (n, N), packages of hundred genomes were processed, and the average of the results for each algorithm was calculated. Each algorithm was executed ten times for each genome provided as input, and the average of the ten results was given as output.

Big radar charts on the left side in Figs. 8–10 compare the average translocation distances for the synchronous PIMs. In general, synchronous PIMs presented better results than asynchronous PIMs. Radar charts for the asynchronous models are included in the supplementary information. Small radar charts on the right side of these figures compare PIMs and the $\mathcal{GA}_S$ accuracies. As for URD charts, the UTD charts in these figures have different ranges in their radii.

5.3.2. Discussion on accuracy for UTD

The experiments showed that asynchronous models are not competitive regarding their synchronous versions. Synchronous and asynchronous 12-island PIMs provided better solutions than their 24-island versions, except for $P_{gbmm24S}$ that gives better solutions than $P_{gbmm12S}$. The best results using synchronous PIMs processing inputs with three and four chromosomes were obtained by static models P_{N12S} and

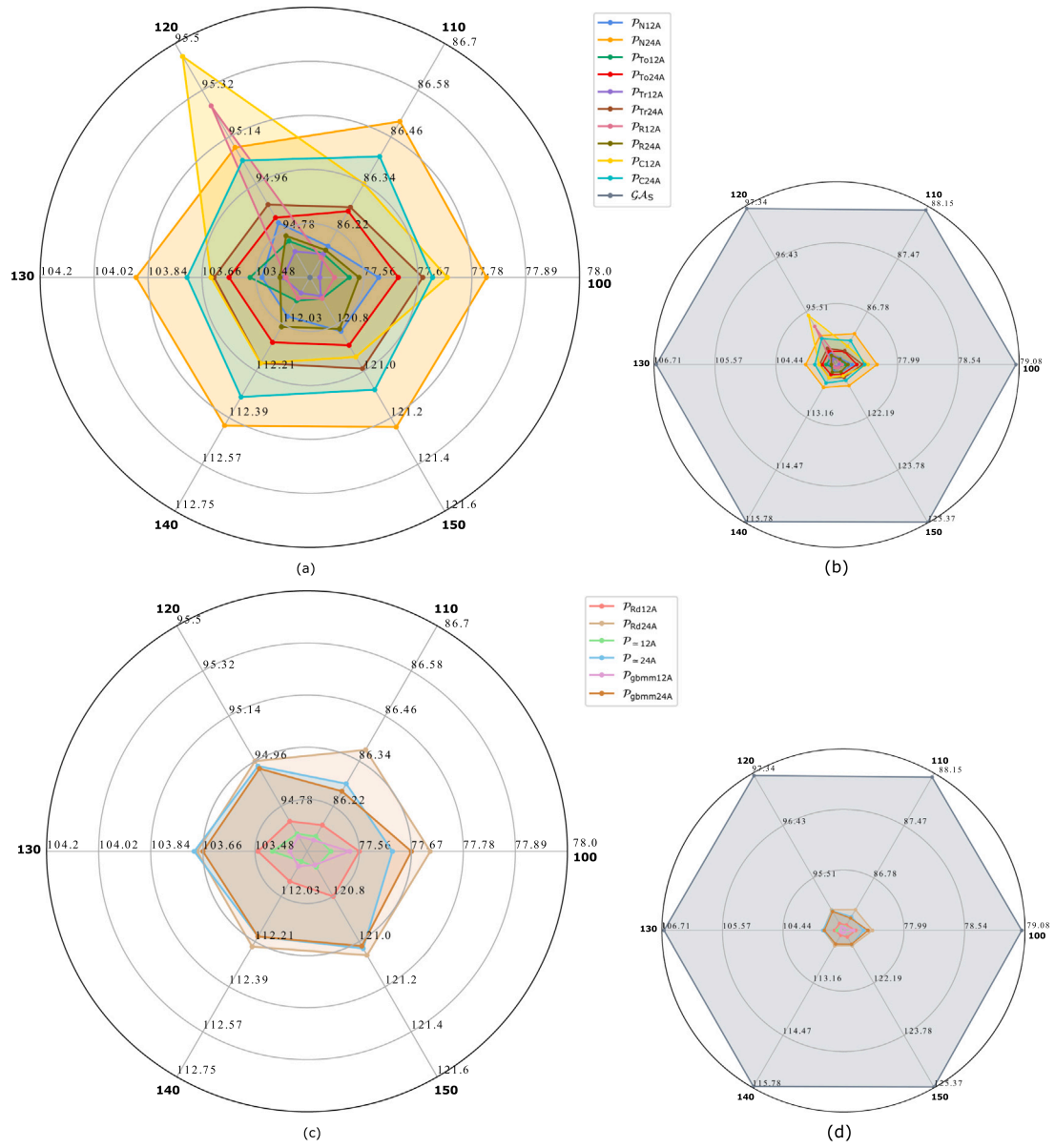


Fig. 7. The average scores of URD for asynchronous: (a) static PIMs; (b) static PIMs versus sequential G_{AS} ; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential G_{AS} .

Table 3

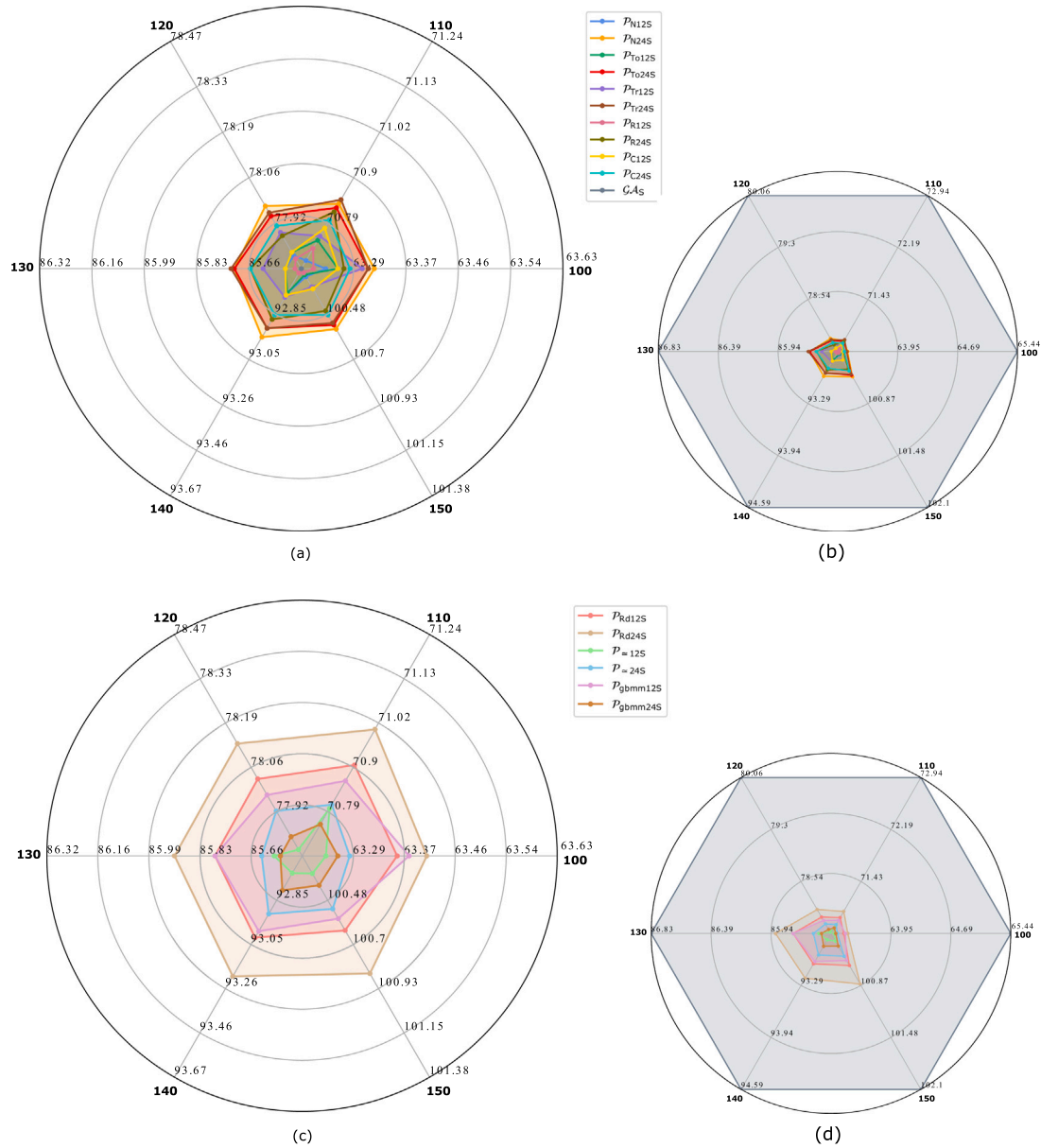
Speed-up considering URD problem for all PIMs (See Table 1).

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
5.79	5.86	5.41	5.92	4.86	5.73	5.35	7.17	5.54	6.28	4.91	5.8	5.67	6.3	5.63	5.47
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
5.68	7.07	5.73	6.45	5.36	6.41	5.07	6.45	6.44	6.85	5.37	6.44	5.33	6.37	5.4	6.14

Table 4

Speed-up considering UTD problem for all PIMs (See Table 1).

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
10.32	13.3	10.65	11.2	11.59	11.21	9.87	10.23	9.46	10.76	19.65	15.83	9.43	10.06	16.06	10.26
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
21.5	21.26	21.40	22.03	18.31	22.55	18.53	22.60	14.48	21.18	15.2	22.34	17.19	23.65	16.41	23.43

**Fig. 8.** The average scores of UTD with three chromosomes for synchronous (a) static PIMs; (b) static PIMs versus sequential $\mathcal{G}_A$ s; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential $\mathcal{G}_A$ s.

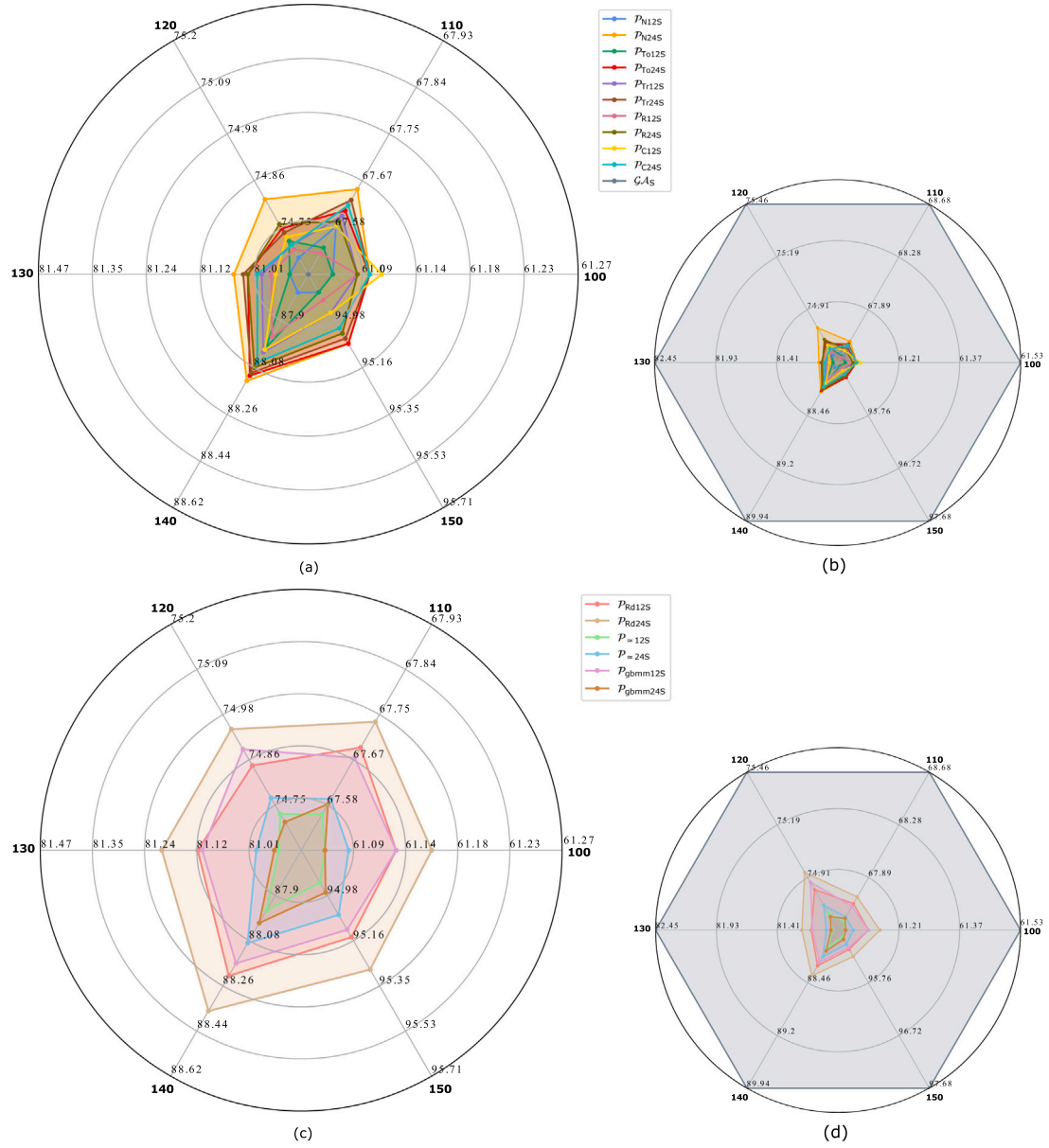


Fig. 9. The average scores of UTD with four chromosomes for synchronous: (a) static PIMs; (b) static PIMs versus sequential $\mathcal{GA}_S$; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential $\mathcal{GA}_S$.

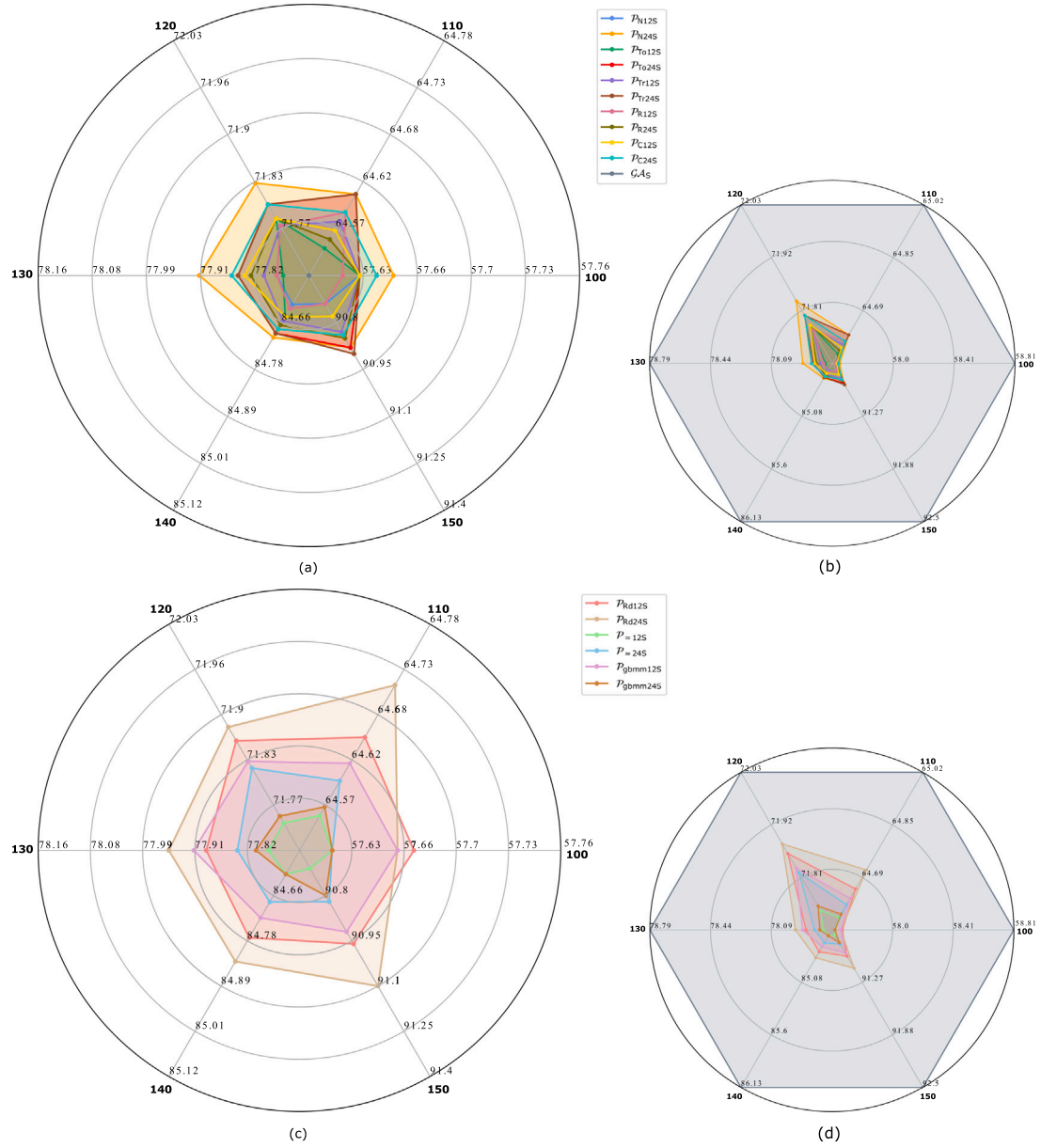


Fig. 10. The average scores of UTD with five chromosomes for synchronous: (a) static PIMs; (b) static PIMs versus sequential $\mathcal{GA}_S$; (c) dynamic PIMs; and (d) dynamic PIMs versus sequential $\mathcal{GA}_S$.

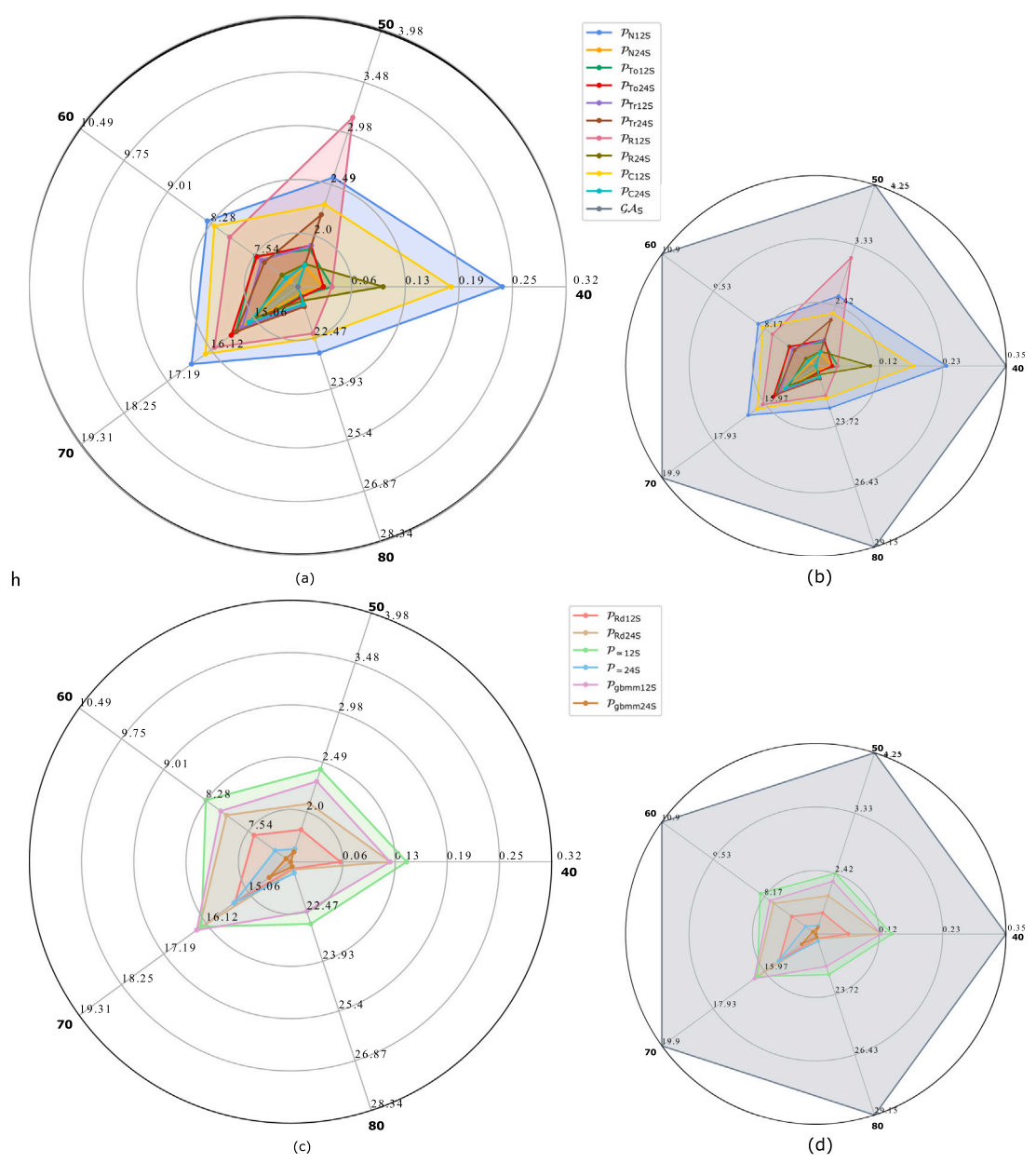


Fig. 11. The average scores of TMP for synchronous: (a) static PIMs; (b) static PIMs versus sequential G_{A_S} ; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential G_{A_S} .

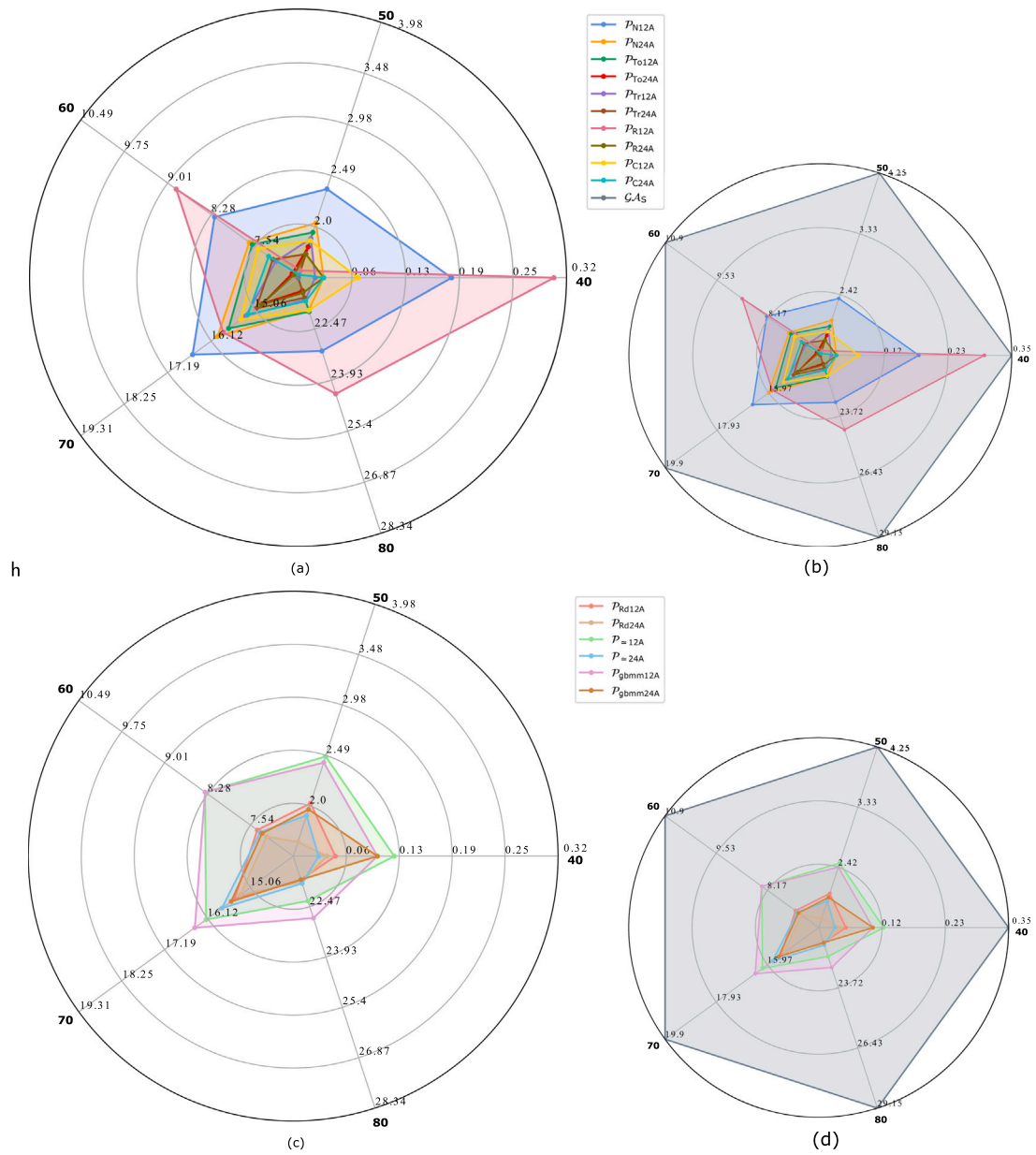


Fig. 12. The average scores of TMP for asynchronous: (a) static PIMs; (b) static PIMs versus sequential GAs ; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential GAs .

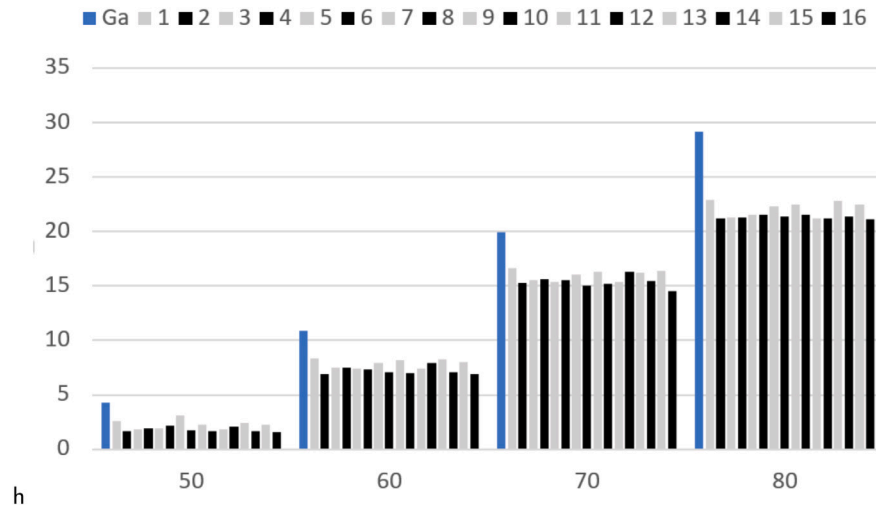


Fig. 13. Adaptation of synchronous 12-island (gray bars) versus 24-island (black bars) PIMs to TMP.

P_{R12S} . In the experiments with five chromosomes, the dynamic model $P_{\approx 12S}$ presented, on average, the best results. In experiments involving asynchronous PIMs, the P_{C12A} static model was the best representative.

The best synchronous static PIMs are P_{N12S} , P_{To12S} , and P_{R12S} , and synchronous static 12-island models provided better solutions than 24-island models. Focusing on synchronous dynamic PIMs, the random PIMs, P_{Rd12S} and P_{Rd24S} , provided the worst solutions being the latter the worst one. In contrast, $P_{\approx 12S}$ is the synchronous dynamic PIMs that provided the best solutions, however, surpassing P_{N12S} , P_{To12S} , and P_{R12S} only for entries containing five chromosomes. In general, for synchronous dynamic PIMs, the second most competitive model is $P_{gbmm24S}$.

For asynchronous PIMs, a different scenario is found: the best static model, P_{C12A} , surpasses the best dynamic PIM $P_{\approx 12A}$ in all experiment scenarios. Also, as for synchronous PIMs, static asynchronous 12-island PIMs provided better solutions than static asynchronous 24-island models. The asynchronous static 24-island models P_{N24A} and P_{To24A} provided the worst solutions. Besides, all asynchronous dynamic 12-island PIMs provided competitive solutions better than those provided by the asynchronous dynamic 24-island models.

5.3.3. Discussion on speed-up for UTD

The speed-ups of the PIMs regarding the sequential $\mathcal{GA}_S$ were measured as the ratio average of the one hundred genomes with 150 genes and five chromosomes. Then, each PIM was executed ten times for each genome, and the average running time was considered. The computed speed-ups are available in Table 4 with the best speed-ups highlighted in bold.

For synchronous PIMs, the best speed-up was given by P_{Rd12S} (19.65) which presents competitive solutions with several other models (P_{N24S} , P_{To24S} , P_{Tr24S} , P_{R24S} , P_{C24S} , P_{Rd24S} and $P_{gbmm24S}$), while the smallest speed-up was delivered by the $P_{\approx 12S}$, one of the best quality solution providers for the UTD problem.

Asynchronous PIMs did not present better accuracy than synchronous versions, but in relation to speed-up they were faster being the best speed-up provided by $P_{\approx 24A}$ (23.65), a model that computes non-competitive accuracy results. The *crossover* and *selection* parameters in synchronous PIMs are higher than those fixed for their asynchronous versions causing a slower breeding cycle, which increments the running time of synchronous PIMs.

5.4. Results and discussion for TMP

5.4.1. Experiments for TMP

For this experiment, sixty entries were processed, varying the used synthetic generated RTA number of tasks and flows. The sixty entries are divided into tens containing 30, 40, 50, 60, 70, and 80 tasks. For each PIM and $\mathcal{GA}_S$ using input I_{n_i} belonging to the set of ten inputs of length $n \in \{30, 40, 50, 60, 70, 80\}$, the following method was applied: each algorithm executed input I_{n_i} ten times; the average result of the ten runs was taken as result for the algorithm.

Radar charts in Figs. 11 and 12 illustrate the accuracy results for synchronous and asynchronous PIMs, respectively. The small radar charts illustrate how all PIMs improved the accuracy of the $\mathcal{GA}_S$.

5.4.2. Discussion on accuracy for TMP

The best results for static and dynamic topologies are provided by P_{N24S} and $P_{gbmm24S}$ (radar charts in Figs. 11 and 12). According to radar charts in Fig. 12(a) and (c), P_{To24A} and P_{Rd24A} were the best asynchronous static and dynamic PIMs, respectively. The best absolute results were provided by the synchronous dynamic model $P_{gbmm24S}$.

For synchronous PIMs, most (62.5%) 24-island models were better adapted to the problem than their 12-island versions as seen in Fig. 13, where contiguous gray and black bars represent outputs of corresponding 12- and 24-island models.

The model P_{Rd24A} provided the best results in asynchronous dynamic PIMs but was surpassed by P_{To24A} that computed the best outputs for asynchronous static PIMs. Also, the asynchronous models P_{R24A} and $P_{\approx 24A}$ provided competitive solutions. The synchronous static models that provided the worst solutions are P_{N12S} and P_{R12S} as also happens for the asynchronous models; indeed, the asynchronous PIMs that compute the worst solutions are P_{N12A} and P_{R12A} . Besides, for synchronous and asynchronous dynamic models, the worst solutions are computed by $P_{\approx 12S}$ and $P_{gbmm12S}$, and $P_{\approx 12A}$ and $P_{gbmm12A}$. Although this backwardness of 12-island models, some of them provided competitive solutions as P_{Tr12S} , P_{To12S} , and P_{Rd12S} for synchronous PIMs, and P_{Tr12A} , and P_{Rd12A} for asynchronous PIMs.

The synchronous dynamic PIMs provided, on average, better quality solutions than their asynchronous versions; the exception is the model P_{Rd24A} , which computes better solutions than the synchronous version. The scenario is different for static PIMs; versions from the tree topology showed similar results with 12-island and the synchronous models P_{To24S} , P_{R24S} and P_{C12S} were overcome by their respective asynchronous versions.

Table 5

Speed-up considering TMP for all PIMs (See Table 1).

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
14.78	9.55	5.81	12.01	6.9	15.35	7.28	14.63	14.51	15.09	8.25	12.05	15.04	11.98	10.7	8.49
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
8.77	10.97	8.69	8.9	12.56	8.55	5.66	8.79	13.97	7.88	8.61	6.71	14.47	11.78	15.26	12.77

Table 6Speed-up considering N -Queens problem for all PIMs (See Table 1).

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
12.54	9.34	10.9	8.62	9.06	9.04	10.38	8.7	11.86	8.63	9.75	10.03	11.67	8.42	11.86	9.47
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
12.56	9.57	12.92	9.08	9.49	9.52	11.72	9.5	10.23	10.21	13.95	12.45	12.62	8.97	12.72	8.83

5.4.3. Discussion on speed-up for TMP

The speed-up was computed using ten inputs with eighty tasks each. Each PIM and the sequential $\mathcal{GA}_S$ perform each input ten times, and the speed-up was computed as the total ratio average. Table 5 shows the speed-ups achieved for synchronous and asynchronous PIMs, where the best value is highlighted.

Their synchronous versions have overcome the running time of many asynchronous PIMs. This may be explained by the small values set for the breeding cycle parameters of many synchronous PIMs. The synchronous PIM P_{Tr24S} provided the best speed-up (15.35) and competitive outputs surpassing the accuracy of most than 50% of all PIMs. The synchronous PIM $P_{gbmm24S}$ that is the best-adapted to TMP obtained a small speed-up equal (8.49), being approximately 80% slower than P_{Tr24S} .

Regarding the number of islands, synchronous 12-island PIMs have speed-ups that were surpassed by their 24-island versions for the topologies torus, tree, ring, complete graph, and Random, and asynchronous 12-island PIMs were surpassed by their 24-island versions for the topologies net, torus, and ring. Besides, the worst speed-ups were obtained by the synchronous and asynchronous 12-island models P_{To12S} (5.81) and P_{R12A} (5.66), respectively. Better running time of 12-island models than those of their 24-island versions are explained by setting of crossover and/or selection.

5.5. Results and discussion for N -queens

5.5.1. Experiments for N -queens

For the experiments, thirty-one inputs with size $n \in \{50, 55, \dots, 195, 200\}$ were used, where n is the number of queens in an $n \times n$ board. For each entry with $n \in \{50, 55, \dots, 195, 200\}$ queens the following steps were performed: each PIM and the sequential $\mathcal{GA}_S$ were executed ten times; the averages of these ten executions were taken as the solution for each input provided.

Radar charts in Figs. 14 and 15 show the results for synchronous and asynchronous PIMs, respectively. For short, only results for $n \times n$ boards with $n \in \{140, 150, \dots, 190, 200\}$ are presented in these charts. Although, the discussion is based on all the experimental data provided in the appendix. It is worth noting that it was not possible to calculate the standard deviation since a single entry was used for each length. The small radar charts in these figures illustrate how all PIMs improved the accuracy of the $\mathcal{GA}_S$.

5.5.2. Discussion on accuracy for N -queens

The models P_{To12S} and P_{C12S} were the best synchronous static PIMs but P_{Tr24S} provided the best solutions for most of the (small size) inputs (radar chart (a) in Fig. 14), while P_{Rd12S} and $P_{\approx 12S}$ were the PIMs that

provided the best solutions among the synchronous dynamic models (radar chart (c) in Fig. 14). Also, synchronous dynamic 24-island models computed lower quality solutions than synchronous dynamic 12-island PIMs. In experiments involving asynchronous PIMs (radar charts (a) and (c) in Fig. 15), P_{C24A} , P_{R12A} and P_{To12A} were the static models that alternate the most competitive results, while $P_{\approx 12A}$ was the dynamic PIM that provided the most competitive solutions. Despite the model P_{Rd24A} surpasses $P_{\approx 12A}$ in the radar chart, this apparent advantage is not a fact when all experimental data is considered.

Fig. 16 shows the behavior of the synchronous PIMs P_{Tr24S} , P_{Rd12S} and $P_{\approx 12S}$. The experiments show a slight advantage of the synchronous static model P_{Tr24S} regarding the dynamic models P_{Rd12S} and $P_{\approx 12S}$ for small size inputs but, in general, the two synchronous dynamic PIMs deliver better solutions. For the best asynchronous PIMs regarding quality, the dynamic model $P_{\approx 12A}$ is superior to the static models.

Comparing static PIMs, the synchronous models P_{To12S} , P_{To24S} , P_{Tr12S} , P_{Tr24S} and P_{C12S} , and the asynchronous models P_{N12A} , P_{N24A} , P_{R12A} , P_{R24A} and P_{C24A} provided better solutions than their respective asynchronous and synchronous versions. Concerning dynamic PIMs, the synchronous models P_{Rd12S} and $P_{gbmm24S}$, and the asynchronous models P_{Rd24A} , $P_{\approx 12A}$, $P_{\approx 24A}$, and $P_{gbmm12A}$ surpassed their asynchronous and synchronous versions, respectively. Notice that the synchronous models P_{Rd12S} and $P_{\approx 24S}$ and their asynchronous versions provided results that alternate their quality. Regarding the number of islands, all dynamic 12-island models were better than their 24-island versions except $P_{gbmm12S}$, which is worse than $P_{gbmm24S}$ and P_{Rd12A} and P_{Rd24A} that provide very similar results. For synchronous static PIMs, only the 24-island models P_{Tr24S} and P_{R24S} are better than their 12-island versions. Considering asynchronous PIMs, only P_{Tr24A} and P_{C24A} surpassing 12-island versions.

5.5.3. Discussion on speed-up for N -queens

The speed-ups are estimated running the $\mathcal{GA}_S$ and each PIM ten times over a single 250-queens input. The speed-up is computed as the ratio average of these ten runs for each PIM. See Table 6 where the best speed-up is highlighted.

The best speed-ups were obtained by the synchronous and asynchronous 12-island PIMs P_{N12S} (12.54) and P_{Rd12A} (13.95). However, only P_{N12S} presented competitive accuracy. The PIMs with the best accuracy (P_{Tr24S} , P_{Rd12S} and $P_{\approx 12S}$, and P_{C24A} , and $P_{\approx 12A}$) are at the forefront of the worst running times; the exception is the asynchronous PIM $P_{\approx 12A}$ that delivered the fourth-best speed-up. In general, 12-island PIMs presented better speed-ups than their 24-island versions. Such behavior is explained by the values of the breeding cycle parameters, particularly the selection parameter.

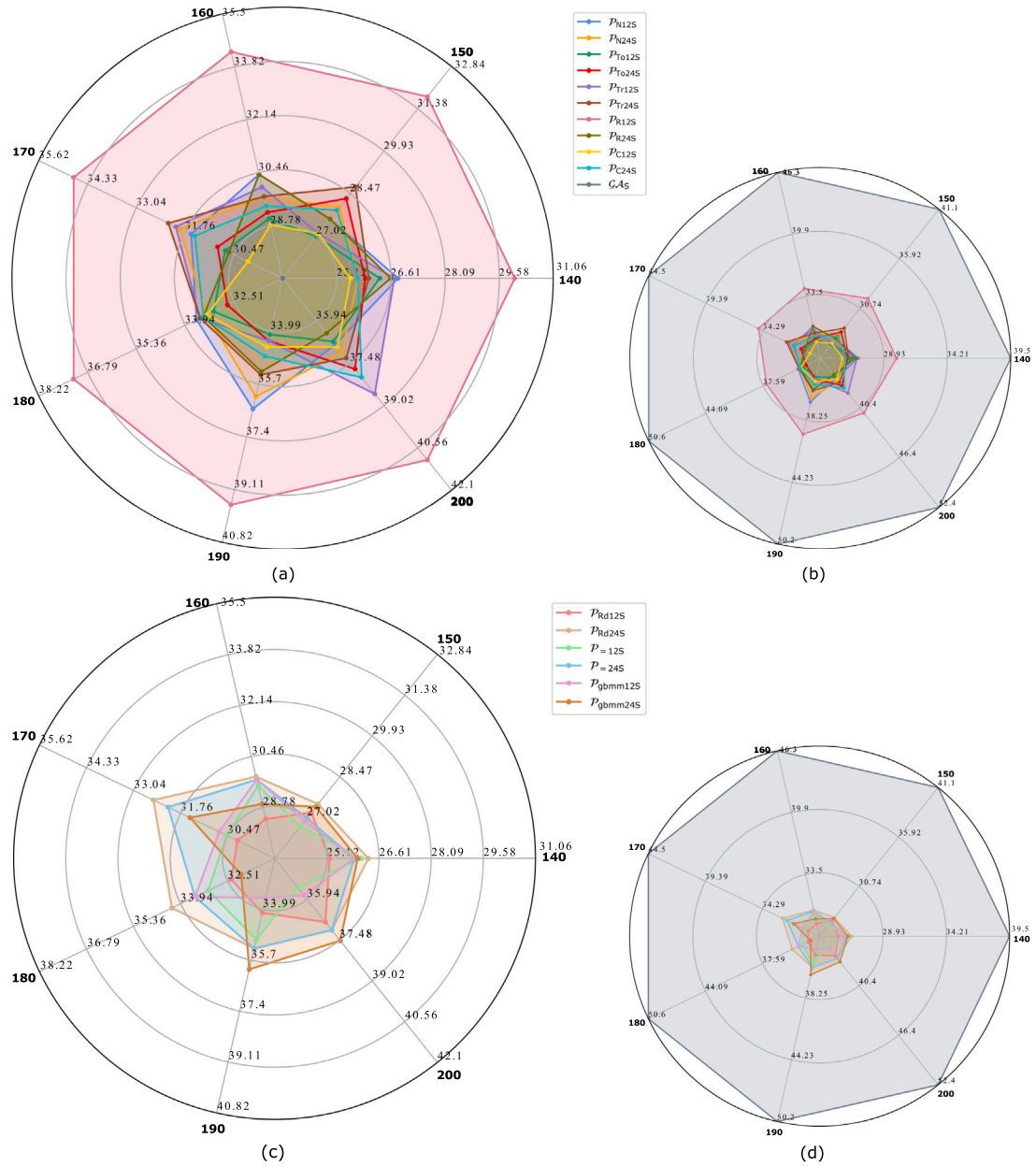


Fig. 14. The average scores of N -Queens for synchronous: (a) static PIMs; (b) static PIMs versus sequential $\mathcal{G}_S$; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential $\mathcal{G}_S$.

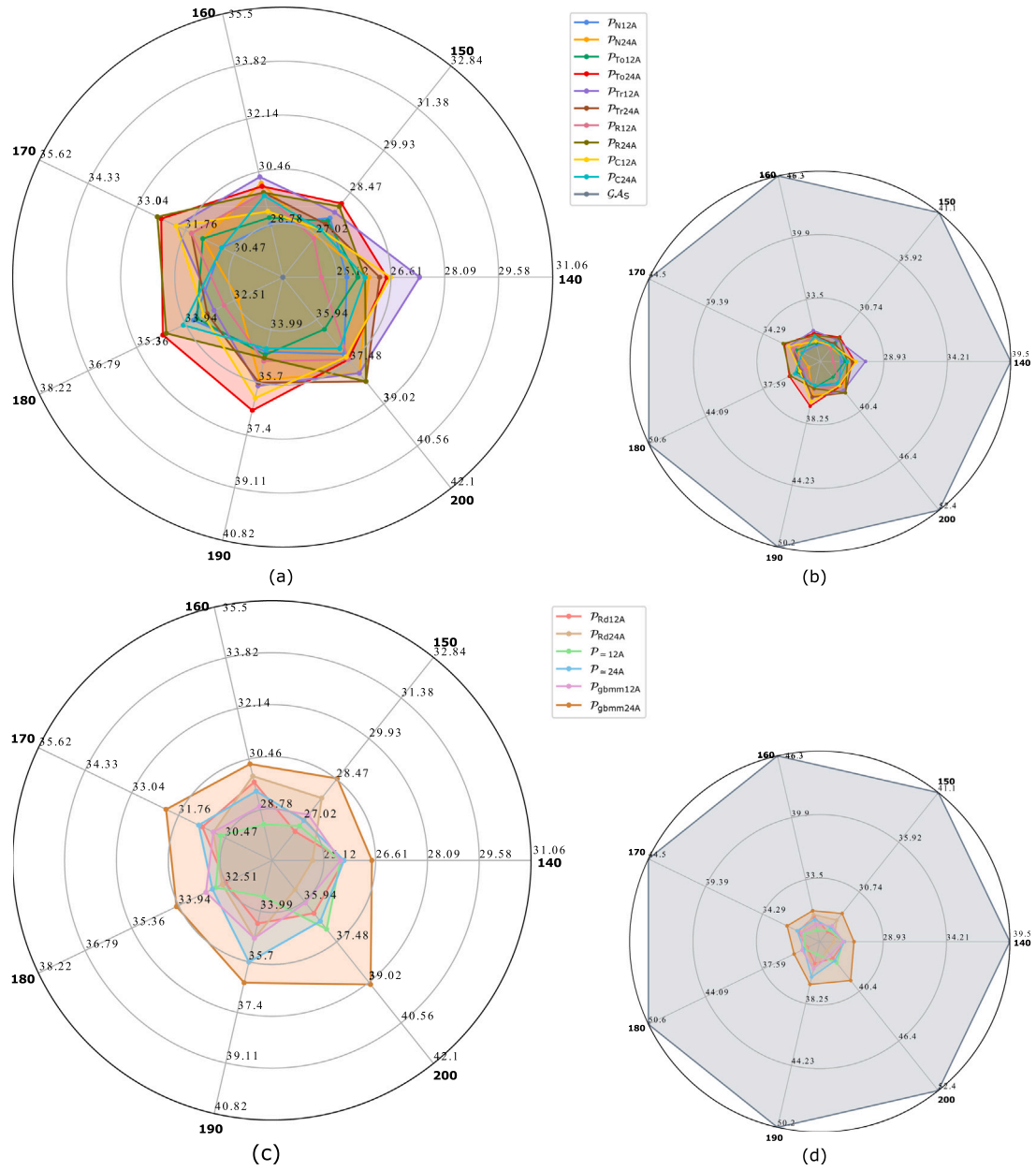


Fig. 15. The average scores of N -Queens for asynchronous: (a) static PIMs; (b) static PIMs versus sequential G_{AS} ; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential G_{AS} .

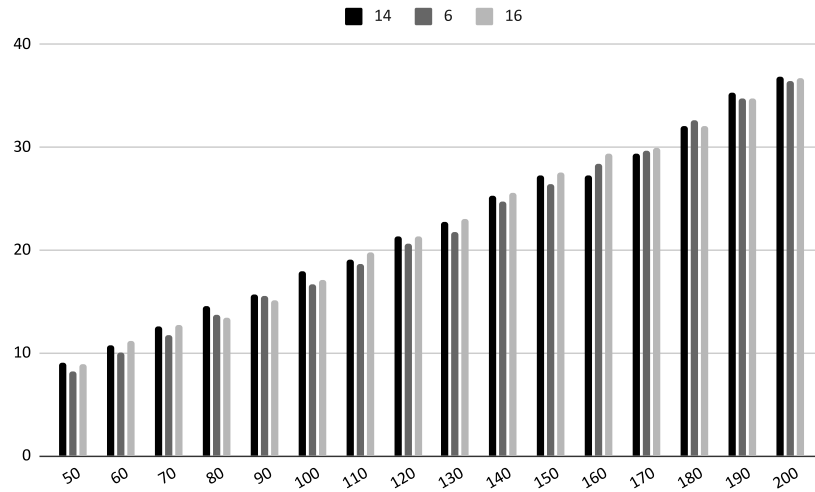


Fig. 16. Accuracy comparison of synchronous PIMs P_{Rd12S} , P_{Tr12S} , and P_{12S} for N -Queens.

Table 7

Holm test for URD.

L	Control Algorithm	i	Algorithm	Rank	P-value	α/i
100	P_{Tr12A} (Rank: 2.30)	4	$\mathcal{GA}_S$	4.89	5.04E-31	0.0125
		3	$P_{gbmm12S}$	2.67	0.1026	0.0167
		2	$P_{gbmm12A}$	2.66	0.112	0.025
		1	P_{Tr12S}	2.47	0.46	0.05
110	$P_{gbmm12S}$ (Rank: 2.435)	4	$\mathcal{GA}_S$	4.97	8.62E-30	0.0125
		3	P_{Tr12A}	2.65	0.3252	0.0167
		2	P_{Tr12S}	2.49	0.788	0.025
		1	$P_{gbmm12A}$	2.445	0.96	0.05
120	P_{Tr12S} (Rank: 2.35)	4	$\mathcal{GA}_S$	4.97	8.01E-32	0.0125
		3	$P_{gbmm12S}$	2.78	0.0517	0.0167
		2	P_{Tr12A}	2.47	0.592	0.025
		1	$P_{gbmm12A}$	2.42	0.75	0.05
130	P_{Tr12S} (Rank: 2.29)	4	$\mathcal{GA}_S$	5.00	1.09E-33	0.0125
		3	$P_{gbmm12S}$	2.73	0.0491	0.0167
		2	P_{Tr12A}	2.54	0.273	0.025
		1	$P_{gbmm12A}$	2.43	0.55	0.05
140	$P_{gbmm12S}$ (Rank: 2.41)	4	$\mathcal{GA}_S$	5.00	5.04E-31	0.0125
		3	P_{Tr12A}	2.64	0.3037	0.0167
		2	$P_{gbmm12A}$	2.49	0.737	0.025
		1	P_{Tr12S}	2.46	0.81	0.05
150	P_{Tr12S} (Rank: 2.26)	4	$\mathcal{GA}_S$	5.00	1.61E-34	0.0125
		3	$P_{gbmm12S}$	2.69	0.0517	0.0167
		2	P_{Tr12A}	2.54	0.210	0.025
		1	$P_{gbmm12A}$	2.51	0.27	0.05

5.6. General observations

Experiments with the four distinct $\mathcal{NP}$ -hard problems ratify several issues presented in the literature.

Our primary goal was to understand whether a PIM is running homogeneously the same BA in all its islands, considering the number of islands, synchronicity, and topology that provided a good adaptation regarding accuracy (and with competitive speed-up) for all supposed problems. The first ratified lesson is that the best-adapted model is entirely problem-dependent as the discussion on Sections 5.2.2, 5.3.2, 5.4.2, and 5.5.2 clarify. An interesting remark is that for the same

problem, a slight change in parameters may give rise to a significant combinatorial impact and, consequently, in the best-adapted model. For example, considering UTD, minor variations in the number of chromosomes hardly influenced PIM adaptation. Indeed, for UTD with 3 and 4 chromosomes, the best accuracy was obtained by static models (P_{R12S} and P_{N12S}), and with 5 chromosomes, the best-adapted model was a dynamic model (P_{12S}). This is synthesized in the mind map given in Fig. 17, where the best- and worst-adapted models for each problem, discriminating the number of chromosomes for UTD, are highlighted.

Although the focus is on the accuracy, all PIMs delivered substantial speed-ups, as given in Tables 3–6, ratifying that such parallel models are great strategies to deal with $\mathcal{NP}$ -hard problems.

The literature portrays that asynchronous models deliver better speed-ups than synchronous versions. However, our experiments show that this statement is valid only if all models have the same parameter configuration. The selection of parameters obtained in the calibration phase (directed towards better accuracy results) influences how fast a model runs. In particular, the parameters *selection*, *crossover*, *MigrationInterval* and *NumMigIndividuals* are sensitive regarding speed-up since they are responsible for controlling the overhead in the models. Instances of corresponding synchronous and asynchronous models in which the former ones present better speed-ups are discussed below.

- For URD, the synchronous model P_{R24S} shows better speed-up than the asynchronous model P_{R24A} , (inputs for models 8 and 24, resp., in Table 3) because the parameters *crossover* and *MigrationInterval* impose too much additional work in the asynchronous case (resp., 0.72 and 32% for the former and 0.98 and 10% for the latter model).
- For UTD, the models P_{Rd12S} and P_{Rd12A} also present better speed-up in the asynchronous case (inputs for models 11 and 27, resp., in Table 4) since the *selection* parameter for the synchronous model is 40% and for the asynchronous 72%.
- For N -Queens, considering the models 9 and 25, and 16 and 32, in Table 6, a similar situation occurs since the *selection* parameter is higher for the corresponding asynchronous models: 54% and 78%, in both cases.

The evaluation of migration topologies shows the potential of dynamic topologies when the focus is to enhance exploitation and exploration. Indeed, for URD, $P_{gbmm12S}$ was the third best; for UTD with

5 chromosomes, $P_{\approx 12S}$ was the best; for TMP, the best solutions were delivered by $P_{gbmm24S}$ and for N -Queens, $P_{\approx 12A}$ was the best fit for large entries, whereas P_{Tr24S} delivered better results for small ones.

The migration parameters show that in the most frequent configurations, less than six *best* individuals emigrate and replace the *worst* individuals in the target islands. Regarding the emigration policy, experiments show a balance between successful PIMs cloning and removing emigrant policies. The speed-up decrements in the cloning-emigrant policy since it implies carrying out genotype similarity tests to prevent the inclusion of equal individuals in the population.

6. Statistical analysis

For validating the experiments' results, the non-parametric statistical Friedman test that accomplishes repeated-measures of ANOVA test [44,45] was applied. Besides, the Holm procedure was applied as a post hoc test to adjust the p-values and characterize the existence of pairwise differences within a multiple comparisons set-up [44] (see also [46,47]). Such statistical analysis approach was already applied in several related works such as [24,28,40,43]. Moreover, our purpose did not need the application of other tests such as the Page's Trend Test for analyzing algorithms' convergence [45,48], or the Bayesian tests as an alternative to Hypothesis Tests [45].

In our analysis, the statistical tests use the best static and dynamic models for synchronous and asynchronous PIMs. The sample for each model is the output experiment set. Except for TMP and N -Queens, for which another set of results has been obtained. The sample size for URD and UTD problems is 100, whereas for TMP and N -Queen problems is 50. Since all case studies are minimization problems, to apply the statistical tests appropriately, each output in a sample, x , was pre-processed computing its multiplicative inverse, $1/x$.

The methodology proposed in [44] is as follows:

- The Friedman test is applied to test the null hypothesis that the performance of all algorithms is the same over a given set of inputs.
- If the null hypothesis is rejected in the previous step then the Holm test is applied to test the null hypothesis that the performance of a control algorithm is the same concerning one of the other algorithms for a given set of inputs.

The Friedman and Holm tests are available in the CONTROLTEST package at <https://sci2s.ugr.es/sicidm>. The significance level used in the tests is $\alpha = 0.05$. The Holm test uses as control PIM the one with the best (minimum) rank obtained by the Friedman test.

The PIMs that appear as control algorithm in most cases for URD; UTD with 3, 4 and 5 chromosomes; TMP; and N -Queens are respectively, P_{Tr12S} ; P_{R12S} ; P_{N12S} and $P_{\approx 12S}$; $P_{gbmm24S}$; and, $P_{\approx 12A}$. The Friedman and Holm tests confirm the affirmations on accuracy given in Sections 5.2.2, 5.3.2, 5.4.2, and 5.5.2,

Table 7 shows the results of the Holm test for URD, for which P_{Tr12S} appears as control PIM in 3 cases out of 6. However P_{Tr12S} just has statistically significance difference regarding GAS .

Tables 8, 9, and 10 show the results of the Holm test for UTD considering 3, 4 and 5 chromosomes. From these tables, the PIMs that appear as control algorithms in most cases are, P_{R12S} for Table 8 (4 cases out of 6), P_{N12S} for Table 9 (4 cases out of 6), and $P_{\approx 12S}$ for Table 10 (5 cases out of 6).

Table 11 shows the results of the Holm test for TMP, where $P_{gbmm24S}$ appears as the control algorithm in all cases. The null hypothesis of the Friedman test for $L = 30$ was not rejected.

Table 12 shows the results of the Holm test for N -Queens. The sample consists of 50 instances of the problem varying from 5 to 250 (5, 10, ..., 250). Here, the model P_{Tr24S} appears as the control algorithm. Since the experiments were conducted with a larger amount of input due to the needs of the statistical methods used, the results presented in Table 12 confirm what had already been observed in the discussion on accuracy in Section 5: as the number of queens is increased, the asynchronous PIM $P_{\approx 12A}$ starts to stand out.

Table 8

Holm test for UTD considering genomes with 3 chromosomes.

L	Control Algorithm	i	Algorithm	Rank	P-value	α/i
100	P_{R12S} (Rank: 2.73)	5	GAS	6.00	5.48E-35	0.01
		4	$P_{\approx 12A}$	3.34	0.0222	0.0125
		3	P_{C12A}	3.24	0.0539	0.0167
		2	P_{N12S}	2.86	0.637	0.025
		1	$P_{\approx 12S}$	2.82	0.75	0.05
110	P_{N12S} (Rank: 2.63)	5	GAS	6.00	3.66E-37	0.01
		4	$P_{\approx 12A}$	3.48	0.0012	0.0125
		3	P_{C12A}	3.26	0.0164	0.0167
		2	P_{R12S}	2.87	0.354	0.025
		1	$P_{\approx 12S}$	2.74	0.66	0.05
120	$P_{\approx 12S}$ (Rank: 2.65)	5	GAS	6.00	9.63E-37	0.01
		4	$P_{\approx 12A}$	3.65	1.57E-4	0.0125
		3	P_{C12A}	3.07	0.1124	0.0167
		2	P_{N12S}	2.81	0.533	0.025
		1	P_{R12S}	2.81	0.53	0.05
130	P_{R12S} (Rank: 2.62)	5	GAS	6.00	2.88E-37	0.01
		4	$P_{\approx 12A}$	3.65	9.90E-5	0.0125
		3	P_{C12A}	3.07	0.0890	0.0167
		2	$P_{\approx 12S}$	2.92	0.265	0.025
		1	P_{N12S}	2.72	0.71	0.05
140	P_{R12S} (Rank: 2.36)	5	GAS	6.00	4.57E-43	0.01
		4	$P_{\approx 12A}$	3.77	8.88E-8	0.0125
		3	P_{C12A}	3.28	5.07E-4	0.0167
		2	P_{N12S}	2.85	0.061	0.025
		1	$P_{\approx 12S}$	2.73	0.16	0.05
150	P_{R12S} (Rank: 2.46)	5	GAS	6.00	1.02E-40	0.01
		4	$P_{\approx 12A}$	3.69	3.66E-6	0.0125
		3	P_{C12A}	3.36	6.70E-4	0.0167
		2	$P_{\approx 12S}$	2.86	0.131	0.025
		1	P_{N12S}	2.61	0.57	0.05

7. Related work

In addition to the references discussed in the introduction, related work includes exciting applications, from which we reference a few.

Saito et al. [49] presented a method for parallel speed-up of decomposition-based multi-objective evolutionary algorithms (MOEA/D). The technique uses many-core environments such as Graphics Processing Units (GPUs) and seeks to prevent degradation in the accuracy of solutions. Degradation is mitigated through the definition of a virtual overlapping zone between partitions and the selection of individuals. Weight vectors of adjacent partitions are used to evaluate individuals in this zone for mating and migration. The method is compared with sequential MOEA/D, no-migration parallel MOEA/D, and a parallel MOEA/D with standard migration. At a specific interval, the worst five solutions of each island are replaced by the five best solutions from the neighboring islands. The case study was a two-objective knapsack problem with constraints. Experiments were conducted in various scenarios, varying the number of islands, population size, and the number of evolutionary cycles. The results showed that the proposed method is effective in improving the diversity of solutions and running time.

A practical study to avoid failures of refrigerated showcases placed in convenience stores and supermarkets was proposed by Otaka et al. [50]. This paper presents a fault detection method by pasting-based artificial neural networks using parallel multi-population modified brain storm optimization and correntropy (see [51]). Variations with a static ring, trigonal pyramid, and cube topologies were used with an elitist migration policy where the best immigrants replace the worst natives every ten generations. The experiments were conducted with a small number of islands (two, four, and eight). Statistical tests showed the superiority of the four-island trigonal pyramid topology model, indicating a success rate of approximately 99.4% of the samples presented.

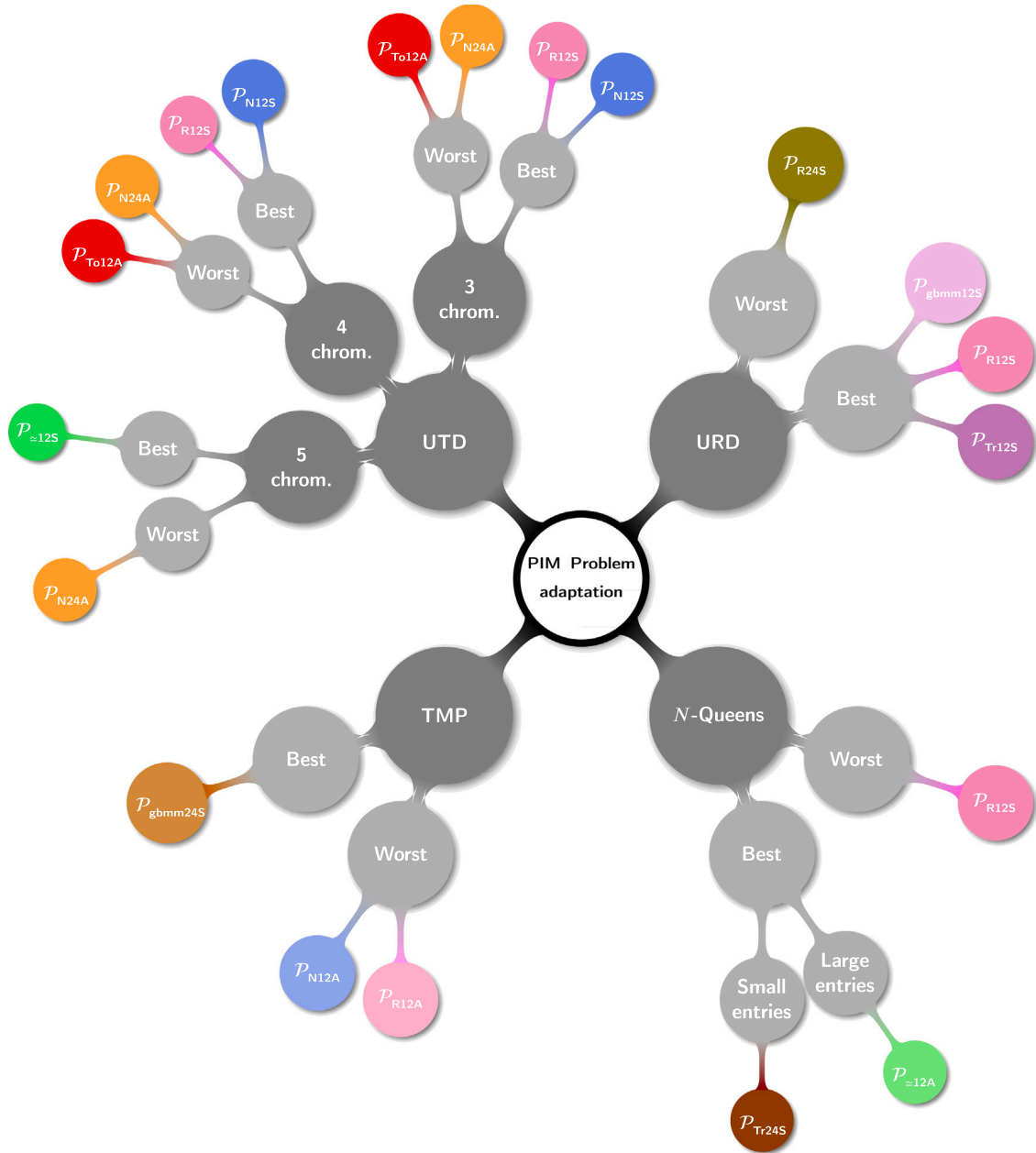


Fig. 17. Mind map on PIM model's adaptation to the NP-hard problems URD, UTD, TMP, and N-Queen. Summarized from the discussion on accuracy in Sections 5.2.2, 5.3.2, 5.4.2, and 5.5.2, respectively.

Table 9
Holm test for UTD considering genomes with 4 chromosomes.

L	Control Algorithm	i	Algorithm	Rank	P-value	α/i
100	P_{N12S} (Rank: 2.89)	5	G_{AS}	5.92	1.84E-30	0.01
		4	P_{C12A}	3.16	0.3075	0.0125
		3	$P_{\approx 12A}$	3.12	0.3847	0.0167
		2	P_{R12S}	2.99	0.705	0.025
		1	$P_{\approx 12S}$	2.91	0.92	0.05
110	P_{R12S} (Rank: 2.70)	5	G_{AS}	5.95	1.11E-34	0.01
		4	$P_{\approx 12A}$	3.25	0.0359	0.0125
		3	P_{C12A}	3.22	0.0494	0.0167
		2	P_{N12S}	3.00	0.257	0.025
		1	$P_{\approx 12S}$	2.87	0.508	0.05
120	P_{N12S} (Rank: 3.05)	5	G_{AS}	4.63	2.63E-9	0.01
		4	$P_{\approx 12A}$	3.68	0.0182	0.0125
		3	P_{C12A}	3.33	0.2899	0.0167
		2	$P_{\approx 12S}$	3.24	0.473	0.025
		1	P_{R12S}	3.05	1.00	0.05
130	P_{N12S} (Rank: 2.76)	5	G_{AS}	5.99	2.81E-34	0.01
		4	$P_{\approx 12A}$	3.34	0.0298	0.0125
		3	P_{C12A}	3.20	0.0963	0.0167
		2	P_{R12S}	2.90	0.597	0.025
		1	$P_{\approx 12S}$	2.79	0.92	0.05
140	P_{R12S} (Rank: 2.68)	5	G_{AS}	6.00	5.12E-36	0.01
		4	$P_{\approx 12A}$	3.41	0.0061	0.0125
		3	P_{C12A}	3.29	0.0211	0.0167
		2	P_{N12S}	2.85	0.533	0.025
		1	$P_{\approx 12S}$	2.76	0.78	0.05
150	P_{N12S} (Rank: 2.62)	5	G_{AS}	6.00	2.26E-37	0.01
		4	$P_{\approx 12A}$	3.66	8.47E-5	0.0125
		3	P_{C12A}	3.18	0.0327	0.0167
		2	$P_{\approx 12S}$	2.80	0.484	0.025
		1	P_{R12S}	2.73	0.68	0.05

Table 10
Holm test for UTD considering genomes with 5 chromosomes.

L	Control Algorithm	i	Algorithm	Rank	P-value	α/i
100	P_{R12S} (Rank: 2.97)	5	G_{AS}	5.75	7.99E-26	0.01
		4	P_{C12A}	3.16	0.4611	0.0125
		3	$P_{\approx 12A}$	3.12	0.5580	0.0167
		2	$P_{\approx 12S}$	3.00	0.910	0.025
		1	P_{N12S}	2.99	0.94	0.05
110	$P_{\approx 12S}$ (Rank: 2.86)	5	G_{AS}	5.95	1.63E-31	0.01
		4	$P_{\approx 12A}$	3.16	0.2568	0.0125
		3	P_{C12A}	3.12	0.3165	0.0167
		2	P_{R12S}	2.98	0.637	0.025
		1	P_{N12S}	2.92	0.82	0.05
120	$P_{\approx 12S}$ (Rank: 2.71)	5	G_{AS}	5.90	1.78E-33	0.01
		4	$P_{\approx 12A}$	3.40	0.0086	0.0125
		3	P_{C12A}	3.15	0.0963	0.0167
		2	P_{N12S}	2.92	0.416	0.025
		1	P_{R12S}	2.91	0.45	0.05
130	$P_{\approx 12S}$ (Rank: 2.84)	5	G_{AS}	5.95	6.68E-32	0.01
		4	$P_{\approx 12A}$	3.34	0.0563	0.0125
		3	P_{C12A}	3.12	0.2814	0.0167
		2	P_{N12S}	2.88	0.880	0.025
		1	P_{R12S}	2.86	0.94	0.05
140	$P_{\approx 12S}$ (Rank: 2.79)	5	G_{AS}	5.99	8.93E-34	0.01
		4	$P_{\approx 12A}$	3.33	0.0394	0.0125
		3	P_{C12A}	3.16	0.1564	0.0167
		2	P_{R12S}	2.87	0.748	0.025
		1	P_{N12S}	2.84	0.85	0.05
150	$P_{\approx 12S}$ (Rank: 2.63)	5	G_{AS}	6.00	3.66E-37	0.01
		4	$P_{\approx 12A}$	3.63	1.57E-4	0.0125
		3	P_{C12A}	3.20	0.0312	0.0167
		2	P_{R12S}	2.78	0.558	0.025
		1	P_{N12S}	2.75	0.64	0.05

Table 11
Holm test for TMP.

L	Control Algorithm	i	Algorithm	Rank	P-value	α/i
40	$P_{gbmm24S}$ (Rank: 2.45)	4	G_{AS}	4.88	1.54E-14	0.0125
		3	P_{Rd24A}	2.58	0.6810	0.0167
		2	P_{N24S}	2.55	0.752	0.025
		1	P_{To24A}	2.54	0.78	0.05
50	$P_{gbmm24S}$ (Rank: 2.08)	4	G_{AS}	5.0	2.61E-20	0.0125
		3	P_{To24A}	2.74	0.0369	0.0167
		2	P_{Rd24A}	2.74	0.037	0.025
		1	P_{N24S}	2.44	0.25	0.05
60	$P_{gbmm24S}$ (Rank: 1.76)	4	G_{AS}	4.80	7.02E-22	0.0125
		3	P_{Rd24A}	3.16	9.55E-6	0.0167
		2	P_{N24S}	2.64	0.005	0.025
		1	P_{To24A}	2.64	0.01	0.05
70	$P_{gbmm24S}$ (Rank: 1.64)	4	G_{AS}	5.0	2.27E-26	0.0125
		3	P_{Rd24A}	3.11	3.34E-6	0.0167
		2	P_{To24A}	2.65	0.001	0.025
		1	P_{N24S}	2.60	0.002	0.05
80	$P_{gbmm24S}$ (Rank: 1.56)	4	G_{AS}	4.99	2.07E-27	0.0125
		3	P_{Rd24A}	3.55	3.12E-10	0.0167
		2	P_{To24A}	2.52	0.002	0.025
		1	P_{N24S}	2.38	0.01	0.05

Table 12
Holm test for N-Queens.

N	Control Algorithm	i	Algorithm	Rank	P-value	α/i
5 to 250	$P_{\approx 12A}$ (rank: 2.67)	5	G_{AS}	6.00	5.59E-19	0.01
		4	P_{C24A}	3.50	0.0265	0.0125
		3	$P_{\approx 12S}$	3.10	0.2505	0.0167
		2	P_{T24S}	2.89	0.557	0.025
		1	P_{Rd12S}	2.84	0.65	0.05

Ohira et al. [52] proposed a homogeneous PIM from the GA for GPU deployment. The approach groups similar individuals on each island to focus on a specific area of the search space. The similarity was computed using the longest common subsequence distance [53], which seeks similarity without considering the positions of genes, in contrast with the use of genotype similarity in our approach, justified because it is more appropriate to the type of problems addressed. Unlike a migration guided by parameters, in Ohira et al. approach at each migration, the global population is grouped in subpopulations that are subsequently separated in islands according to their similarity so that the communication topology is based on similarity, and the migration interval is dependent on diversity. The approach is run on GPU and CPU showing speed-up improvements as the number of islands increases with more significant speed-up on the CPU version. Ohira et al. results support our observations regarding accuracy and speed-up since only speed-up was scalable, but the same scenario was not repeated for accuracy.

Skakovski and Jędrzejowicz propose an island model composed of islands of different sizes performing independently without any communication among themselves [54]. The model successfully executes differential evolution in their islands to solve the discrete-continuous scheduling problem with continuous resource discretization. Execution may be sequential. The authors recently improved the model by allowing dynamic adjustment of the island population size [55]. The reported advantages include eliminating fine-tuning the population size, choosing a topology, and a migration policy. Also, Doush et al. proposed an island-based model [56]. Still, they allowed migration policies as PIMs to successfully solve the flow shop scheduling with block problem. Using different island population sizes and their dynamic adjustments are bio-inspired features that may improve our models. But, from the experiments on PIMs, it is clear that maintaining the model diversity through migration and flexibility on the island communication topologies is an essential factor that needs to be explored.

Table 13

Parameter setup in synchronous PIMs to solve URD.

Parameter	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.92	.92	.90	.88	.96	.94	.96	.72	.90	.90	.98	.96	.96	.92	.98	.96
<i>mutation</i>	.014	.01	.01	.01	.01	.01	.01	.012	.01	.013	.01	.01	.01	.01	.01	.01
<i>selection</i>	80%	92%	92%	96%	98%	94%	90%	86%	90%	86%	96%	94%	84%	86%	78%	94%
<i>replacement</i>	60%	38%	38%	38%	50%	50%	60%	46%	50%	60%	38%	50%	40%	40%	50%	46%
<i>NumMigIndividuals</i>	2	1	3	1	3	2	3	5	1	4	6	4	7	10	2	5
<i>TypeEmIndividual</i>	1	2	2	2	1	3	2	2	3	3	1	3	2	3	3	3
<i>EmPolicy</i>	2	1	2	2	2	1	1	2	1	1	2	2	1	1	1	2
<i>TypeImIndividual</i>	1	1	2	1	2	2	1	1	2	1	1	1	2	1	2	3
<i>MigrationInterval</i>	10%	20%	94%	64%	56%	10%	50%	32%	10%	30%	88%	84%	20%	60%	30%	68%

Table 14

Parameter setup in asynchronous PIMs to solve URD.

Parameter	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.98	.82	.94	.94	.98	.94	.98	.98	.86	.92	.94	.94	.94	.94	.96	.94
<i>mutation</i>	.01	.012	.011	.011	.015	.013	.012	.01	.017	.016	.02	.017	.013	.015	.011	.016
<i>selection</i>	80%	84%	86%	86%	92%	86%	94%	82%	84%	80%	88%	86%	98%	86%	94%	86%
<i>replacement</i>	28%	70%	70%	60%	70%	70%	70%	50%	30%	70%	36%	30%	38%	70%	70%	70%
<i>NumMigIndividuals</i>	5	5	5	7	9	5	1	10	8	5	5	5	5	5	5	9
<i>TypeEmIndividual</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>EmPolicy</i>	2	2	2	2	2	2	2	2	1	1	2	2	1	2	2	1
<i>TypeImIndividual</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>MigrationInterval</i>	30%	30%	30%	30%	30%	30%	30%	10%	30%	30%	30%	30%	30%	30%	30%	30%

Table 15

Parameter setup in synchronous PIMs to solve UTD.

Parameter	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.88	.98	.96	.84	.92	.84	.96	.98	.94	.96	.94	.96	.94	.98	.96	.96
<i>mutation</i>	.014	.01	.01	.014	.01	.01	.01	.01	.01	.01	.012	.011	.01	.01	.01	.014
<i>selection</i>	96%	68%	82%	96%	80%	96%	94%	94%	96%	94%	40%	54%	98%	96%	48%	94%
<i>replacement</i>	60%	70%	68%	30%	44%	30%	60%	80%	30%	60%	70%	44%	70%	28%	66%	50%
<i>NumMigIndividuals</i>	3	5	8	7	8	6	9	2	5	4	3	5	9	7	6	5
<i>TypeEmIndividual</i>	1	1	1	1	1	1	3	2	1	1	1	1	3	1	3	1
<i>EmPolicy</i>	2	1	2	1	2	1	1	2	2	1	2	2	2	1	1	1
<i>TypeImIndividual</i>	2	1	1	2	1	2	1	2	1	2	1	1	3	3	2	3
<i>MigrationInterval</i>	32%	30%	42%	56%	90%	56%	10%	60%	30%	10%	10%	10%	10%	40%	40%	10%

Table 16

Parameter setup in asynchronous PIMs to solve UTD.

Parameter	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.64	.62	.92	.62	.66	.82	.68	.84	.86	.72	.72	.72	.74	.72	.72	.78
<i>mutation</i>	.01	.017	.013	.011	.011	.015	.016	.012	.01	.012	.011	.013	.016	.018	.011	.016
<i>selection</i>	50%	40%	40%	40%	66%	40%	64%	40%	70%	50%	72%	40%	64%	40%	68%	40%
<i>replacement</i>	70%	60%	70%	90%	70%	70%	70%	70%	64%	80%	90%	70%	70%	70%	28%	70%
<i>NumMigIndividuals</i>	5	1	5	1	5	4	1	4	5	1	1	2	5	13	5	1
<i>TypeEmIndividual</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>EmPolicy</i>	2	2	2	2	2	2	1	2	1	1	2	1	1	2	2	1
<i>TypeImIndividual</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>MigrationInterval</i>	30%	30%	30%	30%	30%	30%	30%	10%	10%	30%	30%	30%	30%	20%	30%	30%

Table 17

Parameter setup in synchronous PIMs to solve TMP.

Parameter	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.80	.94	.98	.92	.96	.86	.88	.96	.82	0.90	.98	.78	.80	.94	.80	.98
<i>mutation</i>	.014	.01	.011	.012	.012	.01	.01	.01	.01	.013	.011	.01	.01	.01	.01	.01
<i>selection</i>	40%	30%	90%	30%	76%	40%	78%	20%	40%	40%	62%	30%	40%	30%	40%	68%
<i>replacement</i>	70%	70%	36%	70%	70%	90%	70%	80%	16%	70%	78%	18%	66%	70%	68%	72%
<i>NumMigIndividuals</i>	5	5	5	6	2	5	1	2	5	1	5	5	5	5	5	5
<i>TypeEmIndividual</i>	1	1	3	1	1	1	1	3	1	1	1	1	1	1	1	3
<i>EmPolicy</i>	2	2	2	2	2	2	2	2	1	1	2	1	2	2	1	2
<i>TypeImIndividual</i>	1	1	1	1	1	1	1	2	1	1	1	1	1	1	1	1
<i>MigrationInterval</i>	30%	30%	74%	30%	66%	30%	26%	60%	32%	10%	28%	34%	30%	30%	28%	12%

Table 18

Parameter setup in asynchronous PIMs to solve TMP.

Parameter	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.80	.84	.92	.94	.96	.92	.92	.98	90%	.92	.92	.86	.80	.88	.80	.84
<i>mutation</i>	.011	.013	.012	.01	.014	.014	.017	.01	.018	.02	.02	.012	.014	.019	.011	.013
<i>selection</i>	66%	62%	64%	68%	42%	72%	86%	66%	40%	80%	64%	70%	42%	54%	40%	52%
<i>replacement</i>	38%	70%	36%	42%	36%	40%	70%	40%	38%	28%	70%	70%	70%	60%	70%	68%
<i>NumMigIndividuals</i>	5	5	5	5	7	5	3	4	4	5	1	5	5	3	5	4
<i>TypeEmIndividual</i>	3	1	2	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>EmPolicy</i>	2	2	1	1	1	1	2	1	1	1	2	2	2	1	1	1
<i>TypeImIndividual</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>MigrationInterval</i>	32%	30%	34%	28%	32%	30%	34%	26%	10%	10%	74%	28%	30%	10%	26%	12%

Table 19Parameter setup in synchronous PIMs to solve *N*-Queens.

Parameter	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.98	.98	.98	.98	.98	.98	.92	.96	.98	.98	.98	.98	.98	.98	.98	.98
<i>mutation</i>	.014	.01	.011	.01	.01	.01	.01	.01	.01	.011	.011	.01	.01	.01	.01	.01
<i>selection</i>	50%	54%	66%	74%	94%	60%	72%	80%	54%	70%	56%	40%	56%	54%	54%	54%
<i>replacement</i>	28%	50%	46%	60%	54%	54%	60%	74%	50%	70%	36%	26%	56%	44%	54%	56%
<i>NumMigIndividuals</i>	2	11	6	5	13	7	2	13	12	4	1	4	3	7	13	4
<i>TypeEmIndividual</i>	3	3	3	1	2	1	1	1	1	1	1	3	2	1	1	1
<i>EmPolicy</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2	1
<i>TypeImIndividual</i>	1	3	2	1	3	2	1	1	1	1	2	1	2	2	3	1
<i>MigrationInterval</i>	76%	12%	50%	50%	50%	10%	36%	46%	30%	10%	34%	54%	40%	30%	50%	20%

Table 20Parameter setup in asynchronous PIMs to solve *N*-Queens.

Parameter	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
	Static PIMs										Dynamic PIMs					
<i>crossover</i>	.98	.98	.98	.96	.98	.98	.96	.96	.98	.98	.98	.98	.98	.98	.98	.98
<i>mutation</i>	.01	.011	.015	.013	.014	.011	.013	.012	.01	.02	.01	.016	.011	.012	.014	.013
<i>selection</i>	68%	66%	54%	74%	94%	60%	68%	54%	78%	56%	52%	60%	54%	74%	54%	78%
<i>replacement</i>	54%	50%	46%	60%	70%	54%	60%	54%	70%	68%	36%	70%	70%	64%	54%	34%
<i>NumMigIndividuals</i>	2	11	4	12	9	7	2	13	12	4	1	4	4	5	13	1
<i>TypeEmIndividual</i>	3	3	3	1	1	1	1	1	1	1	3	3	3	1	1	2
<i>EmPolicy</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2	1
<i>TypeImIndividual</i>	1	3	2	1	3	2	1	1	1	1	2	1	2	2	3	1
<i>MigrationInterval</i>	54%	50%	50%	50%	20%	54%	26%	10%	32%	10%	10%	54%	56%	44%	60%	30%

Table 21

Parameter setup for the sequential GAs.

Parameter	URD	UTD	TMP	<i>N</i> -Queens
<i>crossover</i>	.90	.90	.90	.88
<i>mutation</i>	.02	.02	.016	.012
<i>selection</i>	60%	80%	92%	96%
<i>replacement</i>	60%	70%	38%	38%

Recently, other island-based optimization models were proposed applying bio-inspired algorithms such as the Cuckoo Search algorithm (CS), the Gray Wolf Optimizer (GWO), and the Whale Optimization Algorithm (WOA) (see, respectively, [57,58], and [59]). Abed-alguni et al. [60] introduced an island-based model running WOA on its islands called *i*WOA. It uses a dynamic random ring topology with a migration phase after a fixed number of generations. The model shows competitive performance regarding several island-based models running other bio-inspired meta-heuristics. The GWO algorithm is inspired by gray wolves' social hierarchy and behavior regarding encircling, hunting, attacking, and searching for prey [58]. At each iteration, the best solutions, classified according to their position regarding the prey's position, are considered as the wolves leading and guiding the pack. Although GWO considers random parameters concerning the trend of candidate solutions converging to or diverging from prey to guarantee exploitation and exploration, respectively, premature convergence is one issue of such a method. Attempting to

avoid premature convergence and increasing diversity in GWO, Abed-alguni et al. [61] proposed an island-based model called Distributed Gray Wolf Optimizer (DGWO), which was adapted to deal with discrete problems [62]. Finally, Abed-alguni et al. [63] proposed another island-based bio-inspired algorithm called *i*CSPM2, which runs CS. It was shown to be efficient when tested over continuous and discrete contexts. *i*CSPM2 uses an elite opposition-based learning method to explore the best solutions in each island's population and also apply four variations of CS synchronously over a unidirectional random ring topology. Experiments showed that this model is efficient compared to other evolutionary algorithms (including DGWO [61]).

Similar algorithmic approaches to the PIMs presented in this paper appeared in solving multi-objective optimization problems (MOP). For instance, Zhang et al. [64] introduce a multi-swarm optimizer that handles each objective function of a MOP with a different slave swarm. Simultaneously, a master swarm covers gaps among non-dominated optima by using a local multi-objective PSO algorithm. More recently, Gong et al. [65] proposed a framework to tackle dynamic interval MOPs that handles a cooperative co-evolutionary optimization based on interval similarity. Dynamic interval MOPs are MOPs with changing ranges of interval parameters in at least one objective or constraint over time. The framework divides decision variables into two groups according to the interval similarity and interval parameters. Two subpopulations are used to optimize decision variables in the two groups cooperatively. To solve dynamic interval multi-objective optimization problems, Xu et al. [66] proposed a model with EAs that also uses two subpopulations. Allowing different island populations,

Table 22

The average scores of URD with synchronous static PIMs: net, torus, tree, ring and complete graph (Numerical data for Fig. 6).

L	$\mathcal{GA}_S$		P_{N12S}		P_{N24S}		P_{T012S}		P_{T024A}		P_{T12S}		P_{T24S}	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	79.05	2.39	77.57	1.82	77.68	1.72	77.58	1.78	77.71	1.75	77.5	1.74	77.64	1.72
110	88.1	2.33	86.13	1.74	86.31	1.69	86.18	1.73	86.38	1.67	86.13	1.64	86.27	1.66
120	97.3	2.4	94.7	1.7	94.88	1.67	94.79	1.69	94.91	1.68	94.67	1.75	94.88	1.7
130	106.67	2.44	103.44	1.9	103.6	1.79	103.48	1.87	103.71	1.85	103.31	1.83	103.64	1.8
140	115.75	2.28	111.89	2.24	112.16	2.21	111.94	2.21	112.23	2.17	111.88	2.19	112.1	2.2
150	125.35	2.25	120.8	1.67	120.96	1.61	120.81	1.58	121.01	1.57	120.64	1.57	120.93	1.52

L	P_{R12S}		P_{R24S}		P_{C12S}		P_{C24S}	
	A	SD	A	SD	A	SD	A	SD
100	77.47	1.79	77.99	1.78	77.56	1.82	77.7	1.75
110	86.15	1.7	86.65	1.64	86.21	1.76	86.34	1.68
120	94.66	1.67	95.36	1.72	94.84	1.73	94.94	1.65
130	103.41	1.8	104.16	1.84	103.51	1.9	103.7	1.83
140	111.89	2.23	112.72	2.27	112.07	2.23	112.31	2.18
150	120.65	1.61	121.58	1.55	120.87	1.68	121.04	1.55

Table 23

The average scores of URD with synchronous dynamic PIMs (Numerical data for Fig. 6).

L	$\mathcal{GA}_S$		P_{Rd12S}		P_{Rd24S}		$P_{=12S}$		$P_{=24S}$		$P_{gbmm12S}$		$P_{gbmm24S}$	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	79.05	2.39	77.9	1.78	77.7	1.78	77.95	1.75	77.71	1.75	77.53	1.75	77.64	1.77
110	88.1	2.33	86.57	1.69	86.36	1.65	86.67	1.7	86.32	1.67	86.12	1.69	86.27	1.69
120	97.3	2.4	95.18	1.71	94.98	1.7	95.32	1.68	94.93	1.67	94.76	1.69	94.84	1.64
130	106.67	2.44	103.97	1.88	103.69	1.83	104.02	1.83	103.7	1.81	103.42	1.8	103.63	1.79
140	115.75	2.41	112.49	2.15	112.25	2.1	112.52	2.15	112.22	2.12	111.87	2.18	112.11	2.18
150	125.35	2.25	121.31	1.59	121.06	1.5	121.43	1.56	121.02	1.61	120.72	1.64	120.92	1.6

Table 24

The average scores of URD with asynchronous static PIMs: net, torus, tree, ring and complete graph (Numerical data for Fig. 7).

L	$\mathcal{GA}_S$		P_{N12A}		P_{N24A}		P_{T012A}		P_{T024A}		P_{T12A}		P_{T24A}	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	79.05	2.39	77.59	1.8	77.81	1.82	77.53	1.82	77.63	1.79	77.47	1.81	77.68	1.79
110	88.1	2.33	86.18	1.72	86.5	1.67	86.16	1.74	86.27	1.72	86.16	1.75	86.28	1.71
120	97.3	2.4	94.81	1.66	95.1	1.73	94.74	1.73	94.83	1.66	94.7	1.7	94.88	1.66
130	106.67	2.44	103.46	1.86	103.88	1.79	103.45	1.77	103.57	1.84	103.39	1.92	103.62	1.78
140	115.75	2.28	112.0	2.24	112.42	2.18	111.94	2.23	112.1	2.15	111.91	1.8	112.18	2.19
150	125.35	2.25	120.83	1.61	121.24	1.66	120.69	1.67	120.89	1.58	120.68	1.64	120.99	1.57

L	P_{R12A}		P_{R24A}		P_{C12A}		P_{C24A}	
	A	SD	A	SD	A	SD	A	SD
100	77.5	1.82	77.55	1.79	77.73	1.84	77.7	1.76
110	86.15	1.7	86.17	1.65	86.34	1.72	86.41	1.73
120	95.26	1.68	94.76	1.67	95.45	1.71	95.05	1.61
130	103.38	1.89	103.45	1.76	103.63	1.79	103.71	1.75
140	111.93	2.2	112.04	2.19	112.18	2.28	112.31	2.19
150	120.69	1.63	120.82	1.6	120.94	1.62	121.08	1.57

Table 25

The average scores of URD with asynchronous dynamic PIMs (Numerical data for Fig. 7).

L	$\mathcal{GA}_S$		P_{Rd12A}		P_{Rd24A}		$P_{=12A}$		$P_{=24A}$		$P_{gbmm12A}$		$P_{gbmm24A}$	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	79.05	2.39	77.56	1.81	77.71	1.79	77.5	1.8	77.63	1.78	77.54	1.76	77.67	1.75
110	88.1	2.33	86.17	1.71	86.37	1.66	86.14	1.73	86.28	1.74	86.13	1.65	86.26	1.67
120	97.3	2.4	94.72	1.66	94.96	1.71	94.67	1.71	94.94	1.66	94.66	1.65	94.93	1.67
130	106.67	2.44	103.47	1.9	103.68	1.86	103.42	1.86	103.69	1.81	103.36	1.76	103.66	1.84
140	115.75	2.41	111.97	2.18	112.23	2.1	111.89	2.16	112.19	2.17	111.91	2.2	112.19	2.18
150	125.35	2.25	120.8	1.63	121.06	1.49	120.67	1.66	121.03	1.56	120.66	1.6	121.02	1.58

Table 26

The average scores of UTD with synchronous PIMs considering genomes with 3, 4 and 5 chromosomes (Numerical data for Figs. 8, 9 and 10).

3 chromosomes																			
L	$\mathcal{GA}_S$		P_{N12S}		P_{N24S}		P_{To12S}		P_{To24S}		P_{Tr12S}		P_{Tr24S}		P_{R12S}		P_{R24S}		
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	
100	65.44	2.33	63.24	3.08	63.32	2.63	63.26	2.17	63.31	2.13	63.3	2.56	63.31	2.74	63.22	2.67	63.27	2.81	
110	72.94	2.21	70.7	2.92	70.84	2.56	70.75	2.88	70.83	2.85	70.76	2.91	70.85	2.97	70.73	2.92	70.82	2.95	
120	80.06	2.67	77.82	2.61	77.97	2.61	77.83	2.66	77.94	2.62	77.89	2.64	77.95	2.62	77.81	2.59	77.88	2.6	
130	86.83	2.78	85.55	2.77	85.71	2.74	85.55	2.69	85.71	2.77	85.62	2.68	85.72	2.76	85.52	2.67	85.66	2.72	
140	94.59	2.88	92.75	2.7	92.95	2.67	92.75	2.68	92.91	2.69	92.77	2.64	92.91	2.68	92.66	2.66	92.87	2.68	
150	102.10	2.75	100.29	2.57	100.55	2.63	100.28	2.59	100.53	2.61	100.34	2.56	100.52	2.59	100.27	2.58	100.46	2.69	
	P_{C12S}		P_{C24S}		P_{Rd12S}		P_{Rd24S}		$P_{\approx 12S}$		$P_{\approx 24S}$		$P_{gbmm12S}$		$P_{gbmm24S}$				
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD			
	63.26	2.77	63.28	2.73	63.36	2.84	63.41	2.86	63.24	2.62	63.28	2.69	63.38	2.56	63.26	2.51			
	70.78	2.91	70.8	2.92	70.91	2.77	71.0	2.72	70.8	2.92	70.81	2.76	70.87	2.98	70.76	2.93			
	77.83	2.57	77.91	2.64	78.02	2.62	78.13	2.66	77.8	2.6	77.92	2.71	77.97	2.64	77.84	2.59			
	85.55	2.67	85.66	2.73	85.78	2.8	85.91	2.79	85.59	2.73	85.63	2.53	85.78	2.77	85.57	2.7			
	92.76	2.67	92.85	2.64	93.02	2.71	93.2	2.75	92.72	2.65	92.91	2.66	92.99	2.68	92.8	2.67			
	100.35	2.64	100.48	2.61	100.63	2.63	100.85	2.68	100.34	2.59	100.52	2.97	100.57	2.62	100.4	2.6			
	4 chromosomes																		
	L	$\mathcal{GA}_S$		P_{N12S}		P_{N24S}		P_{To12S}		P_{To24S}		P_{Tr12S}		P_{Tr24S}		P_{R12S}		P_{R24S}	
A		SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	
100	61.53	2.46	61.07	2.76	61.1	2.73	61.07	2.86	61.1	2.84	61.09	2.66	61.1	2.75	61.09	2.65	61.09	2.64	
110	68.68	2.73	67.58	2.76	67.65	2.74	67.54	2.88	67.61	2.93	67.6	2.57	67.63	2.76	67.53	2.76	67.59	2.83	
120	75.46	2.81	74.68	2.78	74.82	2.85	74.72	2.86	74.75	2.92	74.71	2.77	74.74	2.82	74.7	2.66	74.76	2.82	
130	82.45	2.89	80.93	2.81	81.05	2.83	80.93	2.81	81.02	2.83	80.99	2.81	81.03	2.85	80.97	2.74	81.02	2.83	
140	89.94	2.82	87.79	2.91	88.13	2.89	88.0	2.89	88.11	2.91	88.02	2.67	88.1	2.98	87.96	2.97	88.08	2.99	
150	97.68	2.91	94.87	2.79	95.07	2.83	94.87	2.83	95.07	2.82	94.95	2.83	95.05	2.82	94.9	2.79	95.03	2.81	
	P_{C12S}		P_{C24S}		P_{Rd12S}		P_{Rd24S}		$P_{\approx 12S}$		$P_{\approx 24S}$		$P_{gbmm12S}$		$P_{gbmm24S}$				
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD			
	61.11	2.35	61.1	2.34	61.13	2.38	61.16	2.35	61.07	2.75	61.09	2.77	61.13	2.65	61.07	2.74			
	67.58	2.57	67.62	2.52	67.69	2.76	67.74	2.78	67.56	2.79	67.59	2.83	67.67	2.52	67.58	2.51			
	74.73	2.86	74.71	2.81	74.85	2.82	74.94	2.83	74.73	2.78	74.77	2.89	74.89	2.26	74.71	2.21			
	80.96	2.85	81.0	2.81	81.12	2.8	81.2	2.87	80.94	2.71	80.99	2.88	81.11	2.86	80.95	2.81			
	88.01	2.81	88.06	2.85	88.22	2.84	88.36	2.88	87.96	2.79	88.09	2.87	88.17	2.89	88.01	2.91			
	94.95	2.9	95.01	2.93	95.15	2.9	95.28	2.93	94.93	2.84	95.06	2.83	95.12	2.97	94.97	2.93			
	5 chromosomes																		
	L	$\mathcal{GA}_S$		P_{N12S}		P_{N24S}		P_{To12S}		P_{To24S}		P_{Tr12S}		P_{Tr24S}		P_{R12S}		P_{R24S}	
A		SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	
100	58.81	2.76	57.63	2.89	57.65	2.86	57.63	2.88	57.63	2.94	57.63	2.67	57.63	2.64	57.62	2.74	57.63	2.75	
110	65.02	2.83	64.58	2.9	64.61	2.94	64.55	2.91	64.61	2.87	64.58	2.89	64.61	2.84	64.59	2.89	64.56	2.84	
120	72.02	2.27	71.77	2.89	71.83	2.88	71.78	2.91	71.8	2.87	71.77	2.91	71.8	2.96	71.77	2.88	71.78	2.85	
130	78.79	2.69	77.79	2.94	77.91	2.93	77.78	2.91	77.85	2.9	77.81	2.97	77.85	2.89	77.79	2.91	77.83	2.9	
140	86.13	2.39	84.62	2.76	84.7	2.69	84.65	2.74	84.69	4.23	84.66	2.93	84.69	2.72	84.63	2.82	84.67	2.72	
150	92.5	2.83	90.74	2.89	90.88	2.81	90.78	2.85	90.88	2.78	90.83	2.87	90.9	2.78	90.74	2.82	90.85	2.79	
	P_{C12S}		P_{C24S}		P_{Rd12S}		P_{Rd24S}		$P_{\approx 12S}$		$P_{\approx 24S}$		$P_{gbmm12S}$		$P_{gbmm24S}$				
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD			
	57.63	2.85	57.64	2.83	57.67	2.69	57.66	2.74	57.62	2.86	57.62	2.82	57.66	2.87	57.62	2.84			
	64.57	2.89	64.59	2.85	64.65	2.94	64.71	2.9	64.56	2.92	64.6	2.64	64.62	2.89	64.57	2.84			
	71.78	2.78	71.8	2.77	71.86	2.87	71.88	2.78	71.74	2.91	71.82	2.87	71.83	2.77	71.75	2.86			
	77.84	2.96	77.86	2.93	77.89	2.99	77.95	2.94	77.79	2.94	77.84	2.77	77.91	2.91	77.81	2.88			
	84.65	2.88	84.68	2.82	84.77	2.93	84.83	2.86	84.61	2.95	84.68	2.84	84.72	2.82	84.61	2.79			
	90.78	2.84	90.84	2.82	90.96	2.87	91.1	2.88	90.71	2.81	90.82	2.75	90.92	2.85	90.8	2.87			

as in [65], and island algorithms, as in [63], PIMs may be adapted to handle such optimization problems.

Hashimoto et al. [67] proposed a PIM to solve multi-task problems. The number of islands and tasks is the same, and each island is responsible for evaluating a single objective. Migration is performed periodically, and a fixed number of migrant individuals are selected and removed randomly on each local island. The worst individuals are replaced, and immigrants have their fitness values set as the worst, assuming that feasible solutions for a task have great chances of being unsuitable for another objective. Experiments were carried out with two-objective problems obtaining similar or better results than those given by the sequential model, mainly when applying small migration intervals.

8. Conclusions and future work

To the best of our knowledge, no previous work addresses our methodology to carefully analyze each parameter involved in migration policies for synchronous and asynchronous PIMs. We use a thorough experimental approach to explore the potential of each migration policy. From this mindset, we proposed and analyzed homogeneous PIMs for four case studies: *Unsigned Reversal Distance*, *Unsigned Translocation Distance*, *Scheduling Task Mapping Problem*, and *N-Queens*. For each case study, synchronous and asynchronous 12- and 24-island PIMs were implemented using static topologies: net, torus, tree, ring, and complete graph, and three dynamic topologies based on the diversity and genotype of the native individuals. In addition, the parameters

Table 27

The average scores of UTD with asynchronous PIMs considering genomes with 3, 4 and 5 chromosomes (Numerical data for Figs. 20, 21 and 22).

3 chromosomes																		
L	$\mathcal{GA}_S$		P_{N12A}		P_{N24A}		P_{To12A}		P_{To24A}		P_{Tr12A}		P_{Tr24A}		P_{R12A}		P_{R24A}	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	65.44	2.33	63.48	2.23	63.63	2.17	63.35	2.22	63.6	2.27	63.38	2.24	63.47	2.2	63.4	2.24	63.41	2.16
110	72.94	2.21	71.02	2.11	71.24	2.28	70.82	2.09	71.23	2.07	70.92	2.08	71.03	2.02	70.93	2.01	71.01	2.04
120	80.06	2.67	78.17	2.82	78.46	2.62	78.06	2.78	78.47	2.78	78.07	2.79	78.21	2.7	78.05	2.75	78.12	2.66
130	86.83	2.78	85.96	2.87	86.32	2.48	85.77	2.9	86.28	2.88	85.86	2.89	85.97	2.79	85.84	2.83	85.93	2.75
140	94.59	2.88	93.26	2.84	93.67	2.35	93.1	2.75	93.63	2.77	93.09	2.79	93.36	2.75	93.09	2.82	93.22	2.75
150	102.10	2.75	100.87	2.85	101.38	2.55	100.6	2.69	101.36	2.76	100.75	2.76	100.99	2.74	100.71	2.78	100.82	2.66
	P_{C12A}		P_{C24A}		P_{Rd12A}		P_{Rd24A}		P_{s12A}		P_{s24A}		$P_{gbmm12A}$		$P_{gbmm24A}$			
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD		
	63.31	2.22	63.47	2.21	63.33	2.21	63.57	2.25	63.34	2.22	63.48	2.2	63.36	2.23	63.55	2.23		
	70.81	2.02	71.06	2.06	70.9	2.03	71.16	2.06	70.86	2.02	71.08	2.0	70.87	2.07	71.08	2.01		
	77.88	2.69	78.21	2.71	78.03	2.73	78.32	2.74	77.99	2.75	78.21	2.72	78.04	2.74	78.21	2.69		
	85.61	2.82	86.03	2.81	85.8	2.87	86.11	2.81	85.72	2.86	86.05	2.83	85.76	2.79	86.06	2.85		
	92.84	2.78	93.29	2.7	93.01	2.81	93.45	2.8	92.98	2.78	93.31	2.8	93.04	2.77	93.39	2.76		
	100.45	2.72	100.95	2.68	100.69	2.7	101.14	2.72	100.55	2.74	101.07	2.74	100.71	2.73	101.05	2.74		
4 chromosomes																		
L	$\mathcal{GA}_S$		P_{N12A}		P_{N24A}		P_{To12A}		P_{To24A}		P_{Tr12A}		P_{Tr24A}		P_{R12A}		P_{R24A}	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	61.53	2.76	61.18	2.78	61.27	2.8	61.13	2.78	61.24	2.79	61.11	2.86	61.23	2.67	61.15	2.63	61.17	2.78
110	68.68	2.73	67.71	2.63	67.93	2.75	67.68	2.62	67.92	2.61	67.67	2.64	67.81	2.59	67.66	2.57	67.77	2.63
120	75.46	2.81	74.93	2.85	75.2	2.81	74.86	2.71	75.17	2.83	74.82	2.89	74.99	2.78	74.82	2.79	74.94	2.73
130	82.45	2.89	81.14	2.81	81.45	2.72	81.14	2.93	81.47	2.96	81.13	2.95	81.27	2.9	81.11	2.93	81.23	2.89
140	89.94	2.82	88.29	2.84	88.6	2.75	88.22	2.71	88.62	2.85	88.18	2.79	88.44	2.82	88.21	2.78	88.36	2.76
150	97.68	2.91	95.27	2.85	95.71	2.82	95.15	2.87	95.66	2.79	95.17	2.76	95.41	2.83	95.19	2.86	95.27	2.73
	P_{C12A}		P_{C24A}		P_{Rd12A}		P_{Rd24A}		P_{s12A}		P_{s24A}		$P_{gbmm12A}$		$P_{gbmm24A}$			
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD		
	61.11	2.76	61.2	2.75	61.1	2.77	61.2	2.87	61.22	2.78	61.17	2.85	61.13	2.74	61.2	2.77		
	67.62	2.61	67.81	2.67	67.7	2.89	67.84	2.76	67.63	2.71	67.8	2.67	67.69	2.68	67.82	2.61		
	74.74	2.88	74.98	2.86	74.89	2.72	75.1	2.83	74.79	2.74	75.03	2.71	74.82	2.89	75.02	2.93		
	81.02	2.87	81.3	2.91	81.09	2.92	81.38	2.92	81.05	2.9	81.25	2.89	81.06	2.92	81.34	2.93		
	88.08	2.87	88.41	2.85	88.25	2.72	88.56	2.78	88.11	2.67	88.38	2.74	88.23	2.83	88.48	2.92		
	95.01	2.61	95.44	2.69	95.16	2.71	95.49	2.79	95.1	2.88	95.43	2.71	95.13	2.85	95.45	2.67		
5 chromosomes																		
L	$\mathcal{GA}_S$		P_{N12A}		P_{N24A}		P_{To12A}		P_{To24A}		P_{Tr12A}		P_{Tr24A}		P_{R12A}		P_{R24A}	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
100	58.81	2.76	57.68	2.57	57.76	2.62	57.67	2.69	57.71	2.61	57.65	2.72	57.69	2.78	57.65	2.69	57.67	2.57
110	65.02	2.83	64.65	2.96	64.78	2.79	64.63	2.93	64.73	2.94	64.66	2.92	64.67	2.92	64.62	2.98	64.63	2.9
120	72.02	2.27	71.89	2.18	72.03	2.8	71.87	2.76	72.02	2.58	71.86	2.96	71.95	2.75	71.84	2.56	71.9	2.61
130	78.79	2.69	77.95	2.64	78.16	2.81	77.92	2.98	78.15	2.55	77.93	2.96	78.07	2.7	77.91	2.81	78.02	2.96
140	86.13	2.39	84.8	2.36	85.12	2.24	84.79	2.34	85.01	2.32	84.74	2.31	84.9	2.3	84.74	2.32	84.89	2.28
150	92.50	2.83	91.05	2.95	91.4	2.79	90.97	2.89	91.36	2.97	90.99	2.89	91.18	2.87	90.94	2.9	91.11	2.89
	P_{C12A}		P_{C24A}		P_{Rd12A}		P_{Rd24A}		P_{s12A}		P_{s24A}		$P_{gbmm12A}$		$P_{gbmm24A}$			
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD		
	57.65	2.69	57.71	2.49	57.68	2.5	57.72	2.5	57.65	2.48	57.68	2.48	57.66	2.48	57.67	2.48		
	64.62	2.91	64.69	2.9	64.67	2.9	64.71	2.94	64.61	2.92	64.66	2.89	64.65	2.89	64.71	2.91		
	71.82	2.13	71.93	2.62	71.87	2.65	72.03	2.67	71.86	2.76	71.94	2.54	71.86	2.63	71.95	2.46		
	77.85	2.94	78.04	2.96	77.91	2.75	78.12	2.62	77.89	2.8	78.06	2.99	77.9	2.98	78.09	2.56		
	84.69	2.28	84.91	2.3	84.79	2.31	84.96	2.25	84.71	2.3	84.88	2.31	84.73	2.31	84.95	2.31		
	90.82	2.84	91.15	2.89	90.99	2.93	91.25	2.87	90.91	2.82	91.15	2.92	90.99	2.93	91.23	2.97		

were properly and separately calibrated for each model, aiming to obtain good quality results and reliable feedback on each different PIM's adequacy.

From the experiments, it is observed that each PIM requires specific breeding cycle and migration parameters to achieve satisfactory accuracy having an impact on its running time; in particular, the setup of migration parameters resulted in very different values. Nevertheless, for all PIMs, the adequate parameter setup provided satisfactory speed-ups and always better accuracy than the one provided by the sequential GA. Thus, adequate parameter tuning determines a suitable integration of the breeding cycle and the migration process, which is crucial to successfully adapting each PIM to each case study.

Experiments show a diverse adaption of PIM features and their dependency on the problems as synthesized in the mind map in Fig. 17. Indeed, synchronous dynamic 24-island PIMs achieved the best accuracy for TMP, while synchronous static and dynamic 12-island PIMs provided the best URD and UTD accuracy. In contrast, an asynchronous dynamic 12-island PIM delivered competitive solutions for large instances of N -Queens. For URD, UTD, and N -Queens, the speed-ups of most asynchronous PIMs are better than those of their synchronous versions. The contrary happens for TMP because of the extra exhaustive computational effort implied by parameter setting. However, for URD and TMP, the best speed-ups were provided by synchronous PIMs; moreover, for URD and UTD, most 24-island PIMs provided better speed-ups than their 12-island versions. For TMP, the best speed-ups

Table 28

The average scores of TMP with synchronous static PIMs using topologies: net, torus, tree, ring and complete graph (Data for Fig. 11).

L	$\mathcal{GA}_S$		$\mathcal{P}_{N12S}$		$\mathcal{P}_{N24S}$		$\mathcal{P}_{To12S}$		$\mathcal{P}_{To24S}$		$\mathcal{P}_{Tr12S}$		$\mathcal{P}_{Tr24S}$	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
30	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
40	0.35	0.67	0.24	0.46	0.03	0.11	0.04	0.13	0.03	0.11	0.0	0.0	0.0	0.0
50	4.25	2.4	2.56	2.31	1.7	1.91	1.86	1.88	1.9	2.27	1.9	2.1	2.2	2.24
60	10.9	2.39	8.34	2.22	6.93	2.38	7.52	2.27	7.5	2.34	7.42	2.3	7.37	1.98
70	19.9	2.26	16.6	3.29	15.27	2.87	15.52	3.0	15.63	3.35	15.36	2.78	15.5	2.81
80	29.15	2.53	22.9	2.63	21.2	2.46	21.27	2.78	21.3	2.87	21.53	2.84	21.57	2.89

$\mathcal{P}_{R12S}$		$\mathcal{P}_{R24S}$		$\mathcal{P}_{C12S}$		$\mathcal{P}_{C24S}$	
A	SD	A	SD	A	SD	A	SD
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.04	0.13	0.1	0.22	0.18	0.4	0.0	0.0
3.14	2.43	1.72	2.0	2.3	2.2	1.71	1.99
7.96	2.21	7.07	2.1	8.22	2.14	7.0	2.09
16.04	2.92	15.03	3.4	16.26	3.17	15.2	3.31
22.34	2.56	21.40	2.60	22.47	2.54	21.5	2.45

Table 29

The average scores of TMP with synchronous dynamic PIMs (Data for Fig. 11).

L	$\mathcal{GA}_S$		$\mathcal{P}_{Rd12S}$		$\mathcal{P}_{Rd24S}$		$\mathcal{P}_{\approx 12S}$		$\mathcal{P}_{\approx 24S}$		$\mathcal{P}_{gbmm12S}$		$\mathcal{P}_{gbmm24S}$	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
30	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
40	0.35	0.67	0.06	0.13	0.12	0.25	0.14	0.27	0.0	0.0	0.12	0.27	0.0	0.0
50	4.25	2.4	1.82	1.97	2.08	2.25	2.42	2.19	1.63	1.99	2.3	2.35	1.6	1.97
60	10.9	2.39	7.44	2.14	7.92	2.2	8.28	2.32	7.07	2.08	8.02	2.28	6.88	1.97
70	19.9	2.26	15.4	2.9	16.3	3.23	16.24	2.93	15.43	3.17	16.36	2.94	14.54	2.99
80	29.15	2.93	21.2	2.47	21.23	2.65	22.83	2.35	21.33	2.74	22.46	2.37	21.14	2.49

Table 30

The average scores of TMP with asynchronous static PIMs using topologies: net, torus, tree, ring and complete graph (Data for Fig. 12).

L	$\mathcal{GA}_S$		$\mathcal{P}_{N12A}$		$\mathcal{P}_{N24A}$		$\mathcal{P}_{To12A}$		$\mathcal{P}_{To24A}$		$\mathcal{P}_{Tr12A}$		$\mathcal{P}_{Tr24A}$	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
30	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
40	0.35	0.67	0.18	0.45	0.03	0.11	0.0	0.0	0.0	0.0	0.02	0.06	0.0	0.0
50	4.25	2.4	2.36	2.15	2.03	2.08	1.94	2.0	1.8	2.09	1.88	2.17	1.73	2.02
60	10.9	2.39	8.22	2.18	7.63	2.1	7.58	2.24	6.9	2.2	7.18	2.22	7.23	2.0
70	19.9	3.26	16.58	3.14	15.97	3.23	15.7	3.04	14.87	2.79	15.28	2.94	15.03	3.28
80	29.15	2.53	23.1	2.17	21.93	2.83	21.97	2.3	21.37	2.53	21.63	2.45	21.57	2.5

$\mathcal{P}_{R12A}$		$\mathcal{P}_{R24A}$		$\mathcal{P}_{C12A}$		$\mathcal{P}_{C24A}$	
A	SD	A	SD	A	SD	A	SD
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.3	0.64	0.03	0.0	0.07	0.21	0.03	0.11
1.57	1.75	1.73	2.12	1.86	1.95	1.53	1.8
8.87	2.13	6.87	2.14	7.48	2.2	7.3	2.11
15.83	3.24	14.9	3.03	15.4	3.03	15.23	3.06
24.33	2.62	21.43	2.49	21.93	2.79	21.67	2.35

Table 31

The average scores of TMP with asynchronous dynamic PIMs (Data for Fig. 12).

L	$\mathcal{GA}_S$		$\mathcal{P}_{Rd12A}$		$\mathcal{P}_{Rd24A}$		$\mathcal{P}_{\approx 12A}$		$\mathcal{P}_{\approx 24A}$		$\mathcal{P}_{gbmm12A}$		$\mathcal{P}_{gbmm24A}$	
	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD	A	SD
30	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
40	0.35	0.67	0.05	0.11	0.04	0.09	0.12	0.32	0.03	0.07	0.1	0.22	0.1	0.25
50	4.25	2.4	2.02	2.2	1.64	1.82	2.48	2.35	1.9	1.97	2.42	2.22	1.96	2.03
60	10.9	2.39	7.42	2.2	7.26	2.31	8.32	2.29	7.37	2.2	8.32	2.26	7.34	2.16
70	19.9	3.26	15.46	2.93	15.32	2.96	16.16	2.89	15.77	3.17	16.44	3.21	15.54	3.0
80	29.15	2.93	21.68	2.67	21.74	2.49	22.3	2.36	21.79	2.48	22.8	2.55	21.68	2.45

Table 32
The average scores of N -Queens with synchronous static PIMs (Data in Fig. 14).

N	$\mathcal{GA}_S$	P_{N12S}	P_{N24S}	P_{N12S}	P_{N24S}	P_{N12S}	P_{N24S}	P_{N12S}	P_{N24S}	P_{N12S}	P_{N24S}
50	18.3	12.2	11.4	11.2	11.0	11.5	9.7	13.5	10.5	11.1	10.7
55	20.7	12.3	11.5	12.2	11.8	11.9	10.7	13.9	12.1	11.3	11.7
60	22.2	12.5	12.5	12.9	12.6	13.6	11.4	15.1	12.8	12.7	12.3
65	24.9	14.3	13.7	14.1	13.3	14.8	12.7	15.7	13.4	13.6	12.8
70	25.1	15.0	14.1	15.1	14.5	16.1	14.0	17.6	14.9	14.1	14.1
75	28.8	16.0	15.2	15.9	14.5	15.8	14.2	17.7	14.6	15.5	15.3
80	27.1	16.1	16.5	15.9	15.7	17.0	14.8	18.8	15.5	15.7	15.8
85	31.7	17.3	17.0	16.6	15.7	17.5	15.8	19.8	16.9	17.2	16.9
90	29.0	19.2	16.9	17.1	17.3	18.6	16.2	21.2	17.0	17.5	17.6
95	29.6	19.1	18.5	18.1	18.4	18.6	18.7	21.6	18.5	19.6	18.1
100	33.3	19.3	19.1	19.0	19.4	19.7	18.4	22.9	18.9	18.7	19.1
105	31.6	20.8	19.6	20.3	20.6	20.5	19.3	23.7	19.7	20.4	19.8
110	35.2	21.7	20.1	20.9	20.7	20.9	20.8	24.9	20.4	20.9	21.0
115	42.5	22.0	21.4	21.3	21.9	22.6	21.3	24.8	21.9	21.1	21.3
120	35.0	22.5	22.6	22.8	22.3	23.1	21.9	25.8	22.5	22.2	22.6
125	35.2	24.4	24.2	23.5	23.6	24.4	22.7	27.5	23.4	24.3	23.2
130	38.9	24.6	24.3	23.8	24.2	24.9	23.4	26.8	24.5	24.6	24.3
135	39.6	24.9	25.2	25.2	25.3	26.4	25.1	29.2	25.1	24.3	25.1
140	39.5	26.8	25.7	26.3	25.9	26.7	26.0	30.0	26.6	25.5	25.7
145	39.2	26.7	27.6	25.8	26.6	26.9	25.8	28.5	27.5	25.9	26.8
150	41.1	27.6	28.0	27.0	28.3	27.1	28.7	31.8	27.6	27.1	27.9
155	45.8	29.5	28.7	29.5	28.6	29.2	29.2	33.5	28.4	28.0	28.7
160	46.3	30.4	29.7	29.0	29.2	30.0	29.7	34.3	30.4	28.8	29.4
165	44.1	31.0	31.0	30.2	31.2	30.7	31.9	33.9	30.5	30.3	31.0
170	44.5	31.6	31.9	30.7	30.9	32.0	32.2	34.7	30.6	30.1	31.5
175	44.7	31.9	31.8	31.8	31.6	33.3	32.1	34.3	32.9	32.8	32.5
180	50.6	33.6	33.3	33.1	32.7	33.5	33.5	37.2	33.4	33.3	33.4
185	47.5	34.6	34.8	33.1	33.9	34.4	33.9	37.3	34.4	32.9	34.3
190	50.2	36.5	36.1	34.1	34.3	34.3	35.4	39.6	35.3	34.5	34.8
195	55.5	37.0	35.8	36.5	35.7	35.5	36.3	40.4	36.7	35.4	35.4
200	52.4	36.8	37.1	36.7	37.7	38.6	37.3	41.0	36.4	36.9	38.0

Table 33
The average scores of N -Queens with synchronous dynamic PIMs (Data in Fig. 14).

N	$\mathcal{GA}_S$	P_{Rd12S}	P_{Rd24S}	P_{N12S}	P_{N24S}	$P_{gbmm12S}$	$P_{gbmm24S}$
50	18.3	11.1	11.5	11.0	10.3	12.3	11.0
55	20.7	11.2	11.9	12.1	11.9	13.2	11.9
60	22.2	12.8	13.1	13.4	12.5	12.6	12.7
65	24.9	13.1	14.1	13.5	13.2	16.7	13.7
70	25.1	14.5	13.9	14.2	13.6	15.1	13.9
75	28.8	14.1	15.2	15.1	14.6	17.9	15.3
80	27.1	15.5	16.0	15.4	15.6	16.1	16.4
85	31.7	16.6	16.6	17.1	16.3	19.9	16.7
90	29.0	17.6	17.5	16.9	17.6	17.2	17.6
95	29.6	17.7	18.1	18.0	18.1	20.8	18.6
100	33.3	19.9	19.5	19.3	19.3	19.3	18.7
105	31.6	19.4	20.3	19.5	20.2	23.2	19.9
110	35.2	21.0	21.3	20.6	21.4	20.4	21.5
115	42.5	21.4	22.4	22.0	21.7	23.8	22.5
120	35.0	22.7	22.5	22.3	22.2	23.0	22.8
125	35.2	23.8	23.1	22.9	23.0	24.2	23.7
130	38.9	24.3	24.1	24.6	24.9	24.1	25.1
135	39.6	24.3	25.0	24.9	25.0	26.9	24.5
140	39.5	25.2	26.3	26.1	25.9	25.9	26.0
145	39.2	26.8	27.4	26.5	27.1	30.1	27.1
150	41.1	27.2	27.5	26.7	27.0	26.9	27.4
155	45.8	28.3	28.9	28.6	28.9	30.4	29.2
160	46.3	28.4	29.8	29.5	29.7	29.7	28.9
165	44.1	30.0	30.7	30.6	30.4	33.3	30.9
170	44.5	30.2	32.5	30.5	32.1	30.7	31.5
175	44.7	31.9	32.2	31.3	32.1	34.8	31.8
180	50.6	32.4	34.2	33.1	33.5	33.5	32.1
185	47.5	32.9	35.2	33.4	34.8	35.3	34.1
190	50.2	34.1	35.3	35.0	35.3	33.6	36.0
195	55.5	36.4	36.1	35.7	35.6	39.2	35.9
200	52.4	36.8	37.1	35.5	37.1	35.8	37.5

were provided by 24-island PIMs, but in several cases, 12-island PIMs presented better speed-ups than their 24-island versions, while for N -Queens, the best speed-up was provided by a 12-island PIM and the majority of 12-island PIMs gave better speed-ups than their 24-island versions.

Our experiments ratify well-known facts on restrictions of PIMs. For instance, tuning the parameters of the breeding cycle and migration policies is vital to guarantee the well-adaptation of PIMs; the number of islands is problem dependent; the performance does not depend only on the synchrony of the model but also on the parameters; the selection between static and dynamic neighboring is also problem dependent, and that model adaptation also depends on the input size of the problem. Moreover, our experiments ratify that to efficiently address any optimization problem with PIMs, well-calibrated experiments should be performed with various evolutionary algorithms before deciding which is the best-adapted model. All that makes of the highest interest the future exploration of PIMs that increase diversity by dynamically and adaptatively applying more than one evolutionary algorithm in each of their islands.

Current work explores heterogeneous PIMs, in which islands may run different algorithms. There is a variety of EAs that can apply in such heterogeneous PIMs, such as particle swarm optimization [68], artificial bee colony [69], shuffled frog leaping [70], elephant herding behavior [71], social spider algorithm [72], differential evolution [73], among others. The research requires the initial development of sequential versions for these EAs specialized for each different case study and the subsequent development of heterogeneous PIMs for which migration policies and speed-up and accuracy need to be investigated. Besides, of course, comparing heterogeneous and homogeneous PIMs. The heterogeneous PIMs were studied by da Silva et al. [74], where islands apply different EAs to solve a single objective. More recently, PIMs were enriched with the capability of algorithmic reconfiguration. Such reconfigurable heterogeneous PIMs can dynamically update their algorithms according to their adaptation to the target problems [75]. Moreover, such heterogeneity provides models that could efficiently address distinct objective functions on each island. Indeed, an interesting potential application of such heterogeneous PIMs is the treatment of multi-objective problems. For such applications, each PIM island may take care of the optimization of a different objective function (as in [64–67]).

Table 34
The average scores of N -Queens with asynchronous static PIMs (Data in Fig. 15).

N	G_{AS}	P_{N12A}	P_{N24A}	P_{T012A}	P_{T024A}	P_{T112A}	P_{T124A}	P_{R12A}	P_{R24A}	P_{C12A}	P_{C24A}
50	18.3	11.7	10.6	11.0	11.0	12.5	10.8	11.3	11.2	12.0	10.6
55	20.7	11.7	11.7	12.2	12.2	13.0	11.5	11.6	11.2	12.3	11.7
60	22.2	12.6	12.6	13.7	12.5	12.7	12.4	12.8	12.9	12.5	11.9
65	24.9	13.4	12.9	13.7	13.3	14.5	13.7	14.3	13.1	13.8	12.8
70	25.1	14.0	14.0	14.5	14.4	16.1	14.0	14.8	14.2	14.7	14.1
75	28.8	15.4	14.3	15.1	15.4	16.2	15.2	15.4	15.3	15.5	15.6
80	27.1	16.1	15.5	15.5	16.1	16.7	15.5	16.3	16.1	17.0	15.5
85	31.7	17.5	16.8	17.1	16.6	18.0	16.9	17.0	17.0	17.2	16.8
90	29.0	17.5	17.8	17.3	17.5	18.7	17.1	17.9	17.2	17.9	16.9
95	29.6	19.0	18.4	17.9	18.7	19.3	18.2	18.9	18.2	19.4	17.7
100	33.3	18.7	19.1	20.2	19.7	21.0	18.6	19.3	18.8	18.6	19.2
105	31.6	20.2	20.5	19.9	20.2	21.4	19.6	21.0	20.6	20.6	19.3
110	35.2	20.0	21.3	20.2	21.2	22.4	20.6	22.0	21.0	20.7	20.6
115	42.5	20.7	21.2	21.0	22.4	23.2	21.8	22.4	21.1	22.3	21.2
120	35.0	22.4	23.4	22.4	23.2	23.5	22.4	23.4	23.5	22.2	23.0
125	35.2	22.5	24.3	22.9	23.3	23.6	22.9	23.8	23.5	23.8	22.9
130	38.9	24.1	24.5	24.0	24.6	25.6	23.9	24.7	24.7	24.0	24.8
135	39.6	25.1	25.3	24.9	24.7	26.1	24.2	25.3	25.6	25.4	24.5
140	39.5	25.4	26.0	25.7	26.5	27.4	26.3	24.7	25.9	26.6	25.9
145	39.2	26.6	26.6	27.2	27.0	28.4	26.6	26.8	27.1	27.3	26.5
150	41.1	27.6	27.0	27.5	28.1	27.8	27.4	26.9	28.0	27.2	27.2
155	45.8	27.9	28.6	29.4	28.8	28.8	28.0	28.0	29.6	27.9	27.3
160	46.3	28.8	30.1	29.0	30.0	30.3	29.8	29.8	29.8	29.2	29.7
165	44.1	30.0	30.9	31.0	29.4	31.0	31.0	30.6	31.2	30.7	29.6
170	44.5	30.8	31.3	31.3	32.4	32.0	31.6	31.6	32.5	32.0	30.8
175	44.7	31.7	31.7	33.2	31.8	33.5	33.0	32.7	32.0	32.8	31.8
180	50.6	33.7	32.4	33.6	34.6	33.1	33.3	32.8	34.5	33.4	34.0
185	47.5	34.4	33.8	33.9	33.7	34.6	34.7	33.0	35.8	34.8	34.1
190	50.2	34.7	35.6	34.8	36.6	35.8	35.7	35.0	34.9	36.2	34.6
195	55.5	35.6	35.7	35.5	37.0	36.7	36.0	36.5	36.8	36.3	35.3
200	52.4	37.2	37.9	36.3	37.4	37.9	38.2	37.4	38.2	37.3	37.0

Table 35
The average scores of N -Queens with asynchronous dynamic PIMs (Data in Fig. 15).

N	G_{AS}	P_{Rd12A}	P_{Rd24A}	$P_{\approx 12A}$	$P_{\approx 24A}$	$P_{gbmm12A}$	$P_{gbmm24A}$
50	18.3	10.5	11.0	10.7	10.9	11.6	11.8
55	20.7	11.7	11.8	11.6	11.8	12.0	12.3
60	22.2	13.5	12.5	12.9	12.7	12.4	11.8
65	24.9	13.5	13.7	14.1	13.9	14.2	13.7
70	25.1	13.9	13.6	13.6	14.7	15.1	14.6
75	28.8	15.3	14.9	15.3	14.7	15.5	15.6
80	27.1	16.0	15.8	15.9	15.2	16.3	15.7
85	31.7	16.5	17.2	16.6	17.3	17.3	17.0
90	29.0	17.4	17.3	17.3	17.0	17.7	17.3
95	29.6	18.3	18.3	18.6	18.6	18.2	18.0
100	33.3	19.0	18.7	19.0	18.4	19.3	19.7
105	31.6	20.3	19.6	19.6	19.9	19.8	20.1
110	35.2	21.0	20.4	20.6	21.1	20.4	21.0
115	42.5	21.9	22.0	21.0	20.5	21.1	21.8
120	35.0	21.7	21.4	23.3	22.6	22.4	23.5
125	35.2	22.5	23.0	22.1	23.4	23.0	23.8
130	38.9	23.9	24.4	23.9	24.8	24.0	24.3
135	39.6	24.5	25.2	24.6	24.7	25.4	26.0
140	39.5	25.7	24.8	25.6	25.7	25.6	26.5
145	39.2	26.4	26.5	26.1	27.3	26.9	28.0
150	41.1	26.6	27.8	26.8	27.0	27.2	28.5
155	45.8	28.3	28.4	27.7	27.7	28.7	29.0
160	46.3	29.7	29.9	28.3	29.4	28.9	30.3
165	44.1	30.4	29.9	28.7	30.5	30.8	30.4
170	44.5	31.1	30.8	30.6	31.2	30.8	32.1
175	44.7	31.6	32.2	32.4	32.6	32.1	33.1
180	50.6	32.5	32.6	32.8	32.9	33.1	34.0
185	47.5	33.8	33.7	32.5	33.5	33.8	34.4
190	50.2	34.4	34.9	33.5	35.7	34.9	36.4
195	55.5	36.2	36.2	35.1	37.0	35.5	37.4
200	52.4	36.4	35.5	37.0	36.7	36.0	39.1

CCRediT authorship contribution statement

Lucas A. da Silveira: Programming, Statistical analysis, Data curation, Writing – original draft, Writing – review & editing. Thaynara A. de Lima: Conceptualization, Methodology, Writing – original draft,

Writing – review & editing. Jessé B. de Barros: Programming, Data curation. José L. Soncco-Álvarez: Conceptualization, Statistical analysis. Carlos H. Llanos: Conceptualization, Rewriting, Formal analysis, Editing. Mauricio Ayala-Rincón: Conceptualization, Supervision, Methodology, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix A. Supplementary information for Section 2: Cases of study

A.1. Reversal distance problem

In bioinformatics, a unichromosomal genome with n genes can be represented as a permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, where $\pi(i) = \pi_i$, $1 \leq i \leq n$ and $\pi_i \in \{1, \dots, n\}$. A reversal $\rho^{j,k}$, for $1 \leq j \leq k \leq n$ is an operation on π that inverts the sub string between π_j and π_k transforming π into $\pi' = (\dots, \pi_{j-1}, \pi_k, \dots, \pi_j, \pi_{k+1}, \dots)$; for example: for $\pi = (1, 4, 3, 5, 8, 2, 7, 6)$, $\rho^{5,7}(\pi)$ gives $\pi' = (1, 4, 3, 5, 7, 2, 8, 6)$. The reversal distance problem consists in finding a shortest sequence of reversals $\rho_1, \dots, \rho_l$ needed to transform a permutation π into another permutation σ . There are two different types of permutations: signed and unsigned. In the unsigned case, genes are abstracted without any orientation. At the same time, in the signed one, each π_i has a positive or negative sign reflecting the orientation of the gene within the genome. A reversal acts over a signed permutation by also inverting the genes' orientation

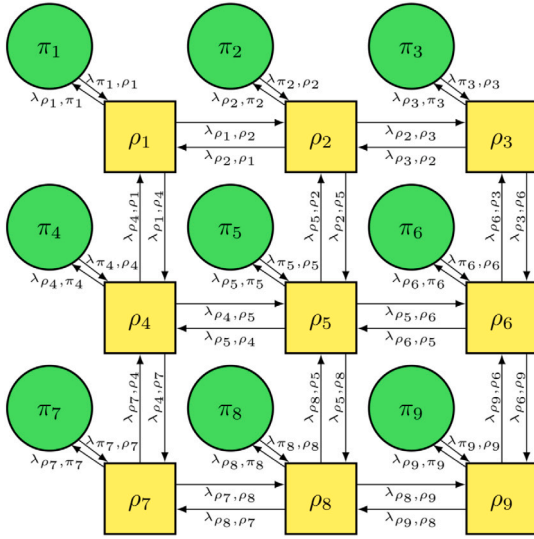


Fig. 18. 3 × 3 mesh-grid RTNoC platform. Circles and squares represent, resp., processors and routers.

in a specific substring. By simple algebraic properties of permutations, the reversal distance problem is equivalent to transform an arbitrary permutation π into the identity permutation ι , by a minimum number of reversals, where ι is the permutation $(1, 2, \dots, n)$, for the unsigned case, and $(+1, +2, \dots, +n)$, for the signed case. Such a minimum is called the *reversal distance* of π , and the *sorting by reversals problem* consists in determining the reversal distance of a permutation [18]. The signed version of this problem was proved to be in $\mathcal{P}$, namely, an $O(n^2)$ algorithm was first given by Hannenhalli and Pevzner [19], and further, a linear one was proposed by Bader et al. [20], while the unsigned one was proved to be $\mathcal{NP}$ -hard by Caprara [21]. The interesting case study considered in this work is the *Unsigned Reversal Distance* (URD) problem, where permutations have no orientation. The signed case, for short SRD, has been extensively studied in the field of combinatorics and algorithmics of permutations (e.g., [22,23]). In evolutionary and genetic approaches, the fitness function to solve URD is computed applying SRD solution approaches as the one proposed by Bader et al. [20] (it is the one used in our procedures) as described in [24].

A.2. Translocation distance problem

The *Translocation Distance problem* consists in computing the length of a minimal sequence of operations called *translocations* that transform one multi-chromosome genome into another. As for the reversal distance problem, signed and unsigned genomes are considered. An unsigned genome consists of a set of chromosomes $\{X_1, \dots, X_t\}$, where each X_v , for $1 \leq v \leq t$, is a gene sequence such that each gene x_i is represented by a different natural number appearing only once in the genome. Consider two chromosomes $X = (x_1, x_2, \dots, x_k)$ and $Y = (y_1, y_2, \dots, y_m)$ of an unsigned genome. There are two kinds of translocations:

- A *prefix-prefix* translocation $\rho_{pp}(X, Y, i, j)$, $1 < i \leq k$, $1 < j \leq m$ that transforms the chromosomes X and Y into two new chromosomes $(x_1, \dots, x_{i-1}, y_j, \dots, y_m)$ and $(y_1, \dots, y_{j-1}, x_i, \dots, x_k)$.
- A *prefix-suffix* translocation $\rho_{ps}(X, Y, i, j)$, $1 < i \leq k$, $1 < j \leq m$ that transforms the chromosomes X and Y into two new chromosomes $(x_1, \dots, x_{i-1}, y_{j-1}, \dots, y_1)$ and $(y_m, \dots, y_j, x_i, \dots, x_k)$.

Formally, the *Unsigned Translocation Distance* (UTD) problem consists in finding the minimum number of translocations for transforming

an unsigned genome A into another genome B , where A and B have the same (number of) genes n , and number of chromosomes N . Suppose the genes in two genomes A and B with an equal number of chromosomes are the same, then, without loss of generality, the genes of A and B can be renamed so that B can be rewritten as an identity. For the experiments, it is assumed that genome B is an identity genome, that is, a genome with all its genes sorted in increasing order; for instance, $A = \{(1, 3, 7)(5, 2, 6, 4)\}$ and $B = \{(1, 2, 3, 4)(5, 6, 7)\}$. The UTD problem is addressed in this paper and was shown to be $\mathcal{NP}$ -hard by Zhu and Wang [25]. As for SRD, the Signed Translocation Distance problem (STD) considers signed genomes. Let n be the number of genes in the genome; Hannenhalli [26] provided the first polynomial-time algorithm with minimum translocation distance in $O(n^3)$ and the best known algorithm, proposed by Bergeron et al. [27], computes the minimum translocation distance in $O(n)$. In genetic and evolutionary approaches, the fitness function to solve UTD problems is computed by an STD solution as the one proposed by Bergeron et al. [27] that is the solution applied in our approach (e.g., [28]).

A.3. Task mapping and scheduling

The *Task Scheduling* or *Task Mapping Problem* (TMP) consists of mapping tasks of a *Real-Time Application* (RTA) onto one of the multiple processor cores of a *Real-Time System* (RTS). The correctness of an RTS depends on its compliance with deadlines.

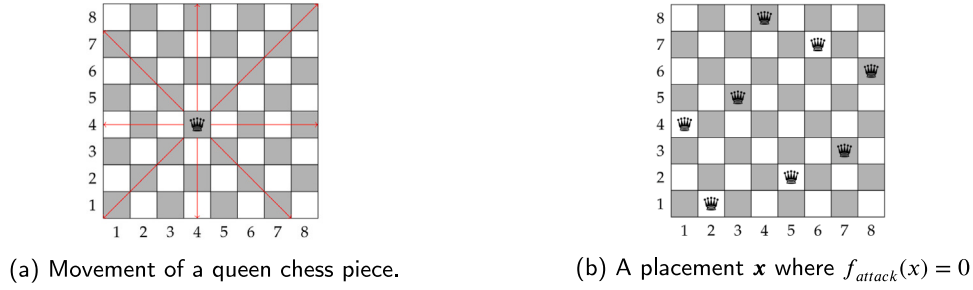
The focus of this case study is the application of PIMs for TMP for a type of multiple processor system called *Multiprocessor System-on-Chip* (MPSoC) as presented by Wolf et al. [29]. The MPSoC used in this case study has a communication architecture that consists of a *Network-on-a-Chip*, a well-established intra-chip communication architecture paradigm as presented in [30]. Due to the real-time setting, the *Network-on-a-Chip* fits the category of a *Real-Time Network-on-a-Chip* (RTNoC) as defined by Hesham et al. [31]. Such an RTNoC consists of a wormhole packet switching strategy and virtual channels, allowing the preemption of data flows during transmission by ones with a higher priority level.

For static schedulability analysis of feasible mapping solutions, this case study uses a specific model for both the RTNoC-based MPSoC platform and the RTA mapped onto it that is similar to the one present by Indrusiak [32]. The platform model is a triple $\Psi = \langle \Pi, R, A \rangle$, where Π is a set of homogeneous processing elements connected to its network interface elements, $\Pi = \{\pi_1, \dots, \pi_n\}$; R is a set of routing elements to switch messages $R = \{\rho_1, \rho_2, \dots, \rho_n\}$ that connect elements in Π and themselves using unidirectional links, $A = \{\lambda_{\pi_1, \rho_1}, \lambda_{\rho_1, \pi_1}, \lambda_{\rho_1, \rho_2}, \dots, \lambda_{\rho_n, \rho_{n-1}}\}$, in a regular mesh-grid topology. Fig. 18, adapted from [32], illustrates an NoC-based platform with its components interconnected in a mesh-grid fashion.

The application model comprises a set of tasks Γ , such that $\tau \in \Gamma$ is given as a tuple $\langle C, T, D, P, \phi \rangle$ formed by the worst-case running time, inter-arrival period, relative deadline time, priority-level, and a transmitted message ϕ , respectively. At the end of its execution, the task τ may transmit a message ϕ towards another task. A message ϕ is represented as a tuple $\langle \tau_d, L, Z \rangle$, where τ_d is the destination task, L the basic latency of message transmission, and Z the message size in bits. For a message, ϕ_i , the basic latency L_i is calculated using Eq. (1). It depends upon the mapping of the source task τ_i that defines the route of links in which this message will travel across the platform. This route of links is a set $route(\phi_i)$. The basic latency calculation also depends on specific parameters such as routers and links maximum latencies per flit, L_ρ and L_λ , respectively.

$$L_i = |route(\phi_i)| \cdot (L_\lambda + L_\rho) - L_\rho + Z_i L_\rho \quad (1)$$

The schedulability analysis considers the utilization factor of processor cores and links. The utilization factor for a processor core, U_π , presents the ratio in which the processor is used by the tasks that are mapped

Fig. 19. Illustration of queen chess pieces interaction on an 8×8 board.

onto it and is expressed by Eq. (2) (according to Liu and Layland [34]), where $map^{-1}(\pi)$ is the set of tasks mapped to processor π .

The same principle is used to evaluate the utilization rate of links in the RTNoC to check whether they do not exceed their maximum communication bandwidth. The link utilization is represented by U_λ , and is calculated by Eq. (3), where $map^{-1}(\lambda)$ is the set of messages that are transmitted over the link λ .

$$U_\pi = \sum_{\tau_i \in map^{-1}(\pi)} \frac{C_i}{T_i} \quad (2)$$

$$U_\lambda = \sum_{\phi_i \in map^{-1}(\lambda)} \frac{L_i}{T_i} \quad (3)$$

This simple analysis is necessary to avoid non-schedulable solutions that are those for which the resource demands exceed the available resource capacity, implying that tasks and messages do not comply with the timing restrictions. This holds for any given processor π if its utilization factor is such that $U_\pi > 1$ and for any given link λ such that $U_\lambda > 1$. However, the utilization analysis is not sufficient. Even in cases where all processors and links are not over-utilized, the task mapping may not be fully schedulable depending on the task set in the mapped RTA. Even though these tests are necessary but not sufficient, they are attractive as methods to quickly identify whether mappings suit the time requirements.

Given a set of tasks Γ and a platform Ψ , a task mapping solution can be encoded as an n -dimensional vector of positive integers $\mathbf{x}$, where each j th component, x_j , represents the index of the processor in the platform Ψ responsible for processing the task $\tau_j \in \Gamma$. The search-based optimization for a task mapping solution that reduces the number of overused resources is expressed in Eq. (4), where f_{util} (see Algorithm 2) is a function that calculates the number of processors and links overburdened in the system given a task placement $\mathbf{x}$ using Eqs. (2) and (3). The optimization goal using f_{util} as a fitness function is to find a task placement $\mathbf{x}^*$ that respects processors and links maximum capacities.

$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}} f_{util}(\mathbf{x}) \quad (4)$$

The considered TMP case study is $\mathcal{NP}$ -complete similar to the knapsack problem as given by Garey and Johnson [33]; thus, a brute force approach is unfeasible. Indeed, a case where n tasks are being mapped onto a system with m processors gives a search-space with m^n possible task mapping solutions. The TMP study in multiple processors RTSs is fundamental to the embedded devices research field since their application is required in critical systems. Another aspect is that the advance of chip manufacturing allied with the increasing computing requirements of current RTSs makes it natural for embedded devices to use a large number of processors in the same chip to tackle massively parallel problems.

Algorithm 2 Fitness function f_{util}

INPUT: Tasks Placement ($\mathbf{x}$)

OUTPUT: Number of Over Capacity Elements

```

1: procedure  $f_{util}(\mathbf{x})$ 
2:   Check Task Mapping  $\mathbf{x}$ 
3:    $overcap = 0$ 
4:   for  $p = 1$  to  $|\Pi|$  do
5:     Calculate  $U_{\pi_p}$  ▷ Eq. (2)
6:     if  $U_{\pi_p} > 1$  then
7:        $overcap = overcap + 1$ 
8:     end if
9:   end for
10:  for  $l = 1$  to  $|\Lambda|$  do ▷ Eq. (3)
11:    Calculate  $U_{\lambda_l}$ 
12:    if  $U_{\lambda_l} > 1$  then
13:       $overcap = overcap + 1$ 
14:    end if
15:  end for
16:  return  $overcap$ 
17: end procedure

```

A.4. N -Queens

The N -Queens problem [35] is an expansion of the 8-Queens placement problem that aims to place n queen chess pieces in an $n \times n$ chessboard in a configuration in which no two queens can attack each other. In other words, a configuration of n points in an $n \times n$ grid that has no pairs of points in the same row, column, or diagonals. The problem is divided into two types: (a) the search of a configuration in which no queen attack each other (*witness problem*), and (b) the search for all witness solutions of a board configuration (*counting problem*). These problems are related with the N -Queens completion problem that consists in completing a configuration from a given partial solution and is known to be both $\mathcal{NP}$ -complete and $\#P$ -complete (see [35]). The case study in this work is the *witness problem* that is commonly used as a benchmark in different machine learning-based methods. For example, it has been used in constructive backtracking algorithms [36], and EA based meta-heuristics such as GA [37], and PSO [38].

The search for a solution for the N -Queens placement corresponds to a minimization problem according to Eq. (5). Considering the search space is given by n different positions in an $n \times n$ -chessboard its size is given by $(n^2)!/(n!(n^2 - n)!)$ and the measure of each possible solution is given by the number of attacking movements between queens at these positions. Here, each candidate placement solution is encoded as an n -dimensional vector $\mathbf{x}$, where the pair (j, x_j) , for j an index of $\mathbf{x}$, represent the placement of a queen onto the j th row and x_j th column of the chessboard. Notice that in this form the search space is reduced

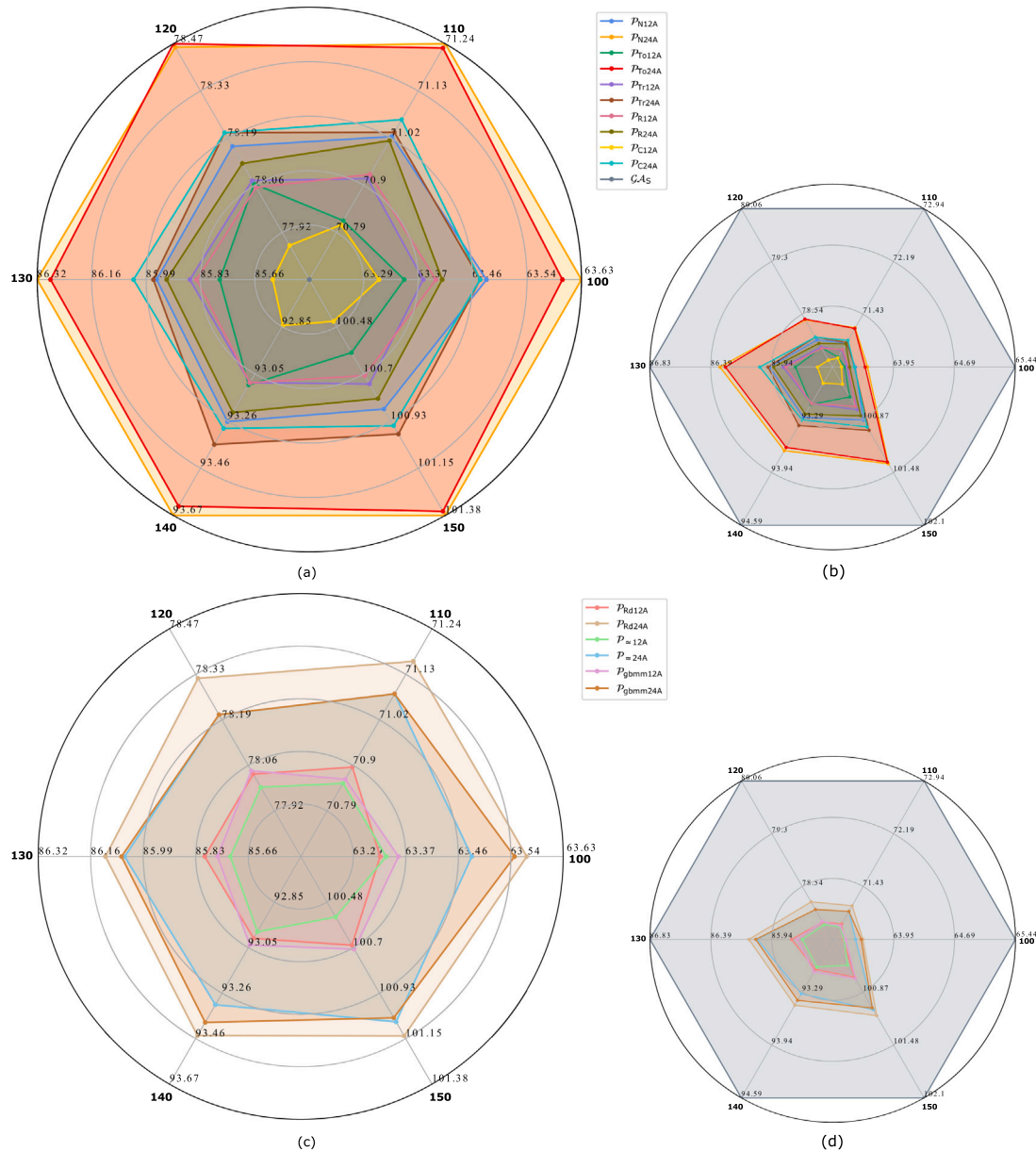


Fig. 20. The average scores of UTD with three chromosomes for asynchronous: (a) static PIMs; (b) static PIMs versus sequential G_A s; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential G_A s.

to a set S of n^n possible placements. The goal is to minimize a function $f_{attack} : S \rightarrow \mathbb{N}$ that maps each $x \in S$ into the number of queen pieces under attacking movements given by a placement x . If $f_{attack}(x) = 0$ then x is a solution that satisfies the N -Queens placement problem. Fig. 19(a) illustrates the placement of a single queen piece at (4, 4) position on an 8×8 board and Fig. 19(b) shows a placement where none of the 8 queen pieces attack each other.

$$x^* = \underset{x}{\operatorname{argmin}} f_{attack}(x) \quad (5)$$

The function f_{attack} is linearly computed according to Algorithm (pseudo-code) 3, where given an x placement, it counts the number of queens in each column, principal and secondary diagonals, and then accumulates the number of attacks in each column and principal and secondary diagonals.

Appendix B. Supplementary information for Section 5: Results of experiments

B.1. Parameter setup

Tables 13, 15, 17 and 19, and Tables 14, 16, 18 and 20, provide the best values obtained for each parameter in synchronous and asynchronous PIMs respectively. Table 21 includes the breeding cycle parameters calibrated for each case study.

B.2. Accuracy obtained for URD

The average (A) results and standard deviation (SD) for each package of permutations of each length (L) are shown in Tables 22 and 23 for synchronous PIMs and in Tables 24 and 25 for asynchronous PIMs. Considering the accuracy of the results, the best synchronous PIMs with

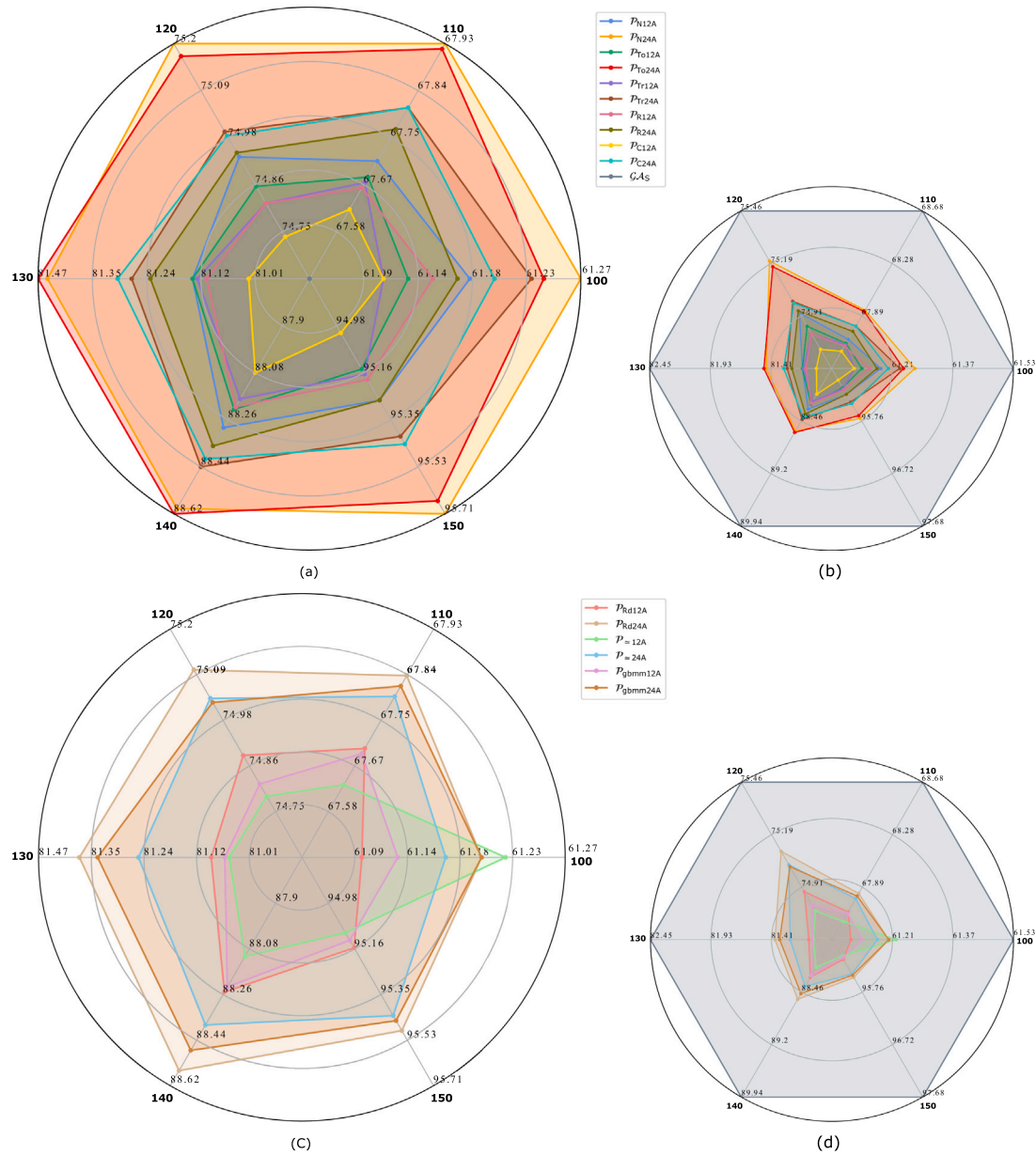


Fig. 21. The average scores of UTD with four chromosomes for asynchronous: (a) static PIMs; (b) static PIMs versus sequential $\mathcal{GA}_s$; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential $\mathcal{GA}_s$.

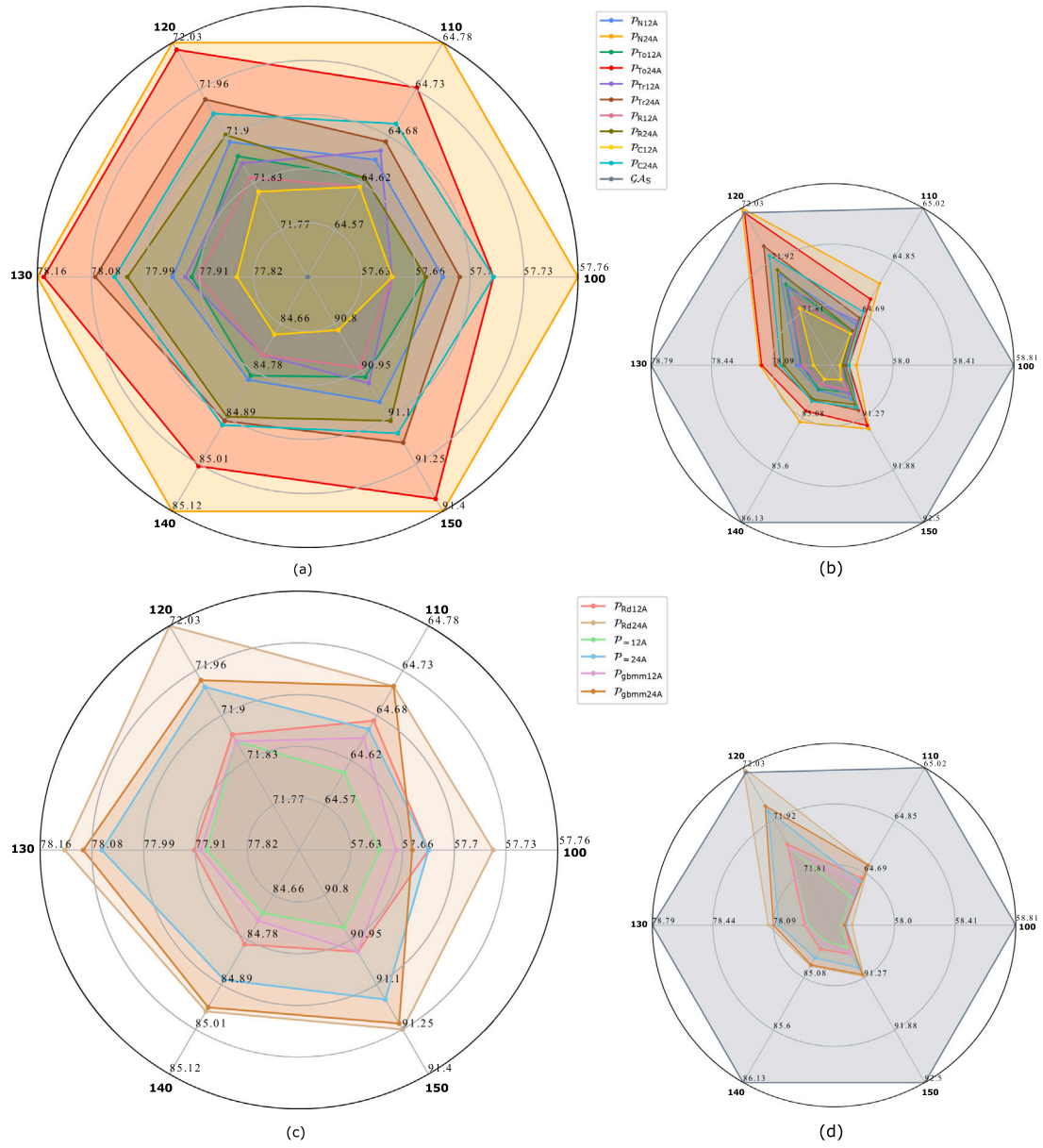


Fig. 22. The average scores of UTD with five chromosomes for asynchronous: (a) static PIMs; (b) static PIMs versus sequential $\mathcal{GA}_S$; (c) dynamic PIMs; and, (d) dynamic PIMs versus sequential $\mathcal{GA}_S$.

Algorithm 3 *N*-Queens Attack Counter

```

INPUT:    Placement ( $x$  with  $|x| = n$ )
OUTPUT:  Number of Attacks (attacks)
1: procedure NQueensAttacks( $x$ )
2:   queens-col[1.. $n$ ], queens-dp[1 -  $n..n - 1$ ], queens-ds[2.. $2n$ ]
3:   attacks = 0
4:   for  $j = 1$  to  $n$  do
5:     queens-col[ $x_j$ ]++
6:     queens-dp[ $x_j - j$ ]++
7:     queens-ds[ $x_j + j$ ]++
8:   end for
9:   for  $j = 1$  to  $n$  do
10:    if queens-col[ $j$ ]  $\geq 2$  then
11:      attacks += queens-col[ $j$ ] (queens-col[ $j$ ] - 1)/2
12:    end if
13:  end for
14:  for  $j = 2$  to  $2n$  do
15:    if queens-ds[ $j$ ]  $\geq 2$  then
16:      attacks += queens-ds[ $j$ ] (queens-ds[ $j$ ] - 1)/2
17:    end if
18:    if queens-dp[ $j - 1 - n$ ]  $\geq 2$  then
19:      attacks += queens-dp[ $j - 1 - n$ ] (queens-dp[ $j - 1 - n$ ] - 1)/2
20:    end if
21:  end for
22: return attacks
23: end procedure

```

static and dynamic topologies are respectively P_{Tr12S} and $P_{gbmm12S}$. Looking at asynchronous PIMs, we have P_{R24A} as the best static and $P_{gbmm12A}$ the best dynamic. The best absolute results are highlighted in blue and are given by the synchronous models.

B.3. Accuracy obtained for UTD

Tables 26 and 27 present the average translocation distance and the standard deviation for the synchronous and asynchronous PIMs. The best results are highlighted for synchronous and asynchronous PIMs, coloring in blue the best absolute results.

Figs. 20–22 compare accuracies of asynchronous PIMs. For easing comparison, the radii's ranges and scales for synchronous (Figs. 8–10 in the paper) and asynchronous PIMs dealing with inputs with the same number of chromosomes are the same.

B.4. Accuracy obtained for TMP

Accuracy of results for synchronous static PIMs are shown in Tables 28 and 29. The best results for static and dynamic topologies are provided by P_{N24S} and $P_{gbmm24S}$. According to Tables 30 and 31, P_{To24A} was the best asynchronous static PIM and P_{Rd24A} the best dynamic PIM. The best absolute results, highlighted in blue, were with two exceptions provided by the synchronous dynamic model $P_{gbmm24S}$.

B.5. Accuracy obtained for *N*-queens

Tables 32–35 present the results for synchronous and asynchronous PIMs, respectively. It is worth noting that for these experiments, it was not possible to calculate the standard deviation since a single entry was used for each length. The P_{Tr24S} was the best synchronous static PIM, while P_{s24S} and $P_{gbmm24S}$ were the ones that provided the best solutions among the synchronous dynamic models. In experiments involving asynchronous PIMs, P_{C24A} and P_{s12A} were the best static and dynamic, respectively.

References

- [1] E. Cantú-Paz, Migration policies, selection pressure, and parallel evolutionary algorithms, *J. Heuristics* 7 (4) (2001) 311–334, <http://dx.doi.org/10.1023/A:1011375326814>.
- [2] D. Sudholt, *Springer Handbook of Computational Intelligence*, Springer, 2015, pp. 929–959, http://dx.doi.org/10.1007/978-3-662-43505-2_46.
- [3] A. Abdelhazef, E. Alba, G. Luque, Performance analysis of synchronous and asynchronous distributed genetic algorithms on multiprocessors, *Swarm Evol. Comput.* 49 (2019) 147–157, <http://dx.doi.org/10.1016/j.swevo.2019.06.003>.
- [4] E. Cantú-Paz, A survey of parallel genetic algorithms, *Calculateurs Paralleles, Reseaux et Systems Repartis* 10 (2) (1998) 141–171.
- [5] A. Hussain, Y.S. Muhammad, M. Nauman Sajid, I. Hussain, A. Mohamd Shoukry, S. Gani, Genetic algorithm for traveling salesman problem with modified cycle crossover operator, *Comput. Intell. Neurosci.* 2017, Article ID 7430125 (2017) <http://dx.doi.org/10.1155/2017/7430125>.
- [6] S. Katoch, S.S. Chauhan, V. Kumar, A review on genetic algorithm: past, present, and future, *Multimedia Tools Appl.* 80 (5) (2021) 8091–8126, <http://dx.doi.org/10.1007/s11042-020-10139-6>.
- [7] G.R. Duarte, A.C.d.C. Lemonge, L.G. da Fonseca, A dynamic migration policy to the island model, in: 2017 IEEE Congress on Evolutionary Computation, CEC, 2017, pp. 1135–1142, <http://dx.doi.org/10.1109/CEC.2017.7969434>.
- [8] G.R. Duarte, A.C.d.C. Lemonge, L.G. da Fonseca, B.S.L.P. de Lima, An island model based on stigmergy to solve optimization problems, *Nat. Comput.* 20 (2020) 1–29, <http://dx.doi.org/10.1007/s11047-020-09819-x>.
- [9] L. Federici, B. Benedikter, A. Zavoli, EOS: A parallel, self-adaptive, multi-population evolutionary algorithm for constrained global optimization, in: 2020 IEEE Congress on Evolutionary Computation, CEC, IEEE, 2020, pp. 1–10, <http://dx.doi.org/10.1109/CEC48606.2020.9185800>.
- [10] E. Alba, J.M. Troya, Analyzing synchronous and asynchronous parallel distributed genetic algorithms, *Future Gener. Comput. Syst.* 17 (4) (2001) 451–465, [http://dx.doi.org/10.1016/S0167-739X\(99\)00129-6](http://dx.doi.org/10.1016/S0167-739X(99)00129-6).
- [11] F. Fernández, G. Galeano, J.A. Gómez, Comparing synchronous and asynchronous parallel and distributed genetic programming models, in: 5th European Conference on Genetic Programming, Springer, 2002, pp. 326–335, http://dx.doi.org/10.1007/3-540-45984-7_32.
- [12] D. Izzo, M. Rucinski, C. Ampatzis, Parallel global optimisation meta-heuristics using an asynchronous island-model, in: 2009 IEEE Congress on Evolutionary Computation, CEC, 2009, pp. 2301–2308, <http://dx.doi.org/10.1109/CEC.2009.4983227>.
- [13] P.B. Grosso, *Computer Simulations of Genetic Adaptation: Parallel Subcomponent Interaction in a Multilocus Model* (Ph.D. thesis), University of Michigan, Ann Arbor, MI, USA, 1985, AA18520908.
- [14] T. Starkweather, D. Whitley, K. Mathias, Optimization using distributed genetic algorithms, in: *Parallel Problem Solving from Nature: 1st Workshop*, Springer Berlin Heidelberg, Berlin, Heidelberg, 1991, pp. 176–185, <http://dx.doi.org/10.1007/BFb0029750>.
- [15] Z. Skolicki, K. De Jong, The influence of migration sizes and intervals on island models, in: *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation, GECCO, ACM*, 2005, pp. 1295–1302, <http://dx.doi.org/10.1145/1068009.1068219>.
- [16] A. Lissouvo, C. Witt, A runtime analysis of parallel evolutionary algorithms in dynamic optimization, *Algorithmica* 78 (2) (2017) 641–659, <http://dx.doi.org/10.1007/s00453-016-0262-4>.
- [17] A. Eiben, S. Smit, Parameter tuning for configuring and analyzing evolutionary algorithms, *Swarm Evol. Comput.* 1 (1) (2011) 19–31, <http://dx.doi.org/10.1016/j.swevo.2011.02.001>.
- [18] V. Bafna, P.A. Pevzner, Genome rearrangements and sorting by reversals, in: *Proceedings of 1993 IEEE 34th Annual Foundations of Computer Science*, 1993, pp. 148–157, <http://dx.doi.org/10.1109/SFCS.1993.366872>.
- [19] S. Hannenhalli, P. Pevzner, Transforming cabbage into turnip: Polynomial algorithm for sorting signed permutations by reversals, in: *Proceedings of the 27th Annual ACM Symposium on Theory of Computing*, STOC, ACM, 1995, pp. 178–189, <http://dx.doi.org/10.1145/300515.300516>.
- [20] D.A. Bader, B.M.E. Moret, M. Yan, A linear-time algorithm for computing inversion distance between signed permutations with an experimental study, *J. Comput. Biol.* 8 (5) (2001) 483–491, <http://dx.doi.org/10.1089/106652701753216503>.
- [21] A. Caprara, Sorting by reversals is difficult, in: *Proceedings of the First Annual International Conference on Computational Molecular Biology*, Association for Computing Machinery, New York, NY, USA, 1997, pp. 75–83, <http://dx.doi.org/10.1145/267521.267531>.
- [22] S. Grusea, A. Labarre, The distribution of cycles in breakpoint graphs of signed permutations, *Discrete Appl. Math.* 161 (10) (2013) 1448–1466, <http://dx.doi.org/10.1016/j.dam.2013.02.002>.
- [23] T.A. de Lima, M. Ayala-Rincón, On the average number of reversals needed to sort signed permutations, *Discrete Appl. Math.* 235 (Supplement C) (2018) 59–80, <http://dx.doi.org/10.1016/j.dam.2017.09.014>.

- [24] J.L. Soncco-Álvarez, D.M. Muñoz, M. Ayala-Rincón, Opposition-based memetic algorithm and hybrid approach for sorting permutations by reversals, *Evol. Comput.* 27 (2) (2019) 229–265, http://dx.doi.org/10.1162/evco_a_00220.
- [25] D. Zhu, L. Wang, On the complexity of unsigned translocation distance, *Theoret. Comput. Sci.* 352 (1) (2006) 322–328, <http://dx.doi.org/10.1016/j.tcs.2005.09.078>.
- [26] S. Hannenhalli, Polynomial-time algorithm for computing translocation distance between genomes, *Discrete Appl. Math.* 71 (1) (1996) 137–151, [http://dx.doi.org/10.1016/S0166-218X\(96\)00061-3](http://dx.doi.org/10.1016/S0166-218X(96)00061-3).
- [27] A. Bergeron, J. Mixtacki, J. Stoye, On sorting by translocations, *J. Comput. Biol.* 13 (2) (2006) 567–578, <http://dx.doi.org/10.1089/cmb.2006.13.567>.
- [28] L.A. da Silva, J.L. Soncco-Álvarez, T.A. de Lima, M. Ayala-Rincón, Parallel Island model genetic algorithms applied in NP-Hard problems, in: 2019 IEEE Congress on Evolutionary Computation, CEC, 2019, pp. 1–8, <http://dx.doi.org/10.1109/CEC.2019.8790257>.
- [29] W. Wolf, A. Jerraya, G. Martin, Multiprocessor System-on-Chip (MPSoC) technology, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 27 (10) (2008) 1701–1713, <http://dx.doi.org/10.1109/TCAD.2008.923415>.
- [30] G. de Micheli, L. Benini, Networks on chips: 15 years later, *Computer* 50 (5) (2017) 10–11, <http://dx.doi.org/10.1109/MC.2017.140>.
- [31] S. Hesham, J. Rettkowski, D. Goehring, M.A. Abd El Ghany, Survey on real-time networks-on-chip, *IEEE Trans. Parallel Distrib. Syst.* 28 (5) (2017) 1500–1517, <http://dx.doi.org/10.1109/TPDS.2016.2623619>.
- [32] L.S. Indrusiak, End-to-end schedulability tests for multiprocessor embedded systems based on networks-on-chip with priority-preemptive arbitration, *J. Syst. Archit.* 60 (7) (2014) 553–561, <http://dx.doi.org/10.1016/j.sysarc.2014.05.002>.
- [33] M.R. Garey, D.S. Johnson, Computers and intractability. A guide to the theory of Np-completeness, *J. Symbolic Logic* 48 (2) (1983) 498–500, <http://dx.doi.org/10.2307/2273574>.
- [34] C.L. Liu, J.W. Layland, Scheduling algorithms for multiprogramming in a hard-real-time environment, *J. ACM* 20 (1) (1973) 46–61, <http://dx.doi.org/10.1145/321738.321743>.
- [35] I.P. Gent, C. Jefferson, P. Nightingale, Complexity of n -queens completion, *J. Artificial Intelligence Res.* 59 (2017) 815–848, <http://dx.doi.org/10.1613/jair.5512>.
- [36] H.S. Stone, J.M. Stone, Efficient search techniques—an empirical study of the N-queens problem, *IBM J. Res. Dev.* 31 (4) (1987) 464–474, <http://dx.doi.org/10.1147/rd.314.0464>.
- [37] M. Lazarova, Efficiency of parallel genetic algorithm for solving N-queens problem on multicomputer platform, in: *The 9th WSEAS International Conference on Evolutionary Computing*, 2008, pp. 51–56.
- [38] X. Hu, R.C. Eberhart, Y. Shi, Swarm intelligence for permutation optimization: A case study of n -queens problem, in: *Proceedings of the 2003 IEEE Swarm Intelligence Symposium. SIS'03 (Cat. No.03EX706)*, 2003, pp. 243–246, <http://dx.doi.org/10.1109/SIS.2003.1202275>.
- [39] D. Whitley, S. Rana, R.B. Heckendorn, The island model genetic algorithm: On separability, population size and convergence, *J. Comput. Inf. Technol.* 7 (1) (1999) 33–47.
- [40] L.A. da Silva, J.L. Soncco-Álvarez, M. Ayala-Rincón, Parallel genetic algorithms with sharing of individuals for sorting unsigned genomes by reversals, in: 2017 IEEE Congress on Evolutionary Computation, CEC, 2017, pp. 741–748, <http://dx.doi.org/10.1109/CEC.2017.7969384>.
- [41] A. Aleti, I. Moser, A systematic literature review of adaptive parameter control methods for evolutionary algorithms, *ACM Comput. Surv.* 49 (3) (2016) 1–35, <http://dx.doi.org/10.1145/2996355>.
- [42] L.A. da Silva, J.L. Soncco-Álvarez, M. Ayala-Rincón, Parallel memetic genetic algorithms for sorting unsigned genomes by translocations, in: 2016 IEEE Congress on Evolutionary Computation, CEC, 2016, pp. 185–192, <http://dx.doi.org/10.1109/CEC.2016.7743794>.
- [43] L.A. da Silva, J.L. Soncco-Álvarez, T.A. de Lima, M. Ayala-Rincón, Parallel multi-island genetic algorithm for sorting unsigned genomes by reversals, in: 2018 IEEE Congress on Evolutionary Computation, CEC, 2018, pp. 1–8, <http://dx.doi.org/10.1109/CEC.2018.8477968>.
- [44] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30, URL <http://jmlr.org/papers/v7/demsar06a.html>.
- [45] J. Carrasco, S. García, M.M. Rueda, S. Das, F. Herrera, Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review, *Swarm Evol. Comput.* 54 (2020) 1–20, <http://dx.doi.org/10.1016/j.swevo.2020.100665>.
- [46] S. García, F. Herrera, An extension on statistical comparisons of classifiers over multiple data sets for all pairwise comparisons, *J. Mach. Learn. Res.* 9 (2008) 2677–2694, URL <http://jmlr.org/papers/v9/garcia08a.html>.
- [47] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of non-parametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, *Swarm Evol. Comput.* 1 (1) (2011) 3–18, <http://dx.doi.org/10.1016/j.swevo.2011.02.002>.
- [48] J. Derrac, S. García, S. Hui, P.N. Suganthan, F. Herrera, Analyzing convergence performance of evolutionary algorithms: A statistical approach, *Inform. Sci.* 289 (2014) 41–58, <http://dx.doi.org/10.1016/j.ins.2014.06.009>.
- [49] Y. Saito, M. Sato, M. Midtlyng, M. Miyakawa, Parallel and distributed MOEA/D with exclusively evaluated mating and migration, in: 2020 IEEE Congress on Evolutionary Computation, CEC, IEEE, 2020, pp. 1–8, <http://dx.doi.org/10.1109/CEC48606.2020.9185559>.
- [50] N. Otaka, Y. Fukuyama, Y. Kawamura, K. Murakami, A. Santana, Refrigerated showcase fault detection by a pasting based artificial neural networks using parallel multi-population modified brain storm optimization and coreentropy, in: 2020 IEEE Congress on Evolutionary Computation, CEC, IEEE, 2020, pp. 1–8, <http://dx.doi.org/10.1109/CEC48606.2020.9185511>.
- [51] W. Liu, P.P. Pokharel, J.C. Principe, Coreentropy: A localized similarity measure, in: *The 2006 IEEE International Joint Conference on Neural Network Proceedings*, 2006, pp. 4919–4924, <http://dx.doi.org/10.1109/IJCNN.2006.247192>.
- [52] R. Ohira, M.S. Islam, H. Kayesh, Speedup vs. quality: Asynchronous and cluster-based distributed adaptive genetic algorithms for ordered problems, *Parallel Comput.* 103 (2021) <http://dx.doi.org/10.1016/j.parco.2021.102755>.
- [53] L. Bergroth, H. Hakonen, T. Raita, A survey of longest common subsequence algorithms, in: *Proceedings Seventh International Symposium on String Processing and Information Retrieval, SPIRE 2000*, 2000, pp. 39–48, <http://dx.doi.org/10.1109/SPIRE.2000.878178>.
- [54] A. Skakovski, P. Jedrzejowicz, An island-based differential evolution algorithm with the multi-size populations, *Expert Syst. Appl.* 126 (2019) 308–320, <http://dx.doi.org/10.1016/j.eswa.2019.02.027>.
- [55] A. Skakovski, P. Jedrzejowicz, A multisize no migration island-based differential evolution algorithm with removal of ineffective islands, *IEEE Access* 10 (2022) 34539–34549, <http://dx.doi.org/10.1109/ACCESS.2022.3162634>.
- [56] I.A. Doush, M.A. Al-Betar, M.A. Awadallah, A.I. Hammouri, M. El-Abd, Island-based modified harmony search algorithm with neighboring heuristics methods for flow shop scheduling with blocking, in: 2020 IEEE Symposium Series on Computational Intelligence, SSCI, IEEE, 2020, pp. 976–982, <http://dx.doi.org/10.1109/SSCI47803.2020.9308556>.
- [57] X. Yang, S. Deb, Cuckoo search via Lévy flights, in: *World Congress on Nature & Biologically Inspired Computing, NaBIC, IEEE*, 2009, pp. 210–214, <http://dx.doi.org/10.1109/NaBIC.2009.5393690>.
- [58] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey Wolf Optimizer, *Adv. Eng. Softw.* 69 (2014) 46–61, <http://dx.doi.org/10.1016/j.advengsoft.2013.12.007>.
- [59] S. Mirjalili, A. Lewis, The Whale Optimization Algorithm, *Adv. Eng. Softw.* 95 (2016) 51–67, <http://dx.doi.org/10.1016/j.advengsoft.2016.01.008>.
- [60] B.H. Abed-Alguni, A.F. Klaib, K.M. Nahar, Island-based whale optimisation algorithm for continuous optimisation problems, *Int. J. Reason.-based Intel. Syst.* 11 (4) (2019) 319–329, <http://dx.doi.org/10.1504/IJRSIS.2019.103525>.
- [61] B.H. Abed-alguni, M. Barhoush, Distributed Grey Wolf Optimizer for numerical optimization problems, *Jordanian J. Comput. Inf. Technol. (JJCIT)* 04 (03) (2018) 130–149, <http://dx.doi.org/10.5455/jjit.71-1532897697>.
- [62] B.H. Abed-alguni, N.A. Alawad, Distributed Grey Wolf Optimizer for scheduling of workflow applications in cloud environments, *Appl. Soft Comput.* 102 (2017) 113 (2021) <http://dx.doi.org/10.1016/j.asoc.2021.107113>.
- [63] B.H. Abed-alguni, D. Paul, Island-based Cuckoo search with elite opposition-based learning and multiple mutation methods for solving optimization problems, *Soft Comput.* 26 (2022) 3293–3312, <http://dx.doi.org/10.1007/s00500-021-06665-6>.
- [64] Y. Zhang, D. Gong, Z. Ding, Handling multi-objective optimization problems with a multi-swarm cooperative particle swarm optimizer, *Expert Syst. Appl.* 38 (2011) 13933–13941, <http://dx.doi.org/10.1016/j.eswa.2011.04.200>.
- [65] D. Gong, B. Xu, Y. Zhang, Y. Guo, S. Yang, A similarity-based cooperative co-evolutionary algorithm for dynamic interval multiobjective optimization problems, *IEEE Trans. Evol. Comput.* 24 (1) (2020) 142–156, <http://dx.doi.org/10.1109/TEVC.2019.2912204>.
- [66] B. Xu, Y. Zhang, D. Gong, Y. Guo, M. Rong, Environment sensitivity-based cooperative co-evolutionary algorithms for dynamic multi-objective optimization, *IEEE/ACM Trans. Comput. Biol. Bioinform.* 15 (6) (2018) 1877–1890, <http://dx.doi.org/10.1109/TCBB.2017.2652453>.
- [67] R. Hashimoto, H. Ishibuchi, N. Masuyama, Y. Nojima, Analysis of evolutionary multi-tasking as an island model, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO '18, Association for Computing Machinery*, New York, NY, USA, 2018, pp. 1894–1897, <http://dx.doi.org/10.1145/3205651.3208228>.
- [68] R. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, in: *Proceedings of the Sixth International Symposium on Micro Machine and Human Science, MHS, 1995*, pp. 39–43, <http://dx.doi.org/10.1109/MHS.1995.494215>.
- [69] D. Karaboga, An Idea Based on Honey Bee Swarm for Numerical Optimization, *Technical Report*, Erciyes University, 2005.
- [70] M. Eusuff, K. Lansey, Optimization of water distribution network design using the shuffled frog leaping algorithm, *J. Water Resour. Plan. Manag.* 129 (3) (2003) 210–225, [http://dx.doi.org/10.1061/\(ASCE\)0733-9496\(2003\)129:3\(210\)](http://dx.doi.org/10.1061/(ASCE)0733-9496(2003)129:3(210)).

- [71] G. Wang, S. Deb, X. Gao, L.D.S. Coelho, A new metaheuristic optimisation algorithm motivated by elephant herding behaviour, *Int. J. Bio-Inspired Comput.* 8 (6) (2017) 394–409, <http://dx.doi.org/10.1504/IJBIC.2016.081335>.
- [72] J. James, V.O. Li, A social spider algorithm for global optimization, *Appl. Soft Comput.* 30 (2015) 614–627, <http://dx.doi.org/10.1016/j.asoc.2015.02.014>.
- [73] R. Storn, K. Price, Differential evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces, *J. Global Optim.* 11 (4) (1997) 341–359, <http://dx.doi.org/10.1023/A:1008202821328>.
- [74] L.A. da Silveira, J.L. Soncco-Álvarez, T.A. de Lima, M. Ayala-Rincón, Heterogeneous parallel island models, in: *Proceedings of the IEEE Symposium Series on Computational Intelligence, SSCI*, 2021, <http://dx.doi.org/10.1109/SSCI50451.2021.9659938>.
- [75] L.A. da Silveira, T.A. de Lima, M. Ayala-Rincón, Reconfigurable heterogeneous parallel island models, in: *Proceedings of the IEEE Symposium Series on Computational Intelligence, SSCI*, 2022, pp. 1618–1625, <http://dx.doi.org/10.1109/SSCI51031.2022.10022289>.